

GEM 510: GIS for Forestry and Conservation

Paul D. Pickell

2026-10-05

Contents

Welcome	5
How to use these resources	5
How to get involved	5
1 Introduction to PostgreSQL and PostGIS	7
Lab Overview	7
Learning Objectives	7
Deliverables	8
Data	8
Task 1: Set up pgAdmin and Connect to the UBC PostgreSQL server	8
Task 2: Explore with QGIS	12
Task 3: Explore with ArcGIS Pro	14
Task 4: Explore with psql Shell	17
Summary	21
2 Analyze a Relational Database	23
Lab Overview	23
Learning Objectives	23
Deliverables	24
Data	24
Task 1: Host your own PostgreSQL Server	24
Task 2: Create a PostGIS Database	25
Task 3: Import Spatial Data Using GDAL	32
Task 4: Analyze a PostGIS Database using SQL	34
Summary	39
3 Cartographic Modelling with Forest Inventories	41
Lab Overview	41
Learning Objectives	42
Deliverables	42
Data	43
Task 1: Query and export data from UBC PostgreSQL server	43
Task 2: Calculate Percentage of Old Growth Forests	57

References	63
Summary	64
4 Exploring QGIS and OpenStreetMap	65
Lab Overview	65
Learning Objectives	65
Deliverables	66
Data	66
Task 1: Downloading OpenStreetMap data to QGIS	66
Task 2: Inspecting and processing OpenStreetMap data	71
Task 3: Simple spatial analysis with QGIS	75
Summary	78
5 Raster and Least Cost Path Analysis	79
Lab Overview	79
Learning Objectives	79
Deliverables	80
Data	80
Task 1: Define a raster area of interest	80
Task 2: Seamless mosaicking	82
Task 3: Calculating movement costs	86
Task 4: Least cost path analysis in ArcGIS Pro	91
Summary	94
6 Digitization and Editing with Kart	95
Lab Overview	95
Learning Objectives	95
Deliverables	96
Data	96
Task 1: Configure Git, install Kart, and create your first repository	96
Task 2: Edit a layer in QGIS with version control	101
Task 3: Working on and merging different branches	103
Task 4: Digitize some buildings in ArcGIS Pro	107
Summary	110
7 Geocoding	111
Lab Overview	111
Learning Objectives	111
Deliverables	111
Data	112
Task 1: Download and Prepare Data	112
Task 2: Prepare Data	113
Task 3: Geocoding a Table of Addresses	114
Task 4: Create a shapefile from tabular data	117
Summary	119

Welcome

These are the course materials for GEM 510 in the Master of Geomatics for Environmental Management program (MGEM) at the University of British Columbia (UBC). These Open Educational Resources (OER) were developed to foster the Geomatics Community of Practice that is hosted by the Faculty of Forestry at UBC.

These materials are primarily lab assignments that students enrolled in GEM 510 will complete and submit for credit in the program. Note that much of the data referenced are either public datasets or otherwise only available to students enrolled in the course for credit. Deliverables for these assignments are submitted through the UBC learning management system and only students enrolled in the course may submit these assignments for credit.

How to use these resources

Each “chapter” is a standalone lab assignment designed to be completed over one or two weeks.

Students enrolled in GEM 510 will submit all deliverables through the course management system at UBC for credit and should consult the schedule and deadlines posted there. The casual user can still complete the tutorials step-by-step, but the data that are not already publicly available are not hosted on this website and therefore you will not have access to them.

Unless otherwise noted, all materials are Open Educational Resources (OER) and licensed under a Creative Commons license (CC-BY-SA-4.0). Feel free to share and adapt, just be sure to share with the same license and give credit to the author.

How to get involved

Because this is an open project, we highly encourage contributions from the community. The content is hosted on our GitHub repository and from there you can open an issue or start a discussion. Feel free to open an issue for any

typos, factual discrepancies, bugs, or topics you want to see. We are always looking for great Canadian case studies to share! You can also fork our GitHub repository to explore the source code and take the content offline.

Chapter 1

Introduction to PostgreSQL and PostGIS

Written by Paul D. Pickell

Lab Overview

In this lab, you will be introduced to a Free and Open Source Software (FOSS) and widely-used database management system known as PostgreSQL. You will learn how to set up your own database, connect to an enterprise database, and write Structured Query Language (SQL) queries both from the command line and a popular postgres server admin graphical user interface. The software used in this lab includes PostgreSQL, psql, pgAdmin, and QGIS.

Learning Objectives

- Distinguish between PostgreSQL server, PostGIS database, and layer views
 - Practice executing SQL queries on an enterprise PostGIS database from various software interfaces
 - Export layer views from a PostGIS database in QGIS and ArcGIS Pro
-

Deliverables

Create a simple map of UBC Vancouver campus using at least five layers from the ubcv database (30 points)

Try querying a layer or two and showing only a subset of the total features. Symbolize each layer however you want in either QGIS or ArcGIS Pro. The goal here is to practice changing symbologies and creating a professional layout. At a minimum, your map should have:

- At least five layers symbolized from the ubcv database
 - Informative title based on the layers you chose (do not use “UBC Vancouver Campus” as your title)
 - North arrow
 - Scale bar or other representation of scale
 - Your name and date
 - Projected in NAD83(CSRS) Canada Atlas Lambert (EPSG:3979)
-

Data

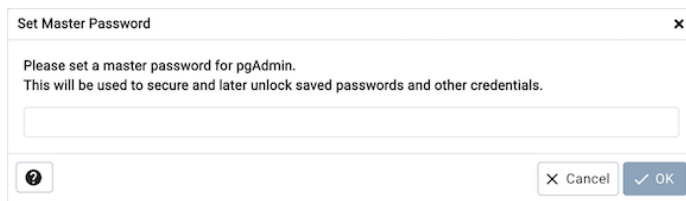
All data for this lab are accessible via the UBC PostgreSQL server. Instructions for connecting to the server are given in the tasks below.

Task 1: Set up pgAdmin and Connect to the UBC PostgreSQL server

Step 1: Ensure that you are authenticated through UBC myVPN. If you are at UBC, use your dedicated ethernet port or connect via the ubcsecure wireless network. If you are away from campus, you will need to first connect to the UBC myVPN service using Cisco AnyConnect Secure Mobility Client. Only authenticated users with a Campus Wide Login (CWL) and connected to the UBC myVPN may access the UBC PostgreSQL server.

Step 2: Start pgAdmin 4. This software is a management tool for PostgreSQL databases, hence “pg”.

Step 3: Upon starting pgAdmin, you will be prompted to set a master password. This is an important step as this master password will be used to encrypt all other passwords you use to connect to various server databases. **DO NOT USE THE SAME PASSWORD AS YOU WILL USE FOR ANY DATABASE!**

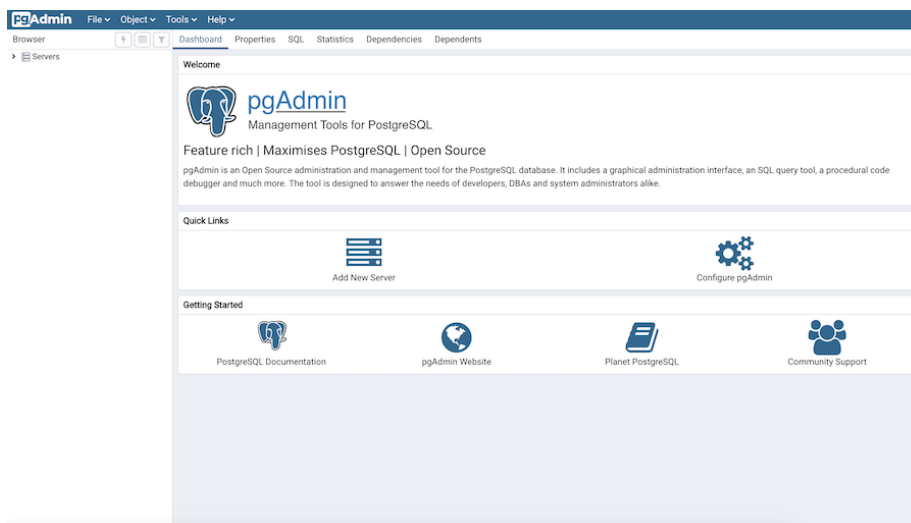


Set Master Password [X]

Please set a master password for pgAdmin.
This will be used to secure and later unlock saved passwords and other credentials.

[?] [X] Cancel [✓] OK

Step 4: From the pgAdmin dashboard, click “Add New Server”.



Step 5: Name the connection “UBC PostgreSQL Server”. Click the “Connection” tab, enter FRST-GIS.ead.ubc.ca for the Host Name, change the Username to “student”, and then enter the password that has been shared with the class. Leave everything else as the default, but you can choose to save the password if you want. Click “Save”. If the password is correct, then pgAdmin should automatically connect to the server and you will see it listed in the Browser, just expand “Servers”.

The screenshot shows the 'Register - Server' dialog box with the 'Connection' tab selected. The fields are as follows:

Field	Value
Host name/address	FRST-GIS.ead.ubc.ca
Port	5432
Maintenance database	postgres
Username	student
Kerberos authentication?	☐
Password	
Save password?	☐
Role	
Service	

At the bottom, there are buttons for 'Close', 'Reset', and 'Save'. A note below the password field states: 'In edit mode the password field is enabled only if Save Password is set to true.'

Although the connection was automatic this time, each time you start pgAdmin, you will need to reconnect to the server after entering your master password. This is as simple as double-clicking the server name from the server list or right-click and select “Connect Server”.

Step 6: From the Browser or Object Explorer pane on the left, expand “Servers”, “PostgreSQL XX” (where “XX” is your current PostgreSQL version), “Databases”, “ubcv”, “Schemas”, then expand “public”, then expand “Tables” to view all the tables in the ubcv database.

Step 7: Right-click on a table and then select “View/Edit Data” > “All Rows”. pgAdmin will create an SQL query and then show the result. This is one way to view the attribute table from pgAdmin.

Query: `SELECT * FROM public.ubcv_buildings ORDER BY ogc_fid ASC`

ogc_fid	bidg_id	name	bidg_code	shortname	postal_code	primary_address
1	VEL10179	Telus Hub	TELS	Telus	V6T 1W5	5895 Berton A
2	VEL10207	Building Operations Storage Shed	BOSS	BOps Shed	V6T 1Z4	2214 Lower A
3	VEL10054	Rose Garden Parkade	PARR	Rose Parkade	V6T 1Z1	6278 NW Mari
4	VEL10093	Leon and Thea Koerner University Centre	UCEN	University Centre	V6T 1Z2	6331 Crescom
5	VEL10045	Pulp and Paper Centre	PAFC	Pulp and Paper	V6T 1Z4	2385 East Mal
6	VEL10154	Walter C. Koerner Library	KLBI	Koerner Library	V6T 1Z2	1958 Main Ma
7	VEL10047	Hebb Building	HEBB	Hebb	V6T 1Z1	2045 East Mal
8	VEL10051	Rodney Graham Millennium Sculpture Pavilion	RSCU	Sculpture	V6T 1Z1	1940 Main Ma
9	VEL10257	Carey Centre	CRVC	Carey Centre	V6T 1J6	5920 Iona Dr.
10	VEL10044	Frank Forward Building	FORW	Forward	V6T 1Z4	6350 Stores R

Total rows: 434 of 434 Query complete 00:00:00.416 Ln 2, Col 22

Step 8: You can also execute your own SQL query. Right-click on a table and select “Query Tool”. At the top, you can write any SQL query you want then click the Run button that looks like a triangle from the top ribbon. pgAdmin will return the resulting table below.

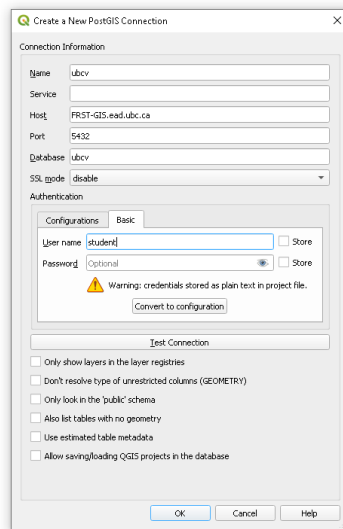
Query: `SELECT * FROM public.ubcv_buildings WHERE postal_code = 'V6T 1Z1';`

ogc_fid	bidg_id	name	bidg_code	shortname	postal_code	primary_address
3	VEL10054	Rose Garden Parkade	PARR	Rose Parkade	V6T 1Z1	6278 NW Marine Drive
7	VEL10047	Hebb Building	HEBB	Hebb	V6T 1Z1	2045 East Mal
8	VEL10051	Rodney Graham Millennium Sculpture Pavilion	RSCU	Sculpture	V6T 1Z1	1940 Main Mal
37	VEL10044	Green College - Ritchie Laundry	GRNZ	Kitchen	V6T 1Z1	6201 Cecil Green Park Road, Bul
39	VEL10435	Green College - D Commons	GRNB	Commons	V6T 1Z1	6201 Cecil Green Park Road, Bul
30	VEL10189	Green College - D Graham House	GRNT	Graham	V6T 1Z1	6201 Cecil Green Park Road, Bul
21	VEL10426	Arts Student Centre	ARTC	ASC	V6T 1Z1	1860 East Mal
24	VEL10234	North Parkade	PARN	North Parkade	V6T 1Z1	6115 Student Union Boulevard
25	VEL10328	Chemistry	CHEM	Chemistry	V6T 1Z1	2036 Main Mal
44	VEL10361	Hillel House - Diamond Foundation Centre for Jewish Campus L.	HILL	Hillel House	V6T 1Z1	6145 Student Union Boulevard
36	VEL10109	Indian Residential School History and Dialogue Centre (IRSHDC)	IRSC	IRSHDC	V6T 1Z1	1985 Learners' Walk
38	VEL10256	Robert H. Lee Alumni Centre	ALUM	Alumni	V6T 1Z1	6163 University Boulevard
37	VEL10421	UBC Counselling Services Annex	CNSL	Counselling	V6T 1Z1	1930 East Mal
38	VEL10441	Recreation Centre North	RCN	Rec North	V6T 1Z1	6140 Student Union Boulevard
35	VEL10407	AMS Nest	NEST	Nest	V6T 1Z1	6133 University Boulevard
36	VEL10071	UBC Aquatic Centre	AQUA	Aquatic	V6T 1Z1	6080 Student Union Boulevard
15	VEL10212	Brook Commons East - Tallwood House	BRCE	Brook East	V6T 1Z1	6088 Walter Gage Road, Building
299	VEL10104	Allard Hall	ALRD	Allard	V6T 1Z1	1822 East Mal
310	VEL10092	Vancouver School of Theology	VST	VST	V6T 1Z1	6015 Walter Gage Road
20	VEL10163	Abdul Latifa Science Student Centre	ALSC	Latifa	V6T 1Z1	2055 East Mal
315	VEL10174	Green College - B East	GRNB	B East	V6T 1Z1	6201 Cecil Green Park Road, Bul

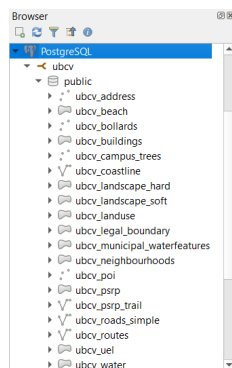
Total rows: 43 of 43 Query complete 00:00:00.361 Ln 1, Col 67

Task 2: Explore with QGIS

Step 1: Open QGIS. From the Browser pane on the left, right-click “PostgreSQL” and select “New Connection”.



Step 2: In your Browser pane, you should now see the ubcv database connection. Expand it to see the available public schema, then expand that to see all available tables that have geometries. You can double-click any of these or click-and-drag into your empty project to view them.

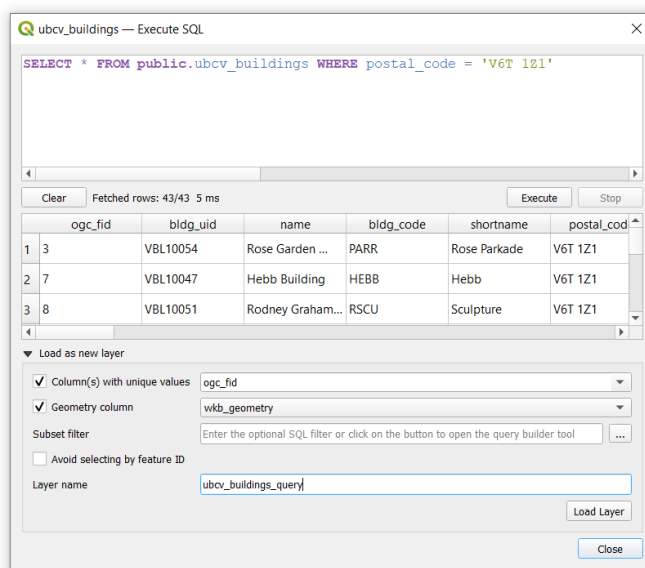


Step 3: Spend some time exploring the data. Open the attribute tables. View the properties and metadata. What is the EPSG code for the Coordinate Reference System (CRS) for these data?

At this point, keep in mind that all the data still live on the postgres server. You may have already seen the other databases on the server. If you want to view the data in those databases, you will need to create a new database connection

for each database that you want to connect to. Feel free to explore the data in the other databases, noting that what is available at any given time may be different from what you see in the screenshots in this lab. The UBC PostgreSQL server hosts many databases that are used for research and teaching!

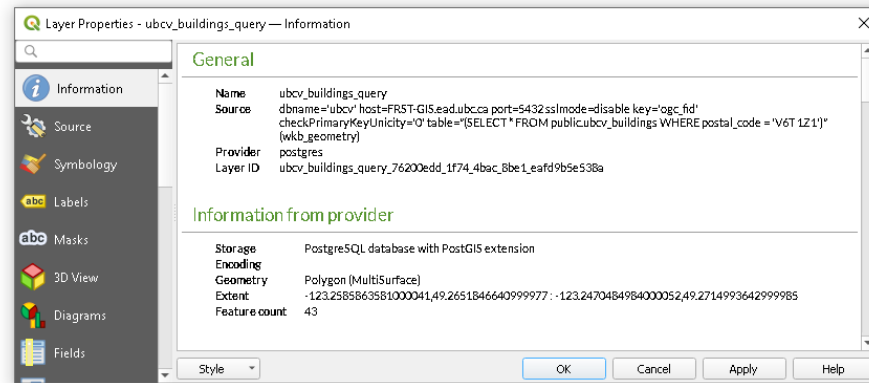
Step 4: You can also execute SQL queries in QGIS directly from the PostGIS database. From the Browser pane, right-click any available table in the PostGIS database and then select “Execute SQL...”. In the dialogue window that opens, you will see a default SQL query that will return the first 10 features (tuples) of the selected relation.



Step 5: Write an SQL query to return some subset of the data then click the “Execute” button. QGIS will return a preview of the attribute table for the query. From here, you can export this as a layer to view the result in the map. Expand “Load as a new layer” and then toggle on “Column(s) with unique values” and ensure the field is set to “ogc_fid”, this is the primary key for the table in PostGIS. Toggle on “Geometry column” and ensure the field is set to “wkb_geometry”, this is the “Well Known Binary” format used to represent features in PostGIS. Finally, change the “Layer name” to something to distinguish your query from others and then click “Load layer” and then “Close”. The result of the SQL query should now appear as a layer in your QGIS map.

Note that the layer you just created does not actually exist outside your QGIS project. If you navigate into the properties of this layer, you will see the Source is simply a reference to the data that are still stored on the postgres server. This can be a really efficient way to handle data without unnecessarily copying it to

a new file!

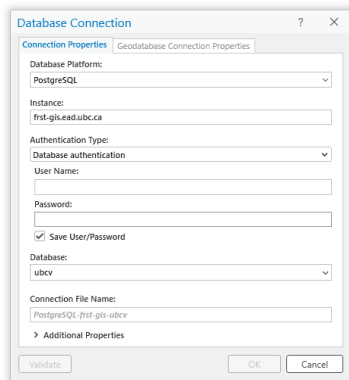


Step 5: You can export any table in the PostGIS database to your local computer by right-clicking the table name in the PostGIS database and then selecting “Export Layer” > “To File...”. From the new dialogue window, you can configure the output file however you want. You can do the same for the query layer by right-clicking the query layer created in Step 5 and selecting “Export” > “Save Features As...”.

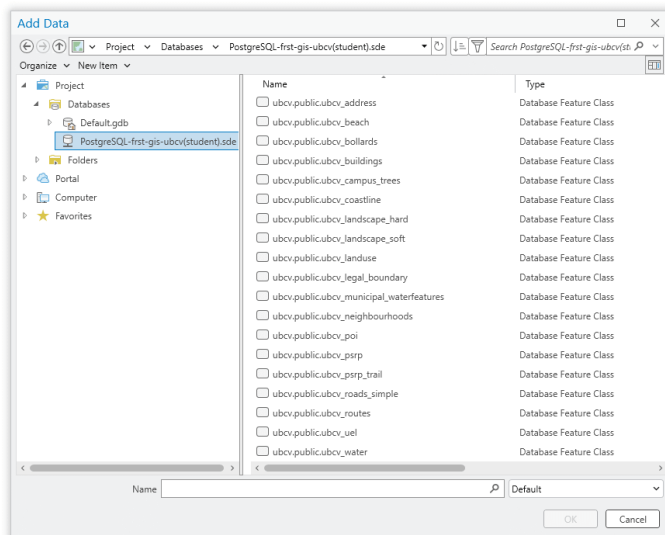
Task 3: Explore with ArcGIS Pro

Step 1: Start ArcGIS Pro and create a new project.

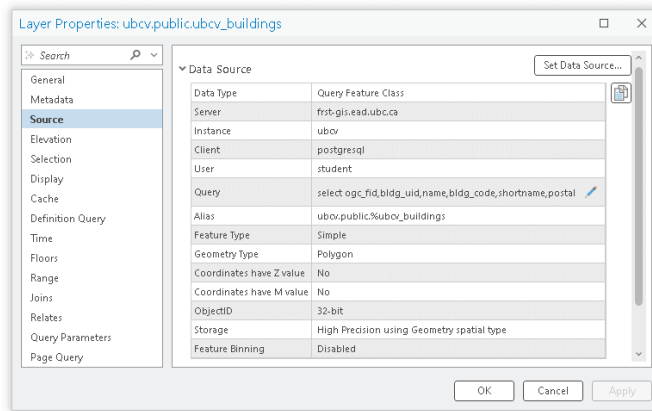
Step 2: From the top ribbon, navigate to the “Insert” tab, then click “Connections”, and from the drop-down menu, select “New Database Connection”. Enter the database connection details in the same way that you did in the prior tasks. Once you have correctly entered the correct credentials, ArcGIS Pro will do a “soft” connection to the server to retrieve the available databases. From the “Database” drop-down menu, select “ubcv” and then click “OK”.



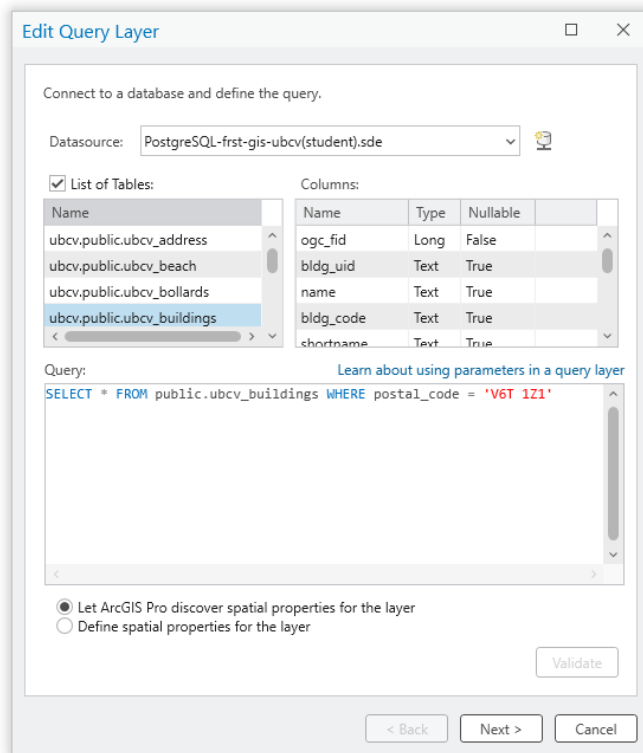
Step 3: From the top ribbon, navigate to the “Map” tab, then click “Add Data”. From the dialogue window that opens, expand “Databases”, and you will see your local project geodatabase that was created when you initiated the project and the newly added postgres server. Click the postgres server connection to view the tables and then add a layer to your map.



Step 4: Once you have added a layer to your map, right-click it in your Contents pane and open the properties. Again, like the query layer you created in QGIS, you will see this layer is a dynamic reference to the data on the PostGIS database.



Step 5: Under Data Source, there is a Query field that includes a generic SQL query statement. Click the pencil icon on the right to open the Edit Query Layer dialogue window. This window is similar to the query window you used in QGIS. Enter the same query you used in Task 2 then click “Validate”. If the query is valid, ArcGIS Pro will allow you to click “Next”. Leave the “Let ArcGIS Pro discover spatial properties for this layer” toggled on. The next window will allow you to select the Unique Identifier Field(s), just like in QGIS. This should automatically have selected “ogc_fid” and automatically identified the geometry type, so you can leave everything as-is and click “Finish”.



You will be returned to the Properties dialogue of the layer you added in your map, but you should see that the Query field has now been updated. Click “OK” to exit the properties and the layer in the map should now only be showing the features that match your query.

Task 4: Explore with psql Shell

Step 1: From your Windows search bar, search for “psql” and open the SQL Shell (psql) application. This will open a command prompt and allow you to interact with the PostgreSQL server directly using mostly SQL statements.

The prompt should say **Server** [localhost]:. Type “FRST-GIS.ead.ubc.ca” and then hit “Enter” on your keyboard.

The prompt should then say **Database** [postgres]:. Type “ubcv” and then press “Enter” on your keyboard.

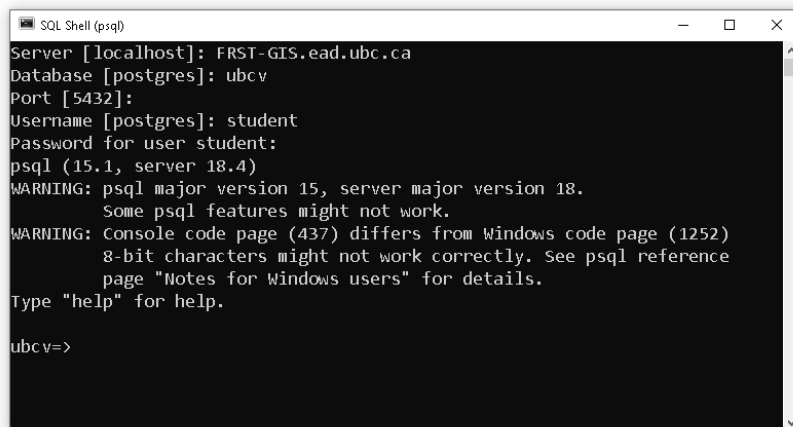
The prompt should then say **Port** [5432]:. This is the default port that the PostgreSQL server uses. Press “Enter” on your keyboard to accept the default

port.

The prompt should then say `Username [postgres]:`. Type “student” and then press “Enter” on your keyboard.

The prompt should then ask you for the password, `Password for user student:`. Type the password that was shared with the class and then press “Enter” on your keyboard.

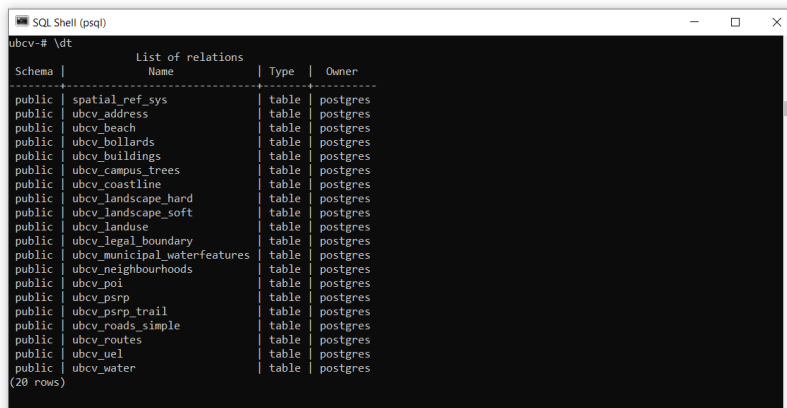
If everything is successful, then you should see `ubcv=>`, which indicates you can now enter either psql commands or SQL statements. (See screenshot below)



```
SQL Shell (psql)
Server [localhost]: FRST-GIS.ead.ubc.ca
Database [postgres]: ubcv
Port [5432]:
Username [postgres]: student
Password for user student:
psql (15.1, server 18.4)
WARNING: psql major version 15, server major version 18.
Some psql features might not work.
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
Type "help" for help.
ubcv=>
```

Step 2: List all the available databases on the postgres server with the `\l` command. If your psql window is too small, you might see `-- More --`. Just continue to press “Enter” to list more tables. You can change to any of these databases by using the `\c [database name]` command like `\c postgres`.

Step 3: List the tables in the ubcv database with the `\dt` command. You can view a full list of psql commands and their useage with the `\?` command.

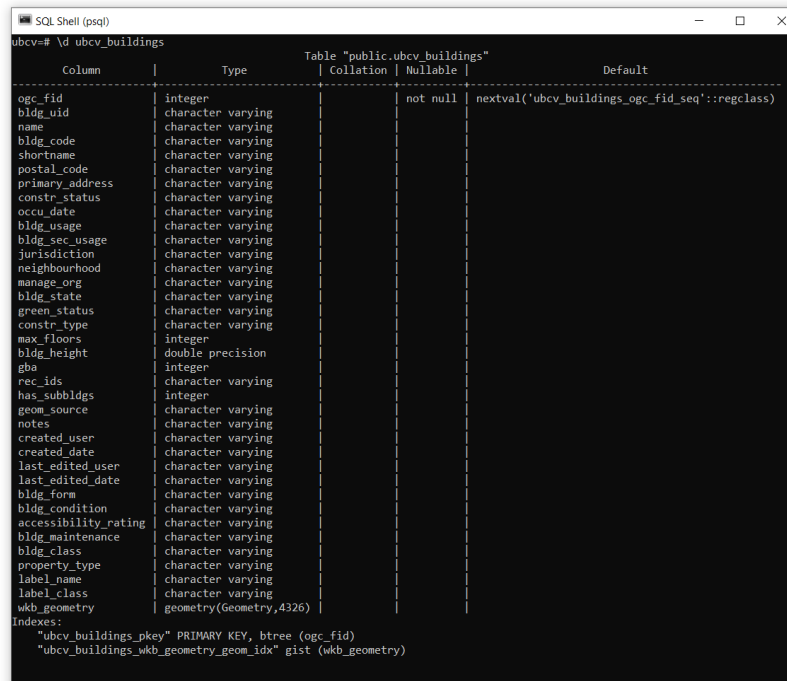


```

SQL Shell (psql)
ubcv=# \dt
          List of relations
Schema |          Name          | Type  | Owner
-----+-----+-----+-----
public | spatial_ref_sys        | table | postgres
public | ubcv_address            | table | postgres
public | ubcv_beach              | table | postgres
public | ubcv_bollards           | table | postgres
public | ubcv_buildings          | table | postgres
public | ubcv_campus_trees       | table | postgres
public | ubcv_coastline          | table | postgres
public | ubcv_landscape_hard     | table | postgres
public | ubcv_landscape_soft     | table | postgres
public | ubcv_landuse            | table | postgres
public | ubcv_legal_boundary      | table | postgres
public | ubcv_municipal_waterfeatures | table | postgres
public | ubcv_neighbourhoods     | table | postgres
public | ubcv_poi                | table | postgres
public | ubcv_psrp               | table | postgres
public | ubcv_psrp_trail         | table | postgres
public | ubcv_road_simple        | table | postgres
public | ubcv_routes             | table | postgres
public | ubcv_wel                | table | postgres
public | ubcv_water              | table | postgres
(20 rows)

```

Step 4: List all the fields for any table with the `\d [table name]` command. For example, `\d ubcv_buildings`.



```

SQL Shell (psql)
ubcv=# \d ubcv_buildings
          Table "public.ubcv_buildings"
          Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
ogc_fid         | integer                |           | not null | nextval('ubcv_buildings_ogc_fid_seq'::regclass)
bldg_uid        | character varying      |           |          |
name            | character varying      |           |          |
bldg_code       | character varying      |           |          |
shortname       | character varying      |           |          |
postal_code     | character varying      |           |          |
primary_address | character varying      |           |          |
constr_status   | character varying      |           |          |
occu_date       | character varying      |           |          |
bldg_usage      | character varying      |           |          |
bldg_sec_usage  | character varying      |           |          |
jurisdiction    | character varying      |           |          |
neighbourhood   | character varying      |           |          |
manage_org      | character varying      |           |          |
bldg_state      | character varying      |           |          |
green_status    | character varying      |           |          |
constr_type     | character varying      |           |          |
max_floors      | integer                |           |          |
bldg_height     | double precision       |           |          |
gba             | integer                |           |          |
rec_ids         | character varying      |           |          |
has_subbldgs    | integer                |           |          |
geom_source     | character varying      |           |          |
notes           | character varying      |           |          |
created_user    | character varying      |           |          |
created_date    | character varying      |           |          |
last_edited_user | character varying      |           |          |
last_edited_date | character varying      |           |          |
bldg_form       | character varying      |           |          |
bldg_condition  | character varying      |           |          |
accessibility_rating | character varying      |           |          |
bldg_maintenance | character varying      |           |          |
bldg_class      | character varying      |           |          |
property_type   | character varying      |           |          |
label_name      | character varying      |           |          |
label_class     | character varying      |           |          |
wkb_geometry    | geometry(Geometry,4326) |           |          |
Indexes:
    "ubcv_buildings_pkey" PRIMARY KEY, btree (ogc_fid)
    "ubcv_buildings_wkb_geometry_geom_idx" gist (wkb_geometry)

```

Step 5: In addition to using psql commands you can also execute SQL statements directly in the console. For example, type `SELECT COUNT(*) FROM ubcv_buildings;` to get a count of all the rows in a table. Notice that `SELECT` statements return tables, the table below has one row and one field.

```
SQL Shell (psql)
ubcv=# SELECT COUNT(*) FROM ubcv_buildings;
count
-----
434
(1 row)
```

Step 6: You can get the unique values of any field by using the `DISTINCT` keyword. For example, `SELECT DISTINCT green_status FROM ubcv_buildings;`. Note that in this returned table, the blank space below “REAP Bronze” is a valid empty value for this field.

```
SQL Shell (psql)
ubcv=# SELECT DISTINCT green_status FROM ubcv_buildings;
green_status
-----
REAP Bronze
LEED Gold
LEED Silver
LEED Platinum
REAP Gold
REAP Silver
REAP Platinum
REAP Gold Plus
(9 rows)
```

Step 7: You can execute the SQL query like you did in the previous tasks, but the psql console is not ideal for wielding large tables with many fields and the formatting will not be very readable. So it is best to return only the fields that you actually need to inspect. For example, `SELECT name, bldg_usage, constr_type, postal_code FROM ubcv_buildings WHERE postal_code = 'V6T 1Z1';`.

```
SQL Shell (psql)
ubcv=# SELECT name, bldg_usage, constr_type, postal_code FROM ubcv_buildings WHERE postal_code = 'V6T 1Z1';
name | bldg_usage | constr_type | postal_code
-----|-----|-----|-----
Rose Garden Parkade | Parking | Concrete | V6T 1Z1
Hebb Building | Academic | Concrete | V6T 1Z1
Rodney Graham Millennium Sculpture Pavilion | Services | OtherMixed | V6T 1Z1
Green College - Kitchen/Laundry | StudentHousing | Wood | V6T 1Z1
Green College - D Commons | Services | Wood | V6T 1Z1
Green College - D Graham House | Services | Wood | V6T 1Z1
Arts Student Centre | Services | Wood | V6T 1Z1
North Parkade | Parking | Concrete | V6T 1Z1
Chemistry | Academic | Concrete | V6T 1Z1
Hilf House - Diamond Foundation Centre for Jewish Campus Life | Other | Concrete | V6T 1Z1
Indian Residential School History and Dialogue Centre (IRSHDC) | Academic | Concrete | V6T 1Z1
Robert H. Lee Alumni Centre | Services | Concrete | V6T 1Z1
UBC Counselling Services Annex | Services | Wood | V6T 1Z1
Recreation Centre North | Athletics | Wood | V6T 1Z1
AMS West | Services | Concrete | V6T 1Z1
UBC Aquatic Centre | Athletics | Concrete | V6T 1Z1
Brock Commons East - Tallwood House | Commons | Wood | V6T 1Z1
Allard Hall | Academic | Concrete | V6T 1Z1
Vancouver School of Theology | Other | Concrete | V6T 1Z1
Abdulla Science Student Centre | Services | Wood | V6T 1Z1
Green College - 8 East | StudentHousing | Wood | V6T 1Z1
Green College - D Coach House | StudentHousing | Wood | V6T 1Z1
Green College - Guest House | StudentHousing | Wood | V6T 1Z1
Green College - Principal's Residence | StudentHousing | Wood | V6T 1Z1
Irving K. Barber Learning Centre | Academic | Concrete | V6T 1Z1
Buchanan Building | Academic | Concrete | V6T 1Z1
Buchanan Tower | Academic | Concrete | V6T 1Z1
Green College - Admin | StudentHousing | Wood | V6T 1Z1
Mary Bollert Hall | Academic | Concrete | V6T 1Z1
Anthropology and Sociology Building | Academic | Concrete | V6T 1Z1
Student Recreation Centre | Athletics | Concrete | V6T 1Z1
Van Remeril Gymnasium | Athletics | Concrete | V6T 1Z1
Green College - A North | StudentHousing | Wood | V6T 1Z1
Chan Centre for the Performing Arts | Services | Concrete | V6T 1Z1
Lander Clock Tower | Services | Concrete | V6T 1Z1
Green College - A South | StudentHousing | Wood | V6T 1Z1
Cecil Green Squash Court | Athletics | Wood | V6T 1Z1
UBC Life Building | Academic | Concrete | V6T 1Z1
Chemistry A Physics | Academic | Concrete | V6T 1Z1
Brock Hall | Services | Concrete | V6T 1Z1
Cecil Green Park Coach House | Administrative | Wood | V6T 1Z1
Cecil Green Park House | Services | Wood | V6T 1Z1
Henings Building | Academic | Concrete | V6T 1Z1
(43 rows)
```

Some useful psql commands:

- \l List all databases on the current PostgreSQL server
 - \c [database name] Connect or switch to any database on the PostgreSQL server
 - \dt List tables in the current database
 - \? List all psql commands
 - \q Quit the SQL Shell
-

Summary

You should now have a working knowledge of PostGIS databases, PostgreSQL servers, and how to connect and manage them from pgAdmin, QGIS, and ArcGIS Pro. You will continue to practice and extend these skills as nearly all data used in later labs will be accessed via the UBC PostgreSQL server.

Return to the **Deliverables** section to check off everything you need to submit for credit in the course management system.

Chapter 2

Analyze a Relational Database

Written by Paul D. Pickell

Lab Overview

In this lab you will learn how to create, manage, and analyze your own relational database. You will continue to practice with the tools that you learned in the prior lab: pgAdmin, QGIS, ArcGIS Pro, and psql. You will also be introduced to the Geospatial Data Abstraction Library (GDAL) and some of the handy programs that are available from this software library for managing data. You will learn more advanced Structured Query Language (SQL) statements that will allow you join tables, insert and update data, and analyze data in a relational database.

Learning Objectives

- Create and host your own PostgreSQL server with a PostGIS database
- Apply best practices for handling, organizing, and managing data
- Import geospatial data using the Geospatial Data Abstraction Library (GDAL)
- Practice joining and relating tables
- Practice inserting and updating data
- Analyze geospatial data in a relational database using SQL and PostGIS

Deliverables

Dump SQL file from Task 2 (15 points)

SQL statement (just the text) from Task 4 (15 points)

Data

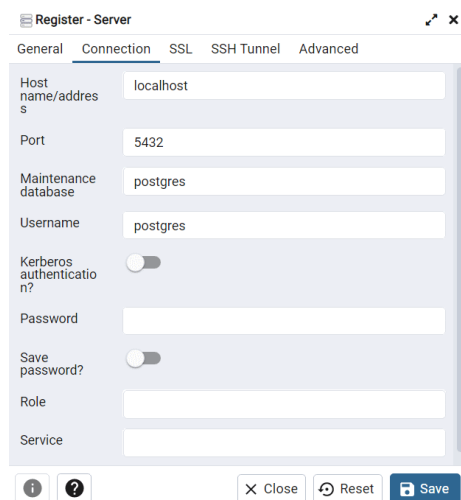
All data for this lab are accessible via the UBC PostgreSQL server. Instructions for connecting to the server are given in the tasks below. We will be using data from the `ubcv` database.

Task 1: Host your own PostgreSQL Server

Step 1: Open pgAdmin.

Step 2: From the “Browser” pane on the left, right-click “Servers” and then select “Register” then “Server...”. We are going to add another server, but this time, it will be hosted on your local machine.

Step 3: Name the server “localhost” then click the “Connection” tab and set the “Host Name/Address” also as “localhost”. Enter a password for the server and leave everything else as the default. When finished, click “Save”. The new localhost server should now appear in your Browser. You can expand it, navigate through “Databases”, “postgres”, “Schemas”, “public”, and “Tables” to find that it is indeed empty.



The screenshot shows the 'Register - Server' dialog box in pgAdmin, with the 'Connection' tab selected. The fields are as follows:

Field	Value
Host name/addresses	localhost
Port	5432
Maintenance database	postgres
Username	postgres
Kerberos authentication?	☐
Password	
Save password?	☐
Role	
Service	

At the bottom, there are buttons for 'Close', 'Reset', and 'Save'.

Step 4: Open the psql shell.

If you are already connected to the UBC PostgreSQL server, you can switch this connection to the localhost server you just made in pgAdmin with the following command: `\c "dbname=postgres host=localhost port=5432 user=postgres"`. You will then be prompted to enter your password.

Alternatively, you can open a new psql terminal window. The prompt should say **Server** [localhost]:. Press “Enter” on your keyboard to accept localhost as the server host name.

The prompt should then say **Database** [postgres]:. This is the default name of the database that was created when you hosted the server from pgAdmin. Press “Enter” on your keyboard to accept postgres as the database to connect to.

The prompt should then say **Port** [5432]:. This is the default port that the PostgreSQL server uses. Press “Enter” on your keyboard to accept the default port.

The prompt should then say **Username** [postgres]:. This is the default username that you created when you set up the PostgreSQL server in pgAdmin. Press “Enter” on your keyboard to accept the postgres username.

The prompt should then ask you for your password, **Password for user postgres**:. Type your password that you set in the earlier step with pgAdmin and then press “Enter” on your keyboard. Note that there is no feedback given to indicate what you are typing.

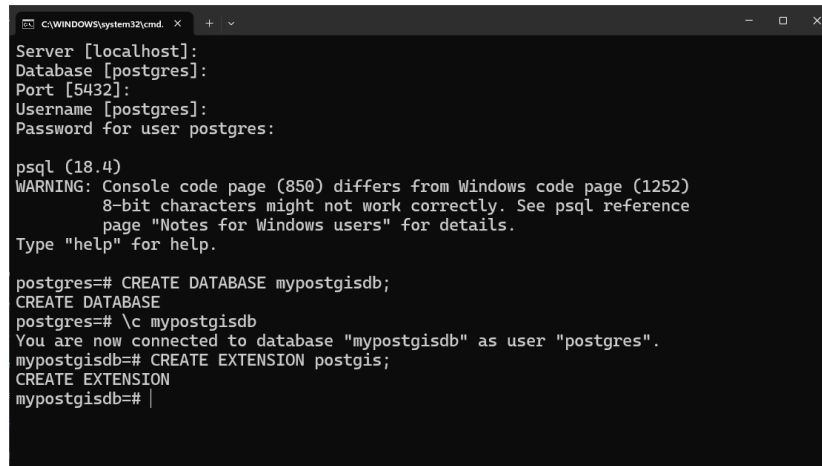
If everything is successful, then you should see **postgres=#**, which indicates you can now enter either psql commands or SQL statements. Note that the hashtag # indicates that you are connected to the current database with the “SUPERUSER” role, which is essentially the highest admin privilege.

Task 2: Create a PostGIS Database

Step 1: Create a new database called “mypostgisdb” with the following SQL command `CREATE DATABASE mypostgisdb;`. It is important to always escape SQL statements with a semi-colon otherwise psql will interpret your input as spanning multiple lines! If you forget the semi-colon, you can always just type it in the console and hit “Enter” on your keyboard and psql will interpret this as a two-line statement.

Step 2: Connect to your new database by using the psql command `\c mypostgisdb`. Note that this is NOT an SQL statement, so there is no need to escape the command with a semi-colon. Your console should now say **mypostgisdb=#**, which indicates you are now connected.

Step 3: Currently this is just a plain-vanilla PostgreSQL relational database that cannot handle spatial data. In order to convert this to a PostGIS database, we need to enable the PostGIS extension using the following SQL: `CREATE EXTENSION postgis;`.



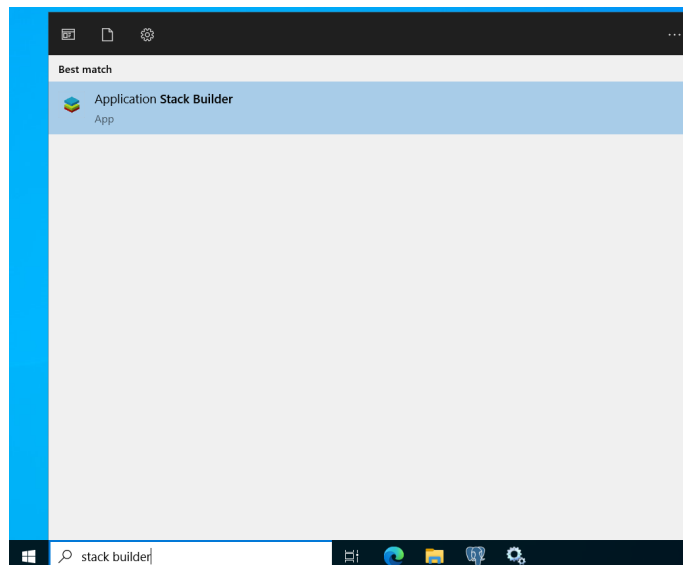
```
C:\WINDOWS\system32\cmd. x + -
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Password for user postgres:

psql (18.4)
WARNING: Console code page (850) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
Type "help" for help.

postgres=# CREATE DATABASE mypostgisdb;
CREATE DATABASE
postgres=# \c mypostgisdb
You are now connected to database "mypostgisdb" as user "postgres".
mypostgisdb=# CREATE EXTENSION postgis;
CREATE EXTENSION
mypostgisdb=# |
```

If the above step fails, then you may need to first install PostGIS. Ordinarily, PostGIS is installed at the same time as PostgreSQL, they come packaged together. To install PostGIS, follow these steps:

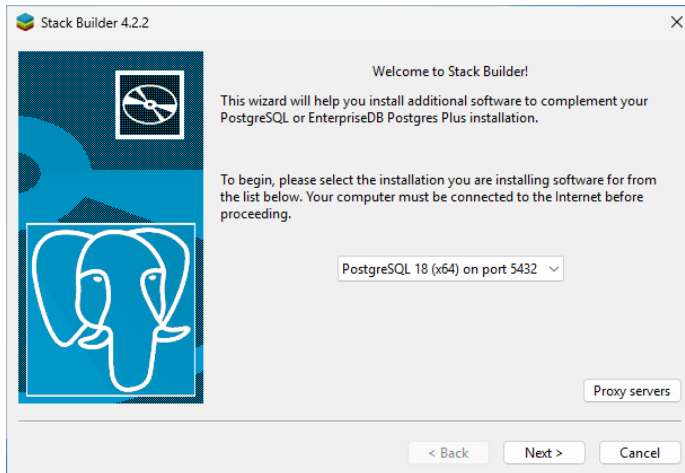
Step 3a: Search “Stack Builder” from your Windows search bar and run the application.



Step 3b: If prompted, enter your admin credential or ask your instructor to

provide this credential for you.

Step 3c: Select “PostgreSQL 18 (x64) on port 5432” from the drop down menu (or whatever version of PostgreSQL is installed on your machine). Click “Next”.



Step 3d: Expand “Categories”, then expand “Spatial Extensions”, and toggle on the PostGIS software version to install. Click “Next”. The Application Stack Builder will prompt for where you want to download PostGIS, just click “Next” to accept the default location. Then you will be prompted to install it after it downloads. Make sure that “Skip Installation” is toggled off, and continue to click “Next” to accept the defaults, and “Agree” to the end user license agreement.

Step 3e: After PostGIS installs successfully, close any psql shell you have open, re-open a new psql shell, and then begin again at Step 2 above.

Step 4: Listing the tables `\dt` will reveal the database has a table called “`spatial_ref_sys`”. List the fields of this table with `\d spatial_ref_sys`.

In addition to the field names, you will see there is an index called “`spatial_ref_sys_pkey`”, which is the dedicated field for the primary key for this table, and a constraint check to ensure that SRID values are valid only between 1 and 998999. You can test this constraint by trying to insert a new srid value of 0: `INSERT INTO spatial_ref_sys (srid) VALUES (0);`. The returned message,

```
ERROR:  new row for relation "spatial_ref_sys" violates check constraint spatial_ref_sys_srid_check
DETAIL:  Failing row contains (0, null, null, null, null).
```

DO NOT modify this table with any srids in the valid range otherwise you will need to recreate the PostGIS database!

Step 5: Return the first ten rows of the table for the fields “`auth_name`” and “`auth_srid`” with `SELECT auth_name,auth_srid FROM spatial_ref_sys`

Table 2.1: Some PostgreSQL data types with example value ranges.

Name	Description
int2	Signed 2-byte integer
int4	Signed 4-byte integer
int8	Signed 8-byte integer
bigserial	Autoincrementing 8-byte integer
float4	Single precision floating-point number (4 bytes)
float8	Double precision floating-point number (8 bytes)
bool	Logical Boolean
char(n)	Fixed length character string where n is the number of permitted characters
varchar(n)	Variable length character string where n is the maximum number of permitted characters
date	Calendar date

LIMIT 10;.

This table contains EPSG codes, which are used to easily relate spheroids, datums, and measurement units to geospatial data. You will find these codes used everywhere when you look at metadata and the properties of different data layers. One very commonly used code is EPSG 4326, which references the WGS 1984 datum.

Use the following query to return the contents of the entire spatial_ref_sys table: `SELECT * FROM spatial_ref_sys;`

Step 6: Write an SQL query to return proj4text of the WGS 1984 datum from the spatial_ref_sys table.

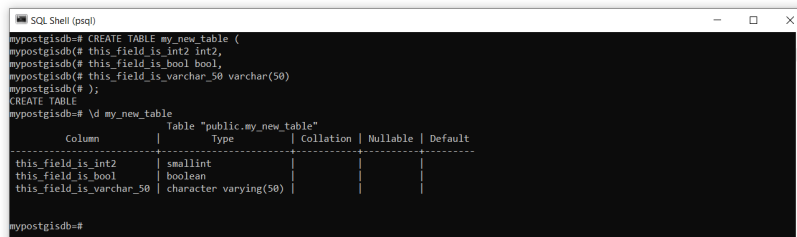
Now we will create some new data, just to show you how to use some useful SQL keywords. We will start with aspatial data first, then move on to spatial data. You can create a new table in your database with the following syntax (note that you cannot simply copy-paste below, you need to supply actual data type variable names):

```
CREATE TABLE table-name (
column-name-1 datatype,
column-name-2 datatype,
column-name-3 datatype,
...
);
```

Data types are very important and the shorthand notation for these in PostgreSQL are listed on this web page and some commonly used data types are reproduced below with examples.

Here is an example of creating a new table with some different data types:


```
CREATE TABLE my_new_table (
  this_field_is_int2 int2,
  this_field_is_bool bool,
  this_field_is_varchar_50 varchar(50)
);
```



```
SQL Shell (psql)
mypostgisdb=# CREATE TABLE my_new_table (
mypostgisdb(# this_field_is_int2 int2,
mypostgisdb(# this_field_is_bool bool,
mypostgisdb(# this_field_is_varchar_50 varchar(50)
mypostgisdb(# );
mypostgisdb=# \d my_new_table
Table "public.my_new_table"
+-----+-----+-----+-----+-----+
| Column          | Type          | Collation | Nullable | Default |
+-----+-----+-----+-----+-----+
| this_field_is_int2 | smallint      |           |          |         |
| this_field_is_bool | boolean       |           |          |         |
| this_field_is_varchar_50 | character varying(50) |           |          |         |
+-----+-----+-----+-----+-----+
mypostgisdb=#
```

Watch out for unescaped parentheses when running multiline SQL statements! If you see `mypostgisdb=#`, then you are able to submit a new psql command or start an SQL statement. If you see `mypostgisdb=-`, then you have already started an SQL statement on a line above in the terminal. If you see `mypostgisdb=(`, then you have added an open parenthesis “(” somewhere in your statement on a line above. When running a SQL statement over multiple lines, you need to ensure that you enclose all parentheses and end the statement with a semicolon `;` in order to return the psql command line back to `mypostgisdb=#`. If you find yourself stuck and unable to exit `mypostgisdb=(`, try to close any open parentheses by typing `)` and pressing “Enter”. If you find yourself unable to exit `mypostgisdb=-`, try ending the SQL statement by typing `;` and pressing “Enter”. If both of those fail, you can try exiting the command with `CTRL + C`, which may also end psql and require you to reconnect to the database.

You can experiment with creating as many new tables as you want. If you need to delete a table, use the `DROP` keyword like this `DROP TABLE my_new_table;`.

It is good practice to assign a field (multiple fields) as a primary key when you create a table. This is done by simply adding `PRIMARY KEY` after the field definition when you make the table. For example:

```
CREATE TABLE my_new_table (
  this_field_is_bigserial bigserial PRIMARY KEY,
  this_field_is_int2 int2,
  this_field_is_bool bool,
  this_field_is_varchar_50 varchar(50)
);
```

The `bigserial` data type is especially useful for this purpose because it auto-increments as you add rows and can accommodate more than 18 *quintrillion* rows, that is more than 18 million trillions! You can also create composite

primary keys that are comprised of two or more fields to uniquely identify all rows:

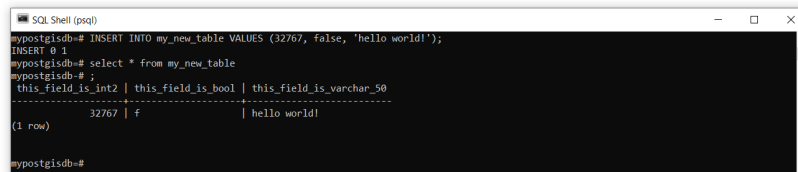
```
CREATE TABLE my_new_table (
  this_field_is_int2 int2,
  this_field_is_bool bool,
  this_field_is_varchar_50 varchar(50),
  PRIMARY KEY(this_field_is_int2, this_field_is_bool)
);
```

Note that if you use a compound primary key, like in the example above, or if you use one of your data fields as the primary key, then you are imposing a logic onto the table that prevents you from creating, updating, or inserting new rows that duplicate those values or combinations of values. After all, that is the purpose of a primary key, to uniquely identify all the tuples. Sometimes, this logic can be useful in the design of a table, but generally it is best practice to create a dedicated field for the primary key and this is also the behaviour that you will observe in attribute tables viewed in ArcGIS Pro and QGIS.

Step 7: Create a new table of assignments that are due next week (up to four total). Include course code, assignment name, the percent weighting of the assignment on your final grade in that course, and the due date. Use the appropriate data types for each field and define a primary key.

Now that the table and fields are defined, we will insert some data into the table. Inserting data uses the `INSERT` keyword followed by `VALUES` and then a comma-separated list of the values you want to insert in parentheses. For example:

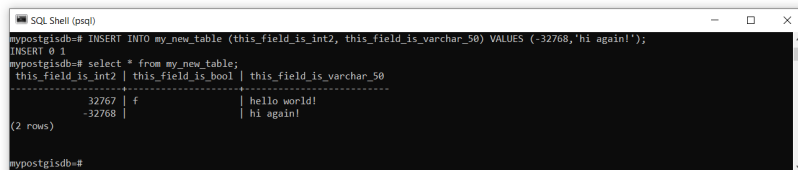
```
INSERT INTO my_new_table VALUES (32767, false, 'hello world!');
```



```
SQL Shell (psql)
mypostgisdb=# INSERT INTO my_new_table VALUES (32767, false, 'hello world!');
INSERT 0 1
mypostgisdb=# select * from my_new_table
mypostgisdb=#
  this_field_is_int2 | this_field_is_bool | this_field_is_varchar_50
-----
32767 | f | hello world!
(1 row)
mypostgisdb=#
```

If you want to insert a value for specific fields, then specify the field name(s) after the table name:

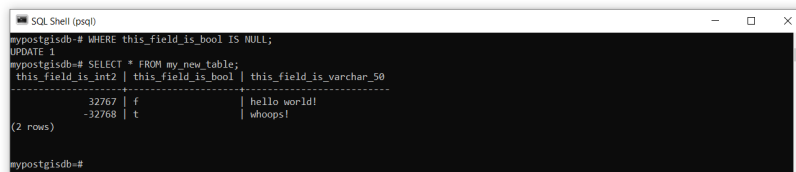
```
INSERT INTO my_new_table (this_field_is_int2, this_field_is_varchar_50) VALUES (-32768, 'hi again!');
```



```
SQL Shell (psql)
mypostgisdb=# INSERT INTO my_new_table (this_field_is_int2, this_field_is_varchar_50) VALUES (-32768, 'hi again!');
INSERT 0 1
mypostgisdb=# select * from my_new_table;
  this_field_is_int2 | this_field_is_bool | this_field_is_varchar_50
-----
32767 | f | hello world!
-32768 | | hi again!
(2 rows)
mypostgisdb=#
```

If you make a mistake or need to update a field later, then you use `UPDATE` and `SET` to identify the set of fields that should be updated. You can also test for `NULL` (empty values) using `IS NULL` or `IS NOT NULL`:

```
UPDATE my_new_table
SET this_field_is_bool = true, this_field_is_varchar_50 = 'whoops!'
WHERE this_field_is_bool IS NULL;
```



```
SQL Shell (psql)
mypostgisdb=# WHERE this_field_is_bool IS NULL;
UPDATE 1
mypostgisdb=# SELECT * FROM my_new_table;
 this_field_is_int2 | this_field_is_bool | this_field_is_varchar_50
-----
32767 | f | hello world!
-32768 | t | whoops!
(2 rows)
mypostgisdb=#
```

Dates require special handling because if you just try to insert them as strings or otherwise, they are treated as literals. For dates, we need to use the special `TO_DATE` function:

```
INSERT INTO my_new_table (this_field_is_date) VALUES (TO_DATE('31-12-1963', 'DD-MM-YYYY'));
```

Step 8: Now fill in your table of assignments by inserting and/or updating the values as needed.

Once you are satisfied with the state of your table, you will dump your entire PostgreSQL database to an output SQL file and this will be one of your deliverables for this lab. PostgreSQL features a utility program called `pg_dump` that will take any database and output an SQL statement in a file that can be used to re-create the database. In other words, `pg_dump` is a backup method. This is also really handy if you want to create a local backup of a remote server! If you want more practice, try using it on the UBC PostgreSQL server.

Step 9: Open a Windows command prompt (search “command”). Take note of your current working directory, which is probably something like `C:\Users\[your username]`. This is where your database SQL file will be saved to. If your command prompt shows your working directory as `C:\Program Files\QGIS X.XX.X` or similar, then you will first need to change to a directory where you have permission to write to. Usually, this is somewhere in your user folder, which you can navigate to in the terminal with the change directory `cd` command: `cd C:\Users\[your username]\Documents`.

To dump your database, use the following command: `pg_dump -d mypostgisdb -U postgres -h localhost > mypostgisdb.sql`. You will be prompted to enter your password and then the new file `mypostgisdb.sql` will be saved in your working directory. If you see a command not found message, then type `where pg_dump`. Then copy the full path to the program executable when running the commands that follow, usually `"C:\Program Files\PostgreSQL\XX\bin"`. Copy the path and then in command prompt type `set PATH=%PATH%;"C:\Program Files\PostgreSQL\XX\bin"`, replacing

XX with your current PostgreSQL version. Try the `pg_dump` command again to ensure it works.

Step 10: Check to make sure that you can load the database back into PostgreSQL. In `psql`, create a new empty database named `mypostgisdb_backup`. Then, enter the following command from the Windows command prompt: `psql -U postgres -h localhost mypostgisdb_backup < mypostgisdb.sql`. You can now query the `postgisdb_backup` database to check that it is a copy of your other `mypostgisdb` database.

Task 3: Import Spatial Data Using GDAL

Now that you have some basic understanding of viewing and manipulating tables, we are going to look at some ways to import spatial data into your PostgreSQL database. To do this, we will be working with some utility programs in the Geospatial Data Abstraction Library (GDAL) pronounced “gee-dall”. You will probably find the GDAL documentation pages very helpful for reference. GDAL is used in ArcGIS, GRASS, SAGA, QGIS, ENVI, Google Earth, and also has Python bindings and an R package with bindings. In short, GDAL is largely the workhorse behind most open source geospatial software packages.

Step 1: Open the OSGeo4W shell by searching for it in Windows. Enter the command `o-help` and inspect all the available programs.

You might notice `psql` can be run from the OSGeo4W shell and `pg_dump` is also there. Some others we will cover in later labs, including `pdal` (Point Data Abstraction Library) pronounced “poodle” or “pee-dahl”, which handles LiDAR data, and many of the raster and image manipulation programs. For this lab, we are going to focus on two commonly used vector programs: `ogrinfo` and `ogr2ogr`.

Step 2: Type `ogr2ogr --help` in the console and press “Enter” on your keyboard. Inspect all the flags - and arguments for this program. If you see a command not found message, then type `where ogr2ogr`. Then copy the full path to the program executable when running the commands that follow, usually “`C:\Program Files\QGIS X.XX.X\bin\`”. Copy the path and then in command prompt type `set PATH=%PATH%;“C:\Program Files\QGIS X.XX.X\bin\”`, replacing `X.XX.X` with your current QGIS version. Try `ogr2ogr --help` again to ensure it works.

`ogr2ogr` is the primary utility program for working with vector data and as you can see, there are a lot of options! To show you some of the power of this little application, we are going to export some data from the UBC PostgreSQL server to our local machine, manipulate it, and then import it to our local PostgreSQL database.

Step 3: `ogr2ogr` is really useful for converting data between different formats

and this is handy for exporting data out of a PostGIS database. Convert the `ubcv_campus_trees` table from the UBC PostgreSQL server with the following command:

```
ogr2ogr -f "ESRI Shapefile" ubcv_campus_trees.shp PG:"host=FRST-GIS.ead.ubc.ca user=student dbname=STUDENT_PASSWORD_HERE"
```

Replace `STUDENT_PASSWORD_HERE` with the password that has been shared with the class.

This command tells `ogr2ogr` to connect to the `ubcv` PostGIS database on the PostgreSQL server `PG:"host=FRST-GIS.ead.ubc.ca user=student dbname=ubcv password=STUDENT_PASSWORD_HERE"`, grab the table named `"ubcv_campus_trees"`, and then return it to the local machine as an ESRI Shapefile `-f "ESRI Shapefile"`. The file will be saved in your current working directory from the OSGeo4W shell.

Step 4: You can also specify an SQL query on the input data. For example, the following command will only return the Forest Sciences Centre polygon from `ubcv_buildings`:

```
ogr2ogr -where "bldg_code='FSC'" -f "ESRI Shapefile" ubcv_FSC.shp PG:"host=FRST-GIS.ead.ubc.ca user=student dbname=STUDENT_PASSWORD_HERE"
```

You can write more complex SQL statements with the `-sql` flag instead of `-where`.

Step 5: Getting data into a PostGIS database is unremarkably similar as to taking it out. Import the FSC polygon into your localhost PostGIS database using the same command as Step 3, but change the following:

- "ESRI Shapefile" to "PostgreSQL"
- delete `ubcv_campus_trees.shp`
- modify the connection parameters for your localhost server
- change `"ubcv_campus_trees"` to `ubcv_FSC.shp`
- specify that we want these data re-projected into UTM Zone 10N coordinates `-t_srs EPSG:3157`

You will receive no feedback in the terminal if you were successful, but you can check in QGIS, ArcGIS Pro, or by using the `ogrinfo` program.

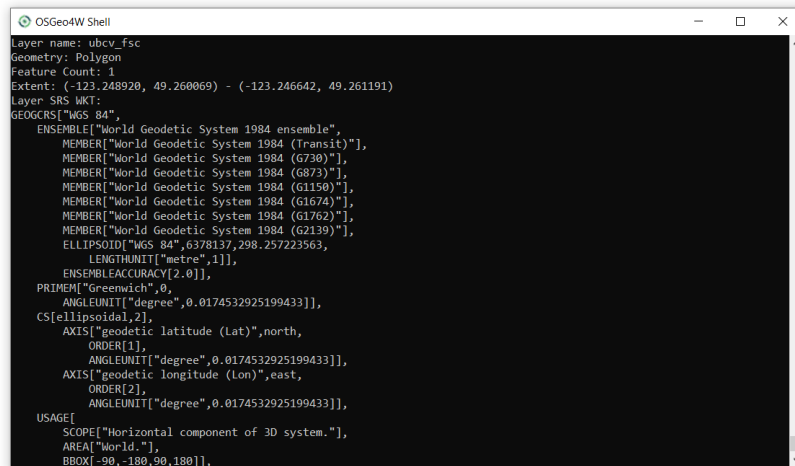
`ogrinfo` is a utility that is able to read a wide-variety of spatial data formats and return metadata for inspection. This program is useful for getting the geometry type of a file, the spatial extent of the features, and the coordinate system or the datum. You can also apply SQL queries and even inspect the attributes of specific features.

Step 6: Check that the FSC polygon was successfully added to your PostGIS database with the following command:

```
ogrinfo PG:"host=localhost user=postgres dbname=mypostgisdb password=YOUR_LOCALHOST_PASSWORD" "ubcv_FSC.shp"
```

The screenshot below is a small snippet of what is returned in the OSGeo4W

terminal window. As you can see, we have access to the feature count, spatial extent, Spatial Reference System (SRS), and even all the field names and data types if you scroll farther down. The `-so` flag gives us this “summary output”.



```

QGIS Shell
Layer name: ubcv_fsc
Geometry: Polygon
Feature Count: 1
Extent: (-123.248920, 49.260069) - (-123.246642, 49.261191)
Layer SRS WKT:
GEOGCRS["WGS 84",
  ENSEMBLE["World Geodetic System 1984 ensemble",
    MEMBER["World Geodetic System 1984 (Transit)"],
    MEMBER["World Geodetic System 1984 (G730)"],
    MEMBER["World Geodetic System 1984 (G873)"],
    MEMBER["World Geodetic System 1984 (G1150)"],
    MEMBER["World Geodetic System 1984 (G1674)"],
    MEMBER["World Geodetic System 1984 (G1762)"],
    MEMBER["World Geodetic System 1984 (G2130)"],
    ELLIPSOID["WGS 84",6378137,298.257223563,
      LENGTHUNIT["metre",1]],
    ENSEMBLEACCURACY[2.0]],
  PRIME["Greenwich",0,
    ANGLEUNIT["degree",0.0174532925199433]],
  CS[ellipsoidal,2],
  AXIS["geodetic latitude (Lat)",north,
    ORDER[1],
    ANGLEUNIT["degree",0.0174532925199433]],
  AXIS["geodetic longitude (Lon)",east,
    ORDER[2],
    ANGLEUNIT["degree",0.0174532925199433]],
  USAGE[
    SCOPE["Horizontal component of 3D system."],
    AREA["World."],
    BBOX[-90,-180,90,180]],

```

Step 7: Repeat this process to import the `ubcv_campus_trees.shp` you created in Step 3 to your localhost PostGIS database.

Task 4: Analyze a PostGIS Database using SQL

Now let us have some fun and explore some of the cool spatial processing features of our PostGIS database. PostGIS supports an unfathomable number of spatial operations that you would ordinarily find in GIS software like QGIS and ArcGIS. This task is merely to give you a taste of what is possible with SQL and PostGIS.

Suppose we want to know what species of trees are planted next to the Forest Sciences Centre at UBC.

Step 1: Open `psql` and connect to your `mypostgisdb` database on your localhost PostgreSQL server.

Step 2: Buffer the `ubcv_fsc` polygon with the following SQL statement:

```
SELECT ST_Buffer(wkb_geometry, 15) INTO ubcv_fsc_buffer FROM ubcv_fsc;
```

This statement tells PostGIS to use the buffer function `ST_Buffer`, which requires a geometry field `wkb_geometry` and a buffer distance 15 (meters in our case, because we re-projected the polygon to UTM Zone 10N in the last task), on the FSC polygon `FROM` `ubcv_fsc`, and then write the resulting buffer to a new table `INTO` `ubcv_fsc_buffer`.

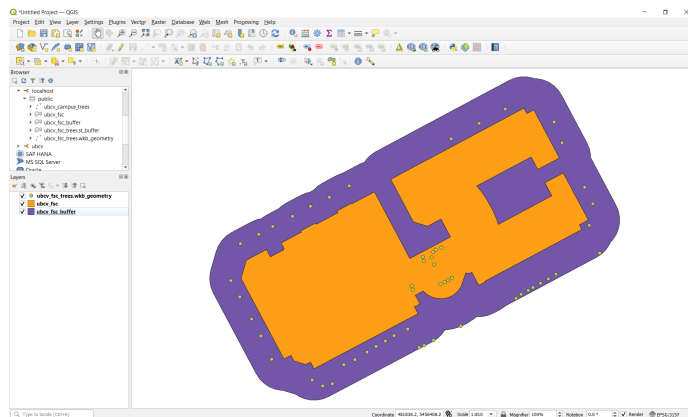
Step 3: Intersect the `ubcv_campus_trees` with the `ubcv_fsc_buffer` polygon you just made with the following SQL statement:

```
SELECT * INTO ubcv_fsc_trees FROM ubcv_campus_trees, ubcv_fsc_buffer WHERE ST_Intersects(wkb_geometry
```

The `ST_Intersects` takes two geometry fields `wkb_geometry` (`ubcv_campus_trees`) and `st_buffer` (`ubcv_fsc_buffer`), computes the intersection, and returns the features from the first geometry argument (`ubcv_campus_trees`) to a new table `INTO ubcv_fsc_trees`.

If you get an error here like this, `ERROR: ST_Intersects: Operation on mixed SRID geometries (Point, 4326) != (Polygon, 3157)` then you know that you have not correctly changed the projection of the `ubcv_campus_trees` data when you imported it into your localhost postgres database. Go back to the last task and check Steps 5-7.

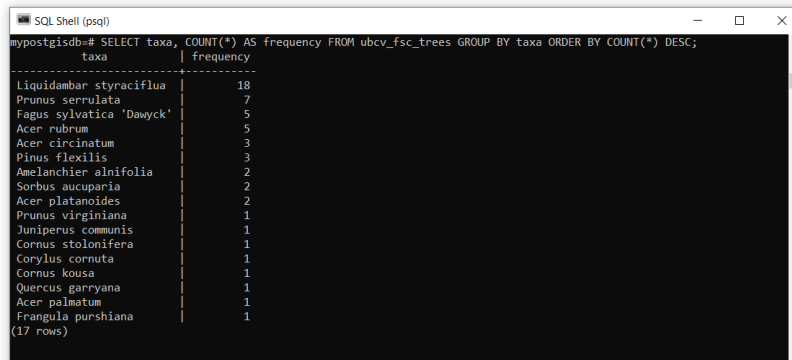
Step 4: Open QGIS and then add `ubcv_fsc`, `ubcv_fsc_buffer`, and `ubcv_fsc_trees.wkb_geometry` to your map to inspect them.



Step 5: Now we will do a simple SQL query to determine the species that are planted around the Forest Sciences Building:

```
SELECT taxa, COUNT(*) AS frequency FROM ubcv_fsc_trees GROUP BY taxa ORDER BY COUNT(*) DESC;
```

This will return the “taxa” column and create a new column called “frequency” that contains values of the `COUNT()` function. The `GROUP BY` statement is needed when we use an aggregation method like `COUNT()`. Finally, the relation is ordered by the aggregation result in descending `DESC` order.



```

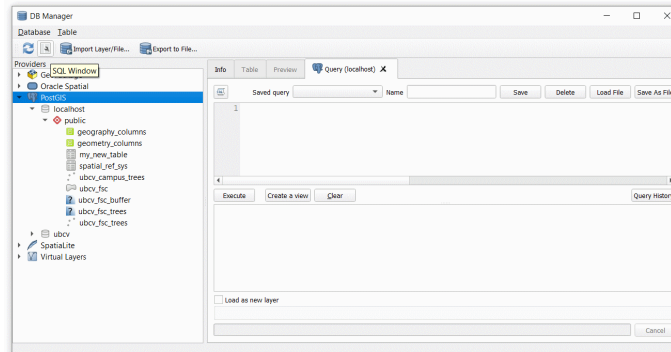
SQL Shell (psql)
mypostgisdb=# SELECT taxa, COUNT(*) AS frequency FROM ubcv_fsc_trees GROUP BY taxa ORDER BY COUNT(*) DESC;
 taxa | frequency
-----+-----
Liquidambar styraciflua |      18
Prunus serrulata        |       7
Fagus sylvatica 'Dawyc' |       5
Acer rubrum             |       5
Acer circinatum         |       3
Pinus flexilis          |       3
Amelanchier alnifolia   |       2
Sorbus aucuparia        |       2
Acer platanoides        |       2
Prunus virginiana       |       1
Juniperus communis      |       1
Cornus stolonifera      |       1
Corylus cornuta         |       1
Cornus kousa            |       1
Quercus garryana        |       1
Acer palmatum           |       1
Frangula purshiana      |       1
(17 rows)

```

Step 6: Modify the statement above to sort the taxa alphabetically.

These last few steps are just to illustrate that you can run the same SQL statements above directly in QGIS.

Step 7: Open QGIS. From the top menu toolbar, select **Database** then **DB Manager...** This will open a new window where you can do basically everything you did in this task and the prior task. You can import/export data to whatever format you want, inspect the database, tables, and even preview the spatial features.



Step 8: At the top of the DB Manager window there is a small button to open an SQL tab. Click that and then enter the buffer statement again, but modify the output table name so that you do not create a conflict:

```
SELECT ST_Buffer(wkb_geometry, 15) INTO ubcv_fsc_buffer_qgis FROM ubcv_fsc;
```

You should now have a new table in the database now called **ubcv_fsc_buffer_qgis**. You can also save the query by giving it a name in the **Name** field and even output the SQL query to a file. This can be helpful for managing large complex queries. The **Create a view** button will display the layer in your map view.

Examine that query above carefully. Remember that **SELECT** operates on column

names, so what exactly are we selecting when we apply a function on a column name? What is the output column name of that function that is returned in **ubcv_fsc_buffer_qgis**? Open the attribute table in QGIS and look at the column name. When we apply a function on a column and do not specify a name for the output column name of the function, then the default column name will simply be the name of the function. So in this case, the output column name will be **st_buffer**, which is not very informative.

Note that the default behaviour of any **SELECT** is to simply return the existing column names. However, we could choose to rename any of those in the table that is returned. For example:

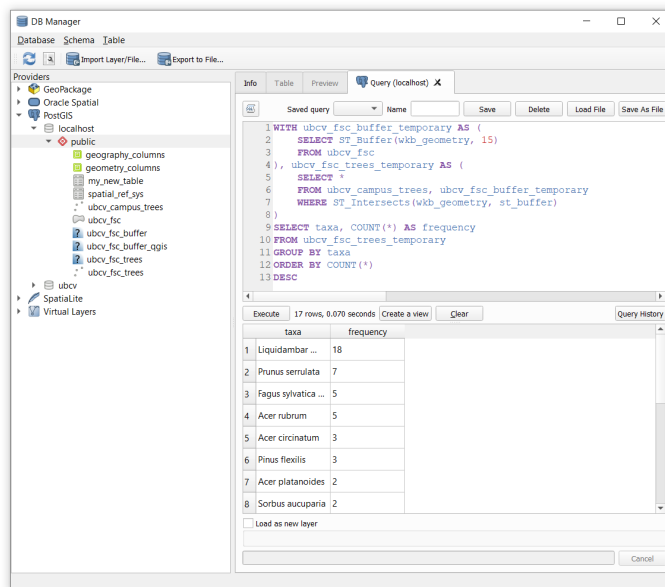
```
SELECT name AS a, bldg_code AS b, postal_code AS c FROM ubcv_buildings
WHERE bldg_code = 'FSC';
```

The **AS** is how we assign aliases to selected column names, which are returned in the new table. We can also completely omit **AS** and just provide the alias after the column name like:

```
SELECT name a, bldg_code b, postal_code c FROM ubcv_buildings
WHERE bldg_code = 'FSC';
```

You should spend some time understanding this behaviour because it is very important for how you structure more complex queries, which we will examine next.

Step 9: Using the **WITH** clause, combine all of our independent queries above into a single query in QGIS that buffers the FSC polygon by 15 m, intersects the buffer with the campus trees, and then reports the number of unique taxa around the building. See the screenshot below.



The `WITH` clause allows us to create temporary relations that can be referenced by name in subsequent `SELECT` statements. In the case above, **ubcv_fsc_buffer_temporary** is the temporary result from the buffer and **ubcv_fsc_trees_temporary** is the temporary result from intersecting the buffer with the campus trees.

As you will see, this is slightly different than using the `INTO` method from our prior examples, which creates a new relation in our database. The benefit of `WITH` and `AS` is that the relation is ephemeral and only exists for the moment that the SQL entire statement is executed; no new relation is written into our database.

Step 10: Using what you have learned, select one of the other PostGIS overlay methods and then apply it to any two datasets in the `ubcv` database on the UBC PostgreSQL server. You must use the `WITH` clause and you must also use one of the SQL aggregation functions. Refer to the examples provided in class and use those as a template to explore how these functions work. It is not important that your analysis makes sense, but it is important that your entire SQL statement can be executed from either QGIS or `psql`. Once you are satisfied with your SQL statement, copy and paste it to the assignment submission page on the course management system.

Hint: Check what the geometry column name is called for the layers that you choose to use and enter those exactly as you see them into your overlay function. Make sure that you are using compatible geometries for the overlay method you choose and read the linked documentation above if you have any doubts!

Summary

PostGIS has many powerful functions for handling geospatial data directly within a PostgreSQL database. These are not always practical for day-to-day use when compared with user-friendly applications like QGIS or ArcGIS Pro. However, you should now appreciate that you can remotely manipulate geospatial data on a server and this opens the possibility of very powerful web-based applications, cloud-based GIS solutions, and automated scripting that requires little visualization. If you are interested in learning more about what others have done, you can check out the Crunchy Data YouTube channel to see past “PostGIS Day” recordings.

Return to the **Deliverables** section to check off everything you need to submit for credit in the course management system.

Chapter 3

Cartographic Modelling with Forest Inventories

Written by Sadie Russell

Lab Overview

Old growth forests are an important feature found in many types of forested ecosystems. They are characterized not only by their age but also by their complex composition and structure. (LePage & Banner, 2014) Old growth stands boast the highest rates of biodiversity and provide critical habitat for rare, threatened, or endangered species. Additionally, they contribute to soil fertility by supporting intricate soil structures formed by micro-organisms, fungi, and plants.

It is generally assumed that the structural complexity desired in a forest develops over time. As such, a stand is designated as old growth once it has reached a certain age without experiencing major disturbance. The length of time required for this designation varies by Biogeoclimatic Ecosystem Classification (BEC) zone.

Old growth forests are also highly valued for timber production due to their superior qualities. (Canadian Institute of Forestry, 2022) These trees have had the opportunity to grow slowly over long periods, resulting in denser wood grain and stronger lumber. Furthermore, their tall, straight trunks are ideal for producing a wide range of wood products.

To balance the benefits of both logging and preserving old growth forests, frameworks and guidelines are put in place to manage Old Growth sites. In BC, Old Growth Forests are assessed using the Cumulative Effects Framework (CEF). This framework was put in place to provide guidelines on assessing the effects

of human activities and natural processes on the provinces forested ecosystems. The provincial old growth and mature-plus-old targets are outlined in the Biodiversity Guidebook (Province of B.C., 1995).

Vancouver Island, British Columbia, has been an area of highly contentious Old Growth logging. Conflicting interests between logging companies, First Nations, Provincial and Federal Governments, and the general public came into the spotlight during the Fairy Creek protests of 2020. Over two years of protest, more than 1,000 protesters were arrested at the site (CBC, 2022). The events at Fairy Creek, and many others like it, have brought the issues of Old Growth conservation to the forefront of timber production relations.

Learning Objectives

By the end of this lab, you will be able to:

- Prepare and clean a forest inventory dataset and related layers for analysis
 - Perform attribute/select-by-expression filtering to isolate old-growth candidate polygons
 - Calculate old-growth area and percentages by polygon using field calculations
 - Produce a cartographic summary (map + table) that communicates old-growth distribution
 - Reflect on limitations and assumptions of VRI-derived old-growth estimates
-

Deliverables

Responses to the questions posed throughout the lab on the course management system (60 points)

A table with fields for total area, old-growth area, and percentage old growth (10 points)

A screenshot of the python expression created for Task 2 (10 points)

A map showing the study area and the distribution of difference in calculated old-growth percentage and targets, by polygon (20 points)

Data

All data for this lab are accessible via the UBC PostgreSQL server. Instructions for connecting to the server are given in the tasks below. These are very large datasets that come from the BC Data Catalogue and are described in the table below. Accessing these data from the UBC PostgreSQL server allows us to extract only the data we need for this lab, which is the extent of Vancouver Island.

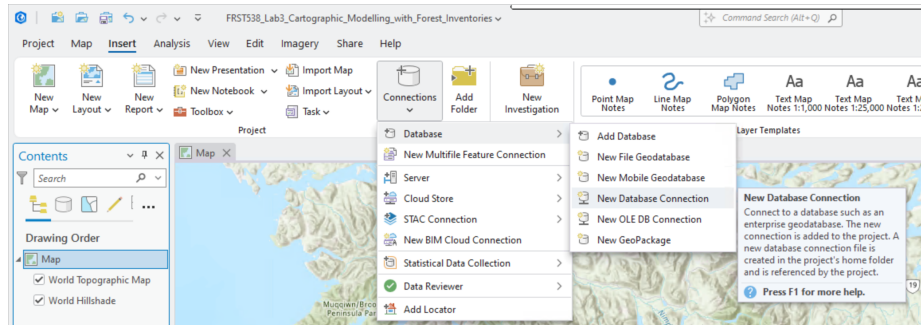
Layer Name	File Name	Reference Link
Vegetation Resource Inventory 2024	vancouver_island_vri	https://catalogue.data.gov.bc.ca/dataset/vri-historical-vegetation-resource-inventory-2002-2024-
Land Designations for Forest Management	vancouver_island_own	https://catalogue.data.gov.bc.ca/dataset/4574f131-470e-407c-bbcc-5b19d9adcc1b
Human Disturbance – 2024	vancouver_island_human_disturbance	//catalogue.data.gov.bc.ca/dataset/human-disturbance-2025
Landscape Units of British Columbia - Current	vancouver_island_landscape_units	//catalogue.data.gov.bc.ca/dataset/landscape-units-of-british-columbia-current

Task 1: Query and export data from UBC PostgreSQL server

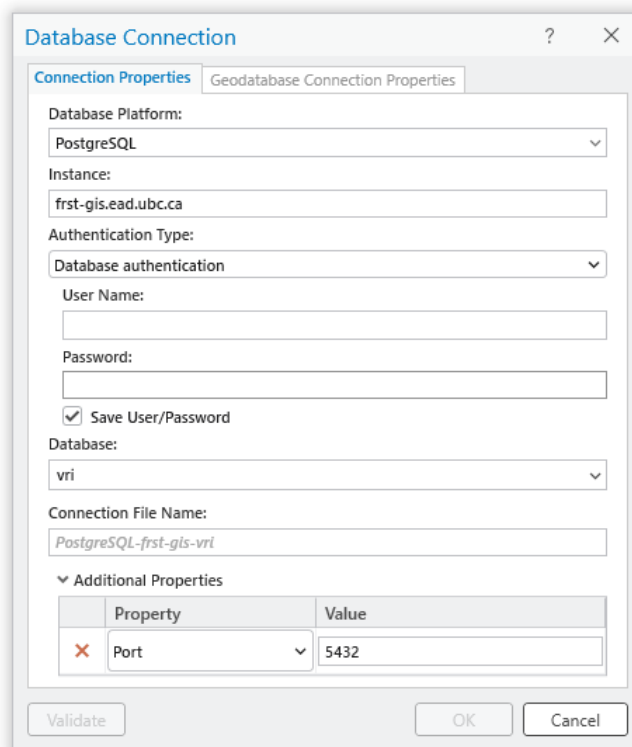
Step 1: Ensure that you are authenticated through UBC myVPN. If you are at UBC, use your dedicated ethernet port or connect via the ubcsecure wireless network. If you are away from campus, you will need to first connect to the UBC myVPN service using Cisco AnyConnect Secure Mobility Client. Only authenticated users with a Campus Wide Login (CWL) who are connected to the UBC myVPN may access the UBC PostgreSQL server. However, if you are not a UBC student, then you can just download the data directly from the BC Data Catalogue.

Step 2: Start ArcGIS Pro and create a new Map project.

Step 3: From the top ribbon, navigate to the “Insert” tab, then click “Connections”, and from the drop-down menu, select “New Database Connection”.



You can name it “UBC PostgreSQL Server” and for the host enter “FRST-GIS.ead.ubc.ca”. The username and password will be provided to students enrolled in the course and it is not your CWL. Once you have correctly entered the correct credentials, ArcGIS Pro will do a “soft” connection to the server to retrieve the available databases. From the “Database” drop-down menu, select “vri” and then click “OK”.



The database connection has now been added to your ArcGIS Pro project, and you can view the tables inside the database from the Catalog Pane (open the “View” tab, open the “Catalog Pane”, expand “Databases”, expand “PostgreSQL-FRST-GIS-vri(student)”, and view the tables in the `vri` database). You can drag and drop any of the layers of the database directly into your map to visualize them if you want. Though be aware that these are large layers and they may take a while to draw on your map. We do not actually need all of these data, so instead, we are going to use the power of SQL (Structured Query Language) to write a query that will return to us the tiny fraction that we need to use for this lab. First, let us break down what a generic SQL SELECT query looks like:

```
SELECT gid, site_index, species_cd_1, species_pct_1, geom
FROM vancouver_island_vri
WHERE bec_zone_code = 'CWH';
```

The **SELECT** keyword is the most common database operation and the only one that we will use for this lab, but understand that there are keywords for other tasks in the database like creating tables, inserting data, updating data, and deleting data, to name a few. By convention, SQL keywords are capitalized, but most software including ArcGIS Pro do not require this.

What follows the **SELECT** keyword is a comma-separated list of column names that we want to retrieve from the table, `gid`, `site_index`, `species_cd_1`, `species_pct_1`, `geom`. You could also replace these with `*` to return all of the column names and this is generally the default behavior across ArcGIS Pro, so for most tasks in ArcGIS Pro you generally will not need to specify any column names when writing a SQL query.

- `gid` is the primary key of the VRI table, this uniquely identifies all of the polygons from each other. When we select any subset of a table, we always want to grab the primary key because this allows us to keep track of which polygon we are dealing with from the originating table.
- `site_index` is the estimate of site productivity for tree growth for the polygon; the values are recorded as decimal numbers.
- `species_cd_1` is the leading tree species code for the polygon; the values are recorded as text strings.
- `species_pct_1` is the percentage of the polygon occupied by the leading species represented by the `species_cd_1`; the values are recorded as integers. Note that “occupy” here is defined differently, depending on whether the stand is young or mature.
- `geom` is the geometry column, which is where the coordinates of the polygons are actually stored. We need this column if we want to map the results of our query.

There are 190 other attributes (columns) in the VRI table for each polygon,

so this is just a small subset. You should review the VRI Relational Data Dictionary if you have any doubts about what a particular field refers to.

Next, the statement describes the table name in the database that these columns are being selected from, `FROM veg_comp_lyr_r1_poly_2024`.

The `WHERE bec_zone_code = 'CWH'` clause is a conditional statement, which limits the rows (features) returned from the originating table. Note here that we can refer to any column names from the originating table, even if they do not appear in the list of column names that we want to return, as long as they exist in the originating table. Finally, all SQL queries are always concluded with a semicolon `;`.

You should think of `SELECT` as *subsetting columns of a table* (along the x-axis) and `WHERE` as *subsetting rows of a table* (along the y-axis). When used together, we are reducing both dimensions of the originating table simultaneously, which substantially reduces the total data that we are retrieving. We will use this principle and expand the SQL statement above in the next step in order to retrieve only the tiny fraction of the VRI data that we need.

Step 4: From the top ribbon in ArcGIS Pro, navigate to the “Map” tab, then click the down arrow under “Add Data” and select “Query Layer...” from the drop-down menu. In the “New Query Layer” dialogue window that appears, click the “Datasource” drop-down menu at the top and you should see the UBC PostgreSQL server connection to the `vri` database listed there that you added at the beginning of this task. It should look something like “PostgreSQL-FRST-GIS-vri(student).sde”.

If not, then click the little database connection button next to the drop-down menu in the dialogue window to add the database connection. (Note: in this specific case, you will need to add the port number to the path string like `FRST-GIS.ead.ubc.ca,5432`)

We are going to write a select query to retrieve these data from the server, but we do not need all the columns, nor all the rows. We only need polygons that are managed forests and for those polygons, we only need a few attributes that we are going to use later in the lab (“`bec_zone_code`”, “`bec_subzone`”, and “`proj_age_1`”).

Step 5: Toggle on “List of Tables” to see the table names and then click on the VRI table in the list “`vri.public.vancouver_island_vri`” to show the list of columns. (If you get an error message here, then save your project, restart ArcGIS Pro and try again). We are going to write a SQL query to select the polygons that meet some of our initial criteria and then return the values of the columns (attributes) that we will need later. In the “Name” field, name the new query layer “`vancouver_island_vri_select`”.

The VRI data include descriptions about both forested and non-forested lands in British Columbia. Since we are only concerned with forests, we are going to only find polygons that are managed forests. We can do this simply with

`SELECT * FROM vancouver_island_vri WHERE for_mgmt_land_base_ind = 'Y'`. See here that we have not defined any specific column names to return, `SELECT *` means *give me all the columns*, so in this example, we are only subsetting on the rows with `WHERE for_mgmt_land_base_ind = 'Y'`, but not the columns. If instead we used `SELECT for_mgmt_land_base_ind FROM vancouver_island_vri`, then we would get all the polygon values for only the `for_mgmt_land_base_ind` column name (subset columns, but not rows). Run the query and inspect the output as a new layer.

Q1. What does `for_mgmt_land_base_ind` select for? What parameters are required to receive a “Yes” value? Hint: take a look at the VRI Relational Data Dictionary. (10 points) Pro tip: When a column value stores a text string, we use single quotation marks in SQL `'abc'` around the values that we are searching for; double quotation marks `"abc"` will fail for a SQL query.

Step 6: Using what you just learned, draft a select statement that limits the VRI polygons to only managed forests and return only the column names `gid`, `bec_zone_code`, `bec_subzone`, `proj_age_1`, and `geom`. Press the “Validate” button on the bottom of the dialogue window. If the statement validates successfully, then the “Next” button will allow you to proceed.

Pro tip: If you are working from a laptop, be sure to plug into your power source as your computer power options may significantly slow down your ArcGIS Pro performance to conserve battery.

Step 7: In the next dialogue, toggle `gid` as the field that uniquely identifies all features in the table, “Unique Identifier Field(s)” (i.e., the primary key), leave the other fields toggled off. Once you click “Finish”, your select statement will be applied and sent to the server. You will see a small dialogue window that indicates that the extent is being calculated, and this should take about 3-5 minutes before you will see a layer appear in your Table of Contents. Read on to understand the next steps while the server processes your query.

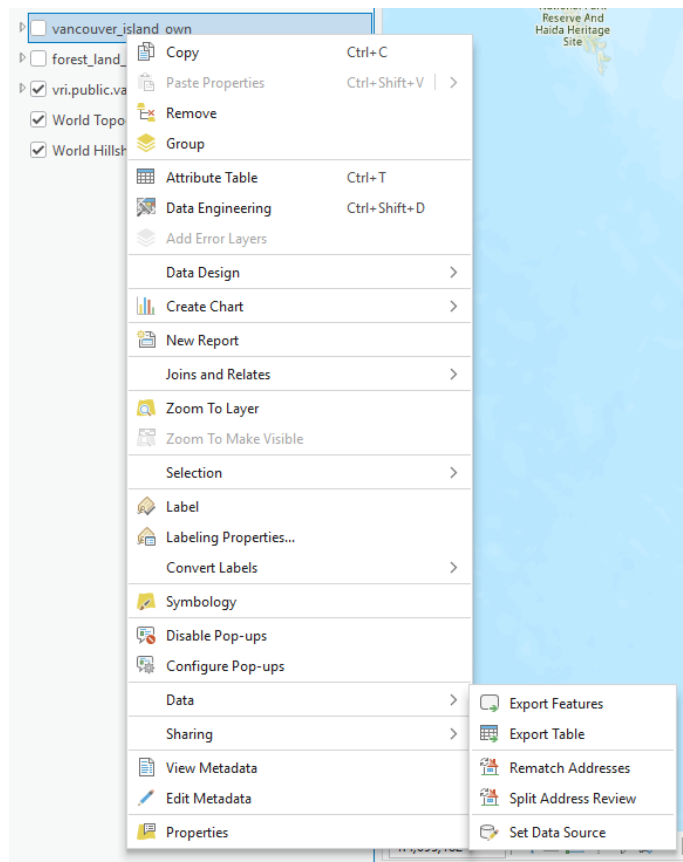
Pro tip: It will take another 3-5 minutes before you will see any polygons drawn on your map. You can speed some steps up if you “Pause” the map drawing mode by pressing the pause button at the very bottom of your map. In general, try to keep large layers toggled off so that they are not drawing in the background.

Save your ArcGIS Pro project now.

Our goal here is not necessarily to visualize the results, though you should see you map automatically pan to Vancouver Island. You can toggle the layer off and visualize it later. If you open the attribute table, you should see the five columns we requested and there should be 250,000+ polygons. (Click the “Load All” button at the bottom of the attribute table to verify, which will take some additional time).

The query layer that you just created is persistent between ArcGIS Pro sessions if you saved and re-open the project later, but the query layer does not actually contain any data because it just provides the instructions to apply the query to the remote database. It will re-run the query each time you re-open your ArcGIS Pro project, which will take some time to retrieve the data. To avoid this, we are going to actually download the data to our local computer.

Step 8: Export the selection you have made your ArcGIS Pro project geodatabase by right-clicking the “vancouver_island_vri_select” layer in your Table of Contents, select “Data”, then select “Export Features”. In the dialogue window that opens, change the output filename to “forest_land_base” (Note: layers stored in geodatabases do not have file extensions), the default location is already the geodatabase for your ArcGIS Pro project. Click “Ok”. This should take about 5 minutes to complete.



Next, we are going to export the other layers and download them directly to our ArcGIS Pro project geodatabase.

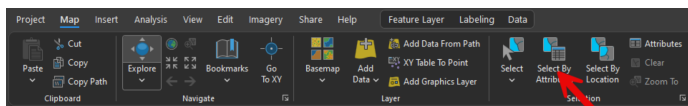
Step 9: From the “Catalog Pane”, export the “vancouver_island_own”

layer to your ArcGIS Pro project geodatabase by right-clicking the “vancouver_island_own” layer, select “Export”, then select “Export Features...”. In the dialogue window that opens, change the output filename to “vancouver_island_own” (Note: layers stored in geodatabases do not have file extensions), the default location is already the geodatabase for your ArcGIS Pro project. Click “Ok”. This should take a few minutes to complete.

Repeat the last step for “vancouver_island_human_disturbance” and “vancouver_island_landscape_units”. Altogether, this is about 600 MB of data, so be sure you have sufficient disk space and be patient as the data transfer completes. When finished, you can drag the layers into ArcGIS Pro to visualize them on the map. If you paused drawing mode earlier, then unpause it now.

The cumulative effects framework only considers forests located on crown land. Next, we will use the forest ownership layer “vancouver_island_own” to select for land ownership types. The `ownership_description` field describes whether the polygon is private or crown land in a text format. There are lots of different types of crown ownership categories, but for our purpose we need all of them. To do this quickly, we can use `LIKE`, which is a pattern-matching operator for strings. We need to use the `%` wildcard to partially-match some pattern. For example, we can search for any row in “vancouver_island_own” with a description of “Crown” using the following statement: `SELECT * FROM vancouver_island_own WHERE ownership_description LIKE 'Crown%'`. Basically `%` is a wildcard for zero or more characters, so when we construct the pattern `'Crown%'`, we are saying *match any string that starts with “Crown” and has any number of characters after it*.

Step 10: Open the attribute table of “vancouver_island_own” by right-clicking the layer in the contents pane. Then click the “Select by Attributes” button then toggle the SQL Editor button to show the SQL codeblock. Write a query to select all the polygons that are crown land as described in the previous step. Note that in this particular interface, you only need to give the conditions for selection, so you can omit `SELECT * FROM table WHERE`. Export the features to your ArcGIS Pro project geodatabase as you did with the the vancouver_island_vri_select, naming the new layer “crown_land” and ensuring that “Use the selected records” is toggled on.



Step 11: Next, we will select the areas where crown land overlaps with potentially forested areas.

Use the **Pairwise Intersect tool**. You can search for it by clicking the toolbox on the top ribbon (open the “Analysis” tab, then click “Tools”).

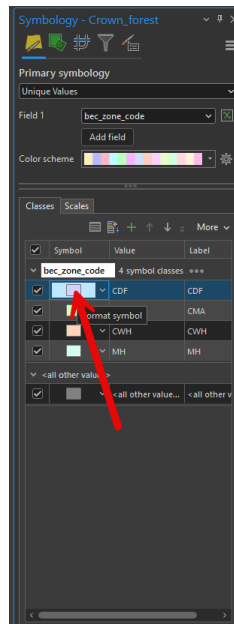
- Input: forest_land_base, crown_land

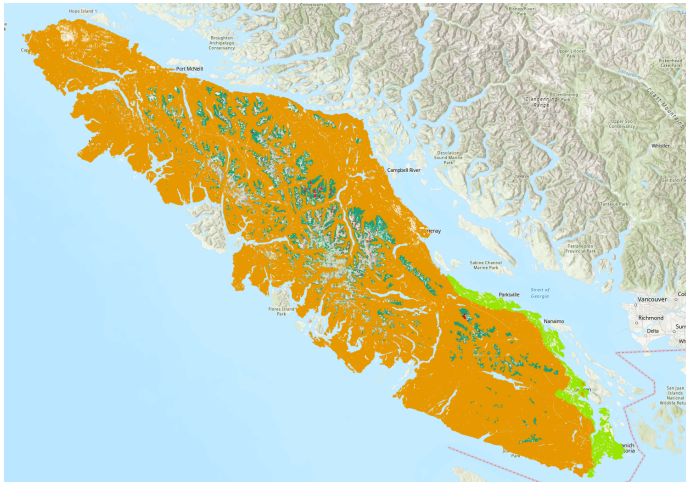
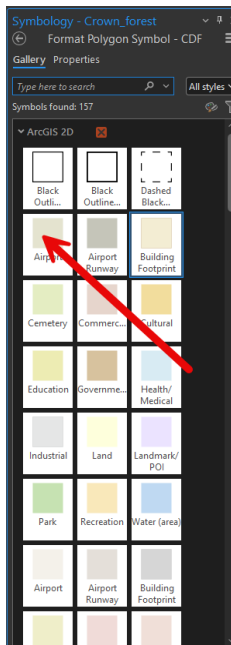
- Output: `Crown_Forest`

Once the tool has finished running, open the new `Crown_Forest` attribute table to ensure that “`bec_zone_code`” and “`bec_subzone`” attributes have been preserved.

Step 12: Turn off visibility for other layers in the contents pane on the left side. If the contents pane has been closed, it can be reopened from the ribbon at the top of the screen. Visualize the `Crown_Forest` layer by `bec_zone_code` or `bec_subzone`. To do this, right-click the layer in the contents pane and select “Symbology”. Change the primary symbology to “Unique Values” and “field 1” to the attribute of your choice.

Pro tip: Polygon outlines can be removed by clicking the symbol of each row in the “Classes” table of the symbology layer. When given the option, select a symbol that does not have an outline. This makes it easier to see the area of many smaller polygons combined.





Step 13: In BC forestry standards, the age of a forest stand is categorized by “Seral Stage.” Since the required ages for mature or old growth forests varies depending on the BEC zone, seral stages are used to denote the relative age class of a stand; “Early”, “Mid”, “Mature”, and “Old”. We will create a new field and classify each polygon depending on its estimated age.

Open the **Crown_Forest** attribute table and select “Add” in the table ribbon. This will take you to the field view. At the bottom, create a new field using:

- Name: “Seral_Stage”

- Data Type: Text

To save this new field, click “Save” on the Ribbon at the top of the window. Once it is saved you can close the field view and return to the **Crown_Forest** attribute table. Once it is saved you can close the field view and return to the **Crown_Forest** attribute table.

Step 14: Right-click the new field and select “Calculate Field”. The age categories for each seral stage depend on which BEC zone the stand falls within. These age categories correspond with the growth rate and productivity of different eco-zones. The provided code defines the breakdown of Early, Mid, Mature and Old for the three BEC zones found on Vancouver Island.

In the expression field paste the following code. This takes the BEC zone code and estimated age as inputs into a function that we will define.

In the code block under “Seral_stage =”, paste:

```
getSeralStage(!BEC_ZONE_CODE!, !PROJ_AGE_1!)
```

In the “Code Block” paste:

```
def getSeralStage(bec, age):
```

```
    if bec == 'CWH':
        if age <= 40:
            return 'Early'
        elif age <= 80:
            return 'Mid'
        elif age <= 250:
            return 'Mature'
        else: return 'Old'
```

```
    elif bec == 'MH':
        if age <= 40:
            return 'Early'
        elif age <= 120:
            return 'Mid'
        elif age <= 250:
            return 'Mature'
        else: return 'Old'
```

```
    elif bec == 'CDF':
        if age <= 40:
            return 'Early'
        elif age <= 80:
            return 'Mid'
        elif age <= 250:
```

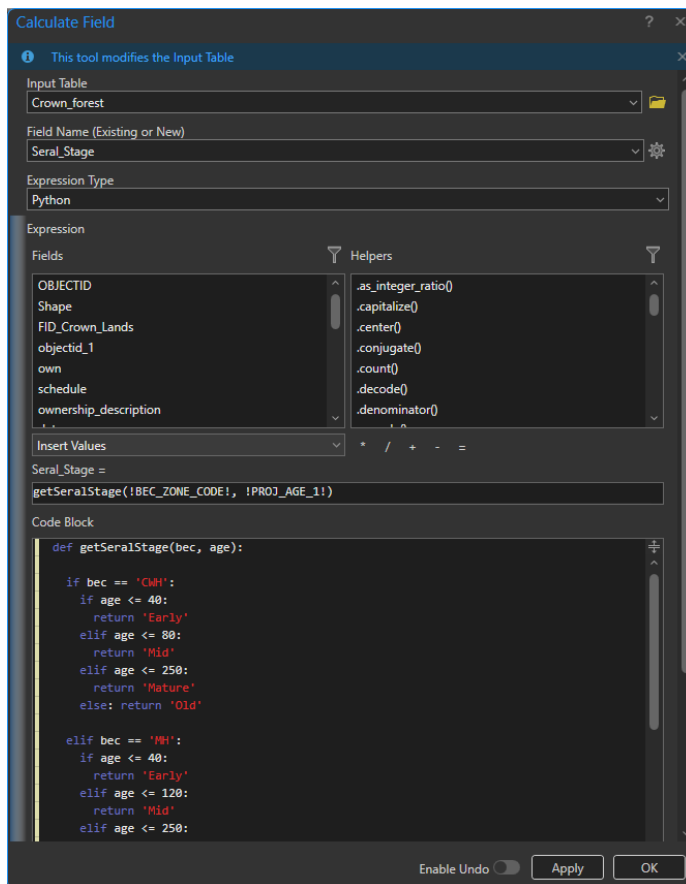


```

        return 'Mature'
    else: return 'Old'

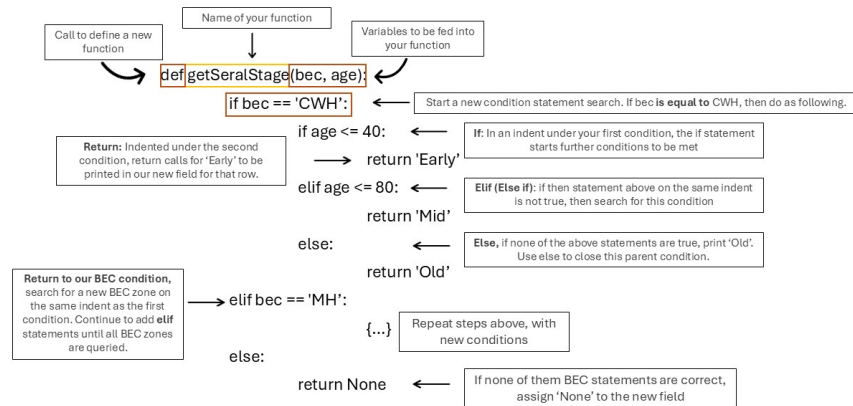
else:
    return None

```



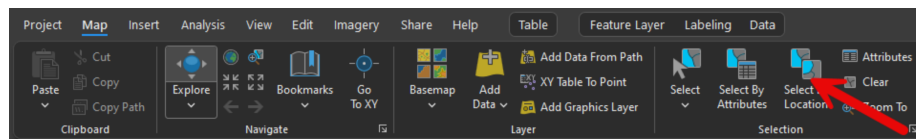
Note: Ignore the warning message that appears.

To better understand what is being done in this code chunk, the syntax is described below.



Check that all the rows have been populated in the `Seral_stage` field.

Q2. Define “old growth” and cite at least one primary source. What characteristics might be overlooked by a definition of “old growth” that only accounts for stand age? (10 points) **Step 15:** Our current layers only indicate where crown forests *could potentially occur*. In this step, we will reset the seral stage in areas that have experienced human disturbances such as roads, buildings, or logging. We are not removing these areas entirely, as they remain part of the landscape unit where old-growth forest could exist if it had not been cleared. We will use `vancouver_island_human_disturbance` to do so. Select by attribute where `cef_human_disturb_flag` is equal to `Human Disturb Current 20yr`. Export the selected attributes as a new feature named `Disturbed_areas`.



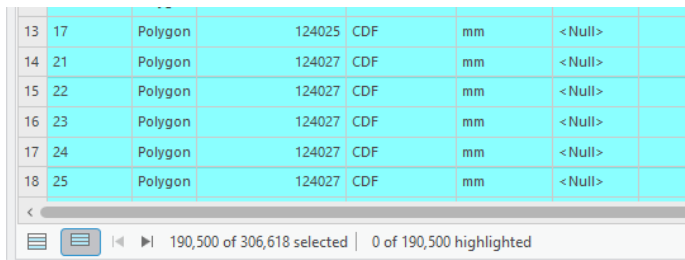
Then, use **Select By Location** in the top ribbon to identify areas of overlap between potential crown forest stands and areas of disturbance.

- **Input Features:** `Crown_Forest`
- **Relationship:** `Intersect`
- **Selecting Features:** `Disturbed_areas`
- **Selection Type:** `New Selection`

Leave other options with their defaults. This may take a couple of minutes to complete.

Step 16: In the Crown_Forest attribute table:

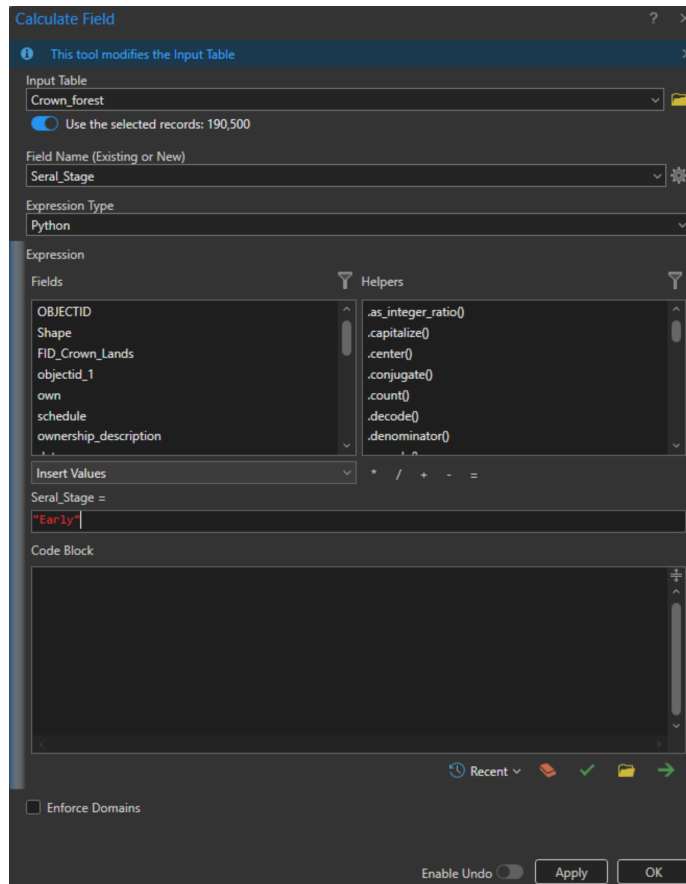
- Click “Show Selected Records”



13	17	Polygon	124025	CDF	mm	<Null>	
14	21	Polygon	124027	CDF	mm	<Null>	
15	22	Polygon	124027	CDF	mm	<Null>	
16	23	Polygon	124027	CDF	mm	<Null>	
17	24	Polygon	124027	CDF	mm	<Null>	
18	25	Polygon	124027	CDF	mm	<Null>	

190,500 of 306,618 selected | 0 of 190,500 highlighted

- Right-click the Seral_Stage field, then click “Calculate Field”
- **Expression:** "Early"
- Ensure that **Use the selected records** is toggled on
- Remove your previous code text if it still remains. Type “Early” (with quotes) into the expression box and leave the Code Block empty.
- Click “Apply”
- Click “Clear” to remove selection



Pro tip: Always clear your selection once you are done working with it. This prevents potential missteps in later steps.

Step 17: Old growth forest quantities are assessed in terms of Land Units and BEC Subzones. Land Units are land parcels pre-determined by the BC Government for ease of land management and assessment. We will need to divide our current Crown Forest Landbase by the pre-determined Landscape Units.

Open the **Pairwise Intersect** tool again:

- Input: Crown_Forest, vancouver_island_landscape_units
- Output Name: CrownForest_LU

Task 2: Calculate Percentage of Old Growth Forests

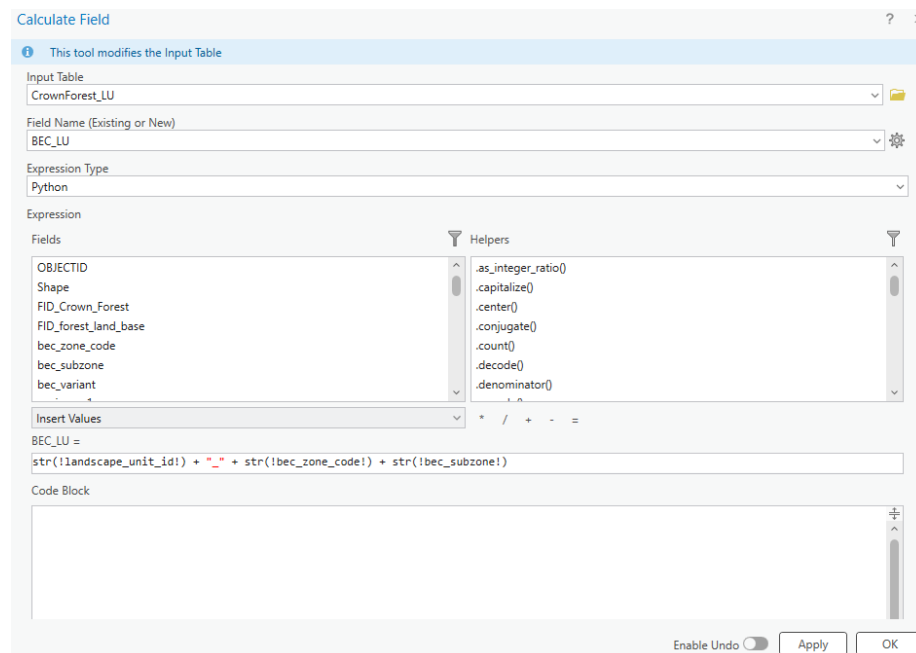
Step 1: Now that we have prepared our layers, we will calculate the percentage of old growth forest within each land unit. To do this we will create a key that uniquely identifies each Landscape Unit + BEC zone + BEC subzone combination. This is a good opportunity to close any unneeded attribute tables or turn off layer visibility.

Add a new field to CrownForest_LU attribute table:

- Add a new field to **CrownForest_LU**
- Name: **LU_BEC**
- Type: **Text**

In the `LU_BEC =` textbox, paste:

```
str(!landscape_unit_id!) + "_" + str(!bec_zone_code!) + str(!bec_subzone!)
```

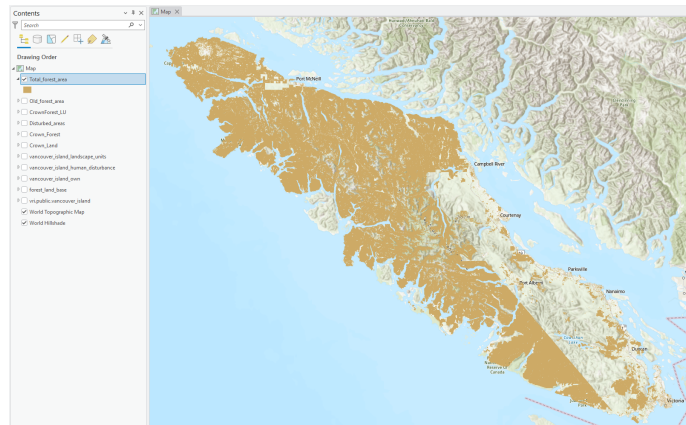


The `str()` function ensures that the attribute values are treated as text strings rather than numeric values for addition. The result should be a new string of landscape unit, BEC zone and subzone.

Step 2: We will now calculate the total amount of forested area in each of these landscape unit/BEC zone combination. With the **Dissolve** tool, we can create new polygons from the landscape identifier:

- Input Table: **CrownForest_LU**
- Dissolve Fields: **LU_BEC**
- Statistics Fields: **Shape_Area** as SUM and **bec_subzone** as FIRST
- Output Table: **Total_Forest_Area**
- Check “Create Multipart Features”

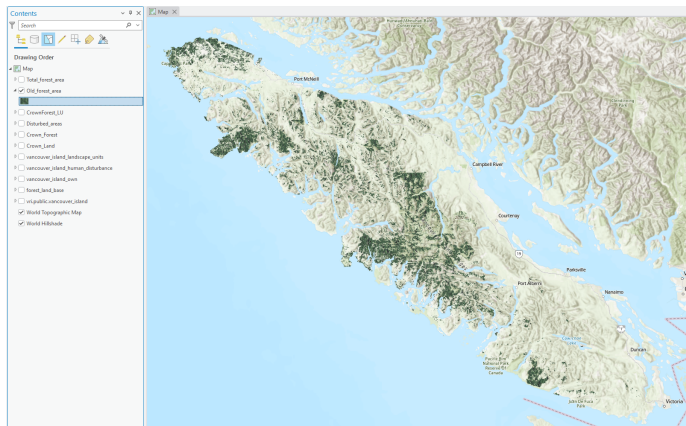
This creates a polygon layer where each feature is one LU_BEC area with total crown forest area as an attribute.



Step 3: To identify areas of assumed old growth, we will calculate the area of only forests where **Seral_stage** equals **old**. We can select for old forest by using **Select by Attributes on CrownForest_LU** before using the **Dissolve** tool. Ensure that the “Use the selected records” option is switched on.

- Select by Attribute: **Seral_Stage = "Old"**
- Dissolve Fields: **LU_BEC**
- Statistics Fields: **Shape_Area** as SUM
- Output Table: **Old_Forest_Area**
- Ensure **Use the selected records** is on
- Check “Create Multipart Features”

Now we have created a table of total crown forest area and a table of old growth crown forest area, each by landscape unit.



Step 4: We can join these tables back together using a

Use **Join Field** with the following expression:

- Input Table: Total_Forest_Area
- Input Field: LU_BEC
- Join Table: Old_Forest_Area
- Join Field: LU_BEC
- Transfer Field: Shape_Area (from Old_Forest_Area)

Geoprocessing

Join Field

This tool modifies the Input Table

Parameters Environments

Input Table: Total_forest_area

Input Field: LU_BEC

Join Table: Old_forest_area

Join Field: LU_BEC

Transfer Method: Select transfer fields

Transfer Fields: Shape_Area

Index Join Fields: Do not add indexes

Validate Join

We now have two fields in `Total_forest_area` named `Shape_Area` which could lead to some confusion. We can assume that the column with the smaller numbers is the old growth forest area. This should be the last column in the attribute table. Right-click the attribute name to open the field view. Rename the alias of `Shape_Area` to `Total_Area` and the alias of `Shape_Area_1` to `Old_Area`. Save your changes.

Step 5: Calculate Percent Old Growth

Add a new field to `Total_Forest_Area`:

- **Name:** PercentOld
- **Type:** Double

Use **Calculate Field** with:

- **Expression:**

```
calcPercent(!Shape_Area_1!, !Shape_Area!)
```

Note: the expression still uses the column name rather than the alias.

- **Code Block:**


```
def calcPercent(old, total):
    if old is None or total is None or total == 0:
        return 0
    return (old / total) * 100
```

Step 6: Classify Percent Compared to Provincial Targets.

The target percentages of old growth forests are dependent on the BEC subzone. Instead of assigning each LU-BEC to a discrete class (Low / Intermediate / High), in this step you will create a numeric field representing the difference between the calculated percent old growth and the subzone-specific “high” threshold. You will do this by writing your own python expression to calculate the difference. Refer to the code in step 10 for a guideline on how to build the function. This difference (PercentOld - HighThreshold) is what you will visualize as a continuous gradient:

- Positive values exceed the high target,
- Negative values fall short of the high target,
- Zero meets the high target exactly.

Begin by adding a new field to **Total_Forest_Area**:

- Name: DiffFromHigh
- Type: Double

The percentage thresholds for each class is as follows:

Subzone Code	Name	Low (less than):	High (greater than):
CDFmm	Coastal Douglas-fir	9	13
CWHmm	Coastal Western Hemlock Moist Maritime	9	13
CWHvh	Coastal Western Hemlock Very Wet Hypermaritime	13	19
CWHvm	Coastal Western Hemlock Wet Hypermaritime	13	19
CWHxm	Coastal Western Hemlock Very dry maritime	9	13
MHmm	Mountain Hemlock Moist Maritime	19	28

Use **Calculate Field** with:

- Expression:

```
diff_from_high(!LU_BEC!, !PercentOld!)
```

Now write your own expression to calculate the difference between the old growth target and the calculate old growth percentage.

Screenshot 1. Upload a screenshot of your python expression (10 points) **Pro Tip:** The subzone code can be isolated by removing the first 5 characters with a slice. This would look something like this:

```
bec = lu_bec[5:]
```

We read this as *give me all the elements from position 5 to the end of the string*, where the colon : separates the “from” and “to” values. If we leave one side of the colon blank, then we are saying we want whatever the minimum (left side of colon) or maximum (right side of colon) number of elements is. Python indexing starts at zero, so if I want to return the letter “g” from the alphabet string `abcdefghijklmnopqrstuvwxyz`, then I would write something like `alphabet[6]` or if I want all letters before “g”, then `alphabet[:5]`.

OBJECTID	Shape	LU_BEC	Shape_Length	Total_Area	Old_Area	PercentOld	DiffFromHigh
1	Polygon	1627_CWHvh	685934.187898	404157323.892434	140984601.454016	34.883594	15.883594
2	Polygon	1627_CWHvm	664612.509813	414153045.770105	15580246.981542	3.761954	-15.238046
3	Polygon	1627_MHvm	12089.612881	594224.522068	<Null>	0	-28
4	Polygon	1630_CWHvh	1632934.364127	268270282.514044	139324178.128542	51.934257	32.934257
5	Polygon	1630_CWHvm	44873.699209	21992383.857746	2302402.51689	10.46909	-8.53091
6	Polygon	1631_CWHvh	530838.309977	136569462.899258	84311044.888369	61.734917	42.734917
7	Polygon	1638_CWHvh	274864.403272	79633222.740218	38746294.575689	48.655942	29.655942
8	Polygon	1638_CWHvm	238860.100607	127104597.351475	31247096.542911	24.583766	5.583766
9	Polygon	1640_CWHvm	908678.910536	324949651.421247	12663930.244541	3.897198	-15.102802
10	Polygon	1640_MHvm	159942.014153	16342406.595165	2676229.500521	16.375982	-11.624018
11	Polygon	1641_CWHvh	179755.988322	39448092.917205	9683010.930447	24.546208	5.546208
12	Polygon	1641_CWHvm	704716.178475	322452457.354111	12578936.639955	3.901021	-15.098979

Step 7: Visualize Results

In the **Contents** pane, right-click the **Total_Forest_Area** layer and select **Symbology**. In the Symbology panel, set the **Field** to **DiffFromHigh**. This will display the data based on the values in that field. Under **Primary symbology**, choose **Graduated Colors** (a gradient).

Select a **color ramp** that makes sense for your data. For values that represent *low to high* differences, use a single-hue light-to-dark ramp (e.g., light yellow to dark green). Avoid using colors that are difficult to interpret (e.g., bright rainbow colors). - Think about your audience—pick colors that clearly show the trend and are readable to someone with color-vision deficiencies. Adjust the **classification method and number of classes** if needed (e.g., Natural Breaks, Quantile). This determines how your data values are grouped into color ranges.

Step 8: Create the Layout Map

From the top ribbon, go to the **Insert** tab and select **New Layout**. Choose an appropriate page size (e.g., Letter 8.5x11 or A4). Use the **Map Frame** tool to insert your map view into the layout. Resize and position it so it fills most of the page. - Add essential map elements: - **Title**: Insert a descriptive map title (e.g., “*Difference from High Threshold in Total Forest Area*”).

- **North Arrow**: select “Insert”, then select “North Arrow”. Place it in a

corner where it does not overlap important data.

- **Legend:** select “Insert”, then select “Legend”. This should explain the symbology of your `DiffFromHigh` values.
- **Scale Bar:** select “Insert”, then select “Scale Bar”. A useful addition if map distance matters.
- **Author/Date/Text:** Add your name and date for documentation.

Rearrange and resize elements so the layout looks clean, balanced, and easy to read. Avoid cluttering the map with overlapping items.

Step 9: Create the Layout Map

When you are satisfied with the layout, go to **select “Share”, then select “Export Layout”**. Choose **PNG** as the file format.

Q3. Describe the results of your final output. What is the distribution of old growth targets met across Vancouver Island? (10 points)

Q4. Select 3 other fields in the VRI layer and briefly discuss how they could provide further insight into current old growth status. (10 points)

Q5. How is VRI data collected and what limitations does it pose? (10 points)

Q6. How might the scale and resolution of GIS data affect the accuracy of old growth assessments? (10 points)

Table 1. A table with fields for total area, old-growth area, and percentage old growth. (10 points)

Map 1. A map showing the study area and the distribution of difference in calculated old-growth percentage and targets, by polygon. (20 points)

References

British Columbia & BC Environment (Eds.). (1995). *Biodiversity guidebook*. Forest Service, British Columbia: BC Environment.

Canadian Institute of Forestry (2022). *Old Growth Information bulletin*. March, 2022

CBC. (2022, February 11). Fairy Creek protesters bid to have charges stayed. *CBC News*. <https://www.cbc.ca/news/canada/british-columbia/protest-court-proceedings-1.6348014>

Forest Analysis and Inventory Branch (2024). VRI - 2024 - Forest Vegetation Composite Rank 1 Layer (R1). British Columbia Data Catalogue.

LePage, P., & Banner, A. (2014). Long-term recovery of forest structure and composition after harvesting in the coastal temperate rainforests of northern British Columbia. *Forest Ecology and Management*, 318, 250–260. doi.org/10.1016/j.foreco.2014.01.031

Ministry of Water, Land and Resource Stewardship (WLRS). (2024). Old Growth Forests in British Columbia: Cumulative Effects Assessment Backgrounder. Victoria, British Columbia.

Ministry of Water, Land and Resource Stewardship. 2024. Old Growth Forest Management in British Columbia: Provincial Backgrounder. Victoria, British Columbia.

Summary

In this lab you standardized VRI layers, filtered the Forest Management Landbase, calculated old-growth areas, and derived percentages to compare across polygons. You then produced a map and exportable table to communicate your findings. The key skills reinforced were: consistent data preparation, careful attribute logic for inventory fields, transparent calculation workflows, and interpreting results within an ecological and management context.

Return to the **Deliverables** section to check off everything you need to submit for credit in the course management system.

Chapter 4

Exploring QGIS and OpenStreetMap

Written by Paul D. Pickell

Lab Overview

In this lab you will learn some basic functionality of handling data with the QGIS software and learn how to extract and import an important source of Volunteered Geographic Information (VGI): OpenStreetMap (OSM). QGIS (Quantum GIS) is a Free and Open Source Software (FOSS) that is a flagship project maintained by the Open Source Geospatial Foundation (OSGeo). QGIS features much of the same functionality as other proprietary software like ESRI's ArcGIS and navigating the interface will be the primary purpose of this lab. OpenStreetMap is one of the most popular and comprehensive VGI datasets available for the planet and contains geographic information that is crowd-sourced. The spatial data model of OpenStreetMap data is significantly different from other vector models and therefore requires some special handling and consideration. At the end of this lab, you will produce a map of some OpenStreetMap data using QGIS.

Learning Objectives

- Import and export various types of geospatial data in QGIS
- Practice spatial and attribute queries
- Calculate fields with new values
- Install plugins and extensions for QGIS

- Create map layouts and modify layer symbologies
-

Deliverables

Answers to the questions posed throughout the lab (15 points)

Your final symbolized map (15 points)

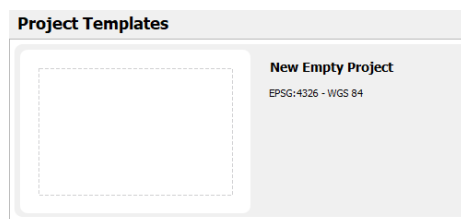
- The map should show the Voronoi polygons symbolized with a quantile classification that is 50% opaque
 - The OpenStreetMap nodes need to be symbolized and distinguishable underneath the Voronoi polygons
 - You need to use the OpenStreetMap basemap
 - You can add additional layers, but be sure to avoid making the map too cluttered
 - All features on the map should appear in the legend
 - Map should be 11"x17" either as a landscape or portrait layout
 - You should export the map as a PDF document
 - Your map should have a title, north arrow, scale bar, legend, your name, and date
-

Data

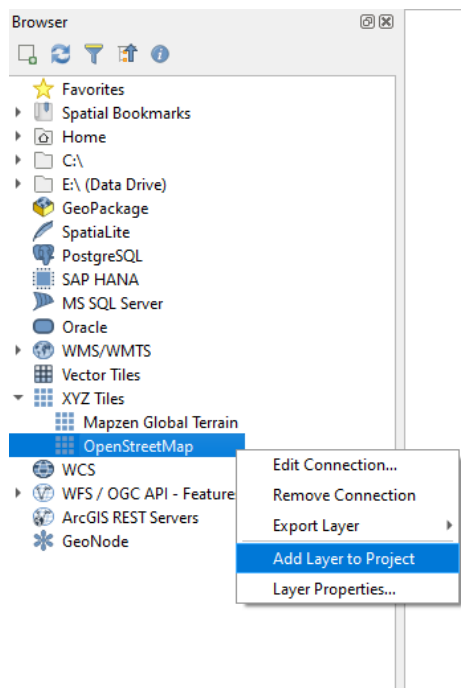
Data for this lab will be downloaded directly from OpenStreetMap following the tasks below.

Task 1: Downloading OpenStreetMap data to QGIS

Step 1: Start QGIS and click **New Empty Project** from the Project Templates on your screen.

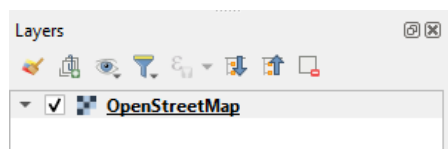


The Project Templates prompt will disappear to a white canvas, which is where you are able to view mapped data. On the left of your screen you will see two panes: **Browser** and **Layers**. These are where you can find data sources on your computer or network and the layers that are currently loaded into your map view. Currently, we do not have any data to view, so the Layers pane will be blank. But we do have many sources for data, so let us look at those. Depending on your computer and networking connections, your Browser pane may look different than what is shown below.



Step 2: Save your project. In the top menu, select **Project** then click **Save As...** and navigate to a location on your computer where you want to store your project and related files. You should create a directory specifically for this purpose.

Step 3: Expand **XYZ Tiles** and then either double-click **OpenStreetMap** or right-click and select **Add Layer to Project**.

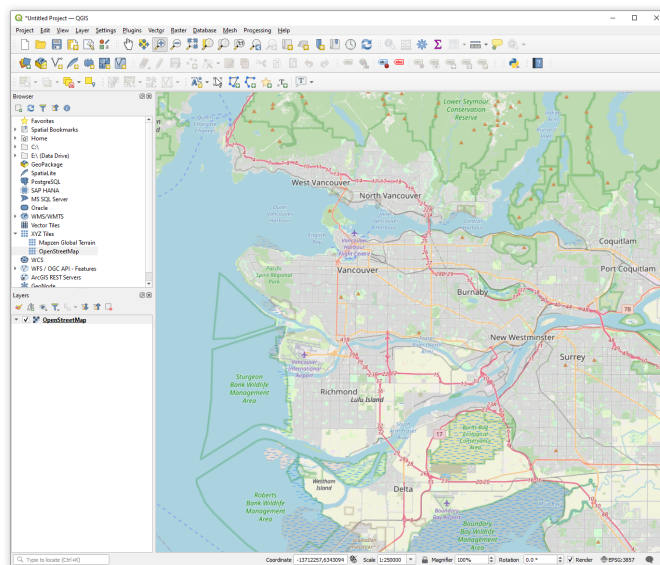


This will add the OpenStreetMap base map to your map view, so you should now see your first layer in the Layer pane and the map view should automatically

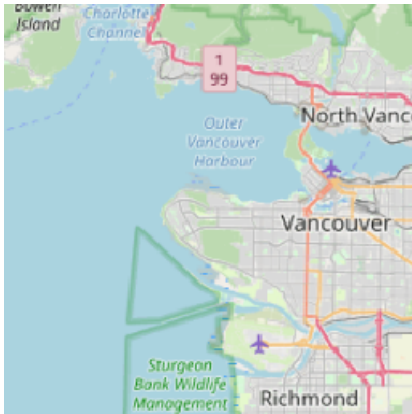
display the new base map.



Step 4: Locate the Map Navigation Toolbar at the top. Use the magnifying glass to zoom into the area for Metro Vancouver. Once you find Metro Vancouver, change the scale of the map to 1:250000 at the bottom of your QGIS interface.

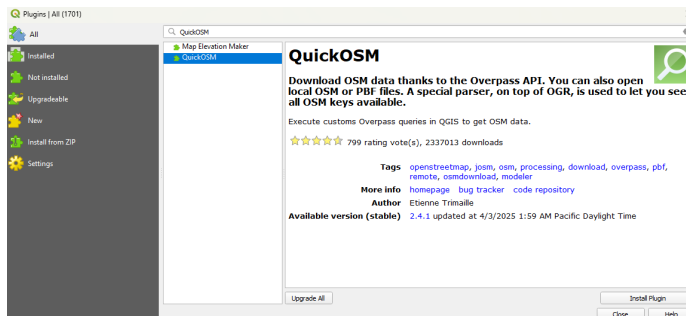


This current view does not directly show you the OpenStreetMap data. Instead, you are looking at a tile service provided by OpenStreetMap, which symbolizes the OpenStreetMap data and then, depending on your zoom level and location, sends the necessary information to your computer in the form of a tile like the one pictured below showing a 256x256 pixel tile at zoom level 10 (zoom level 1 is the smallest scale world map).



In order to get the OpenStreetMap data, we need to install a plugin for QGIS. Plugins are written by a community of volunteer developers, some from organizations that pay for the plugin to be developed and then released as FOSS, and others just volunteer their time. Plugins are incredibly important for QGIS, because only basic features end up in the core GIS software. If there is ever something you want to do that is not a basic feature of QGIS, you probably need to search for a plugin or write one yourself!

Step 5: From the menu bar at the top, click **Plugins** and then select **Manage and Install Plugins...** to open the **Plugin Manager**. Since these plugins are fetched from a server, you need to be connected to the internet to see plugins available for download. In the search bar, search for the **QuickOSM** plugin. Click **Install Plugin**.



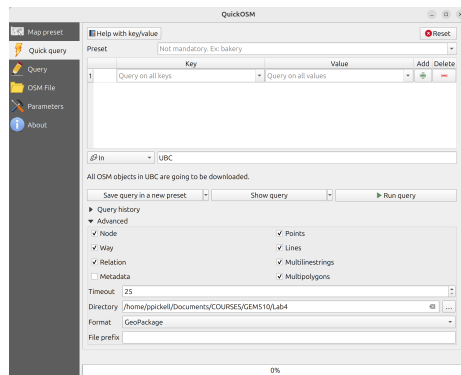
Once the plugin is installed, you can click **Close** to dismiss the Plugin Manager. Most plugins will automatically add an icon to your toolbar at the top, so search for and click the QuickOSM icon that looks like this:



Step 6: When the QuickOSM dialogue window opens, select “Quick Query” from the left navigation panel. Find the field where it says “A village, a town...” and type “UBC” in the field, leave it set to “In”. Note that you can also apply filters to subset the OSM data based on key-value pairs, but for this lab,

we are going to grab all of the data for UBC so make sure the filters at the top are empty. Expand the “Advanced” option at the bottom and ensure that “Node”, “Way”, “Relation”, “Points”, “Lines”, “Multilinestrings”, and “Multipolygons” are all toggled on. Finally, in the “Directory” field, navigate to your QGIS project location or folder you are using for the lab and select “Geopackage” from the drop down menu. You do not need to specify an output filename. Run the query.

Note: if you receive a timeout message when you run the query, try re-running the query until it succeeds.



Now, you should now see an updated list of layers available in your QGIS Layers pane.

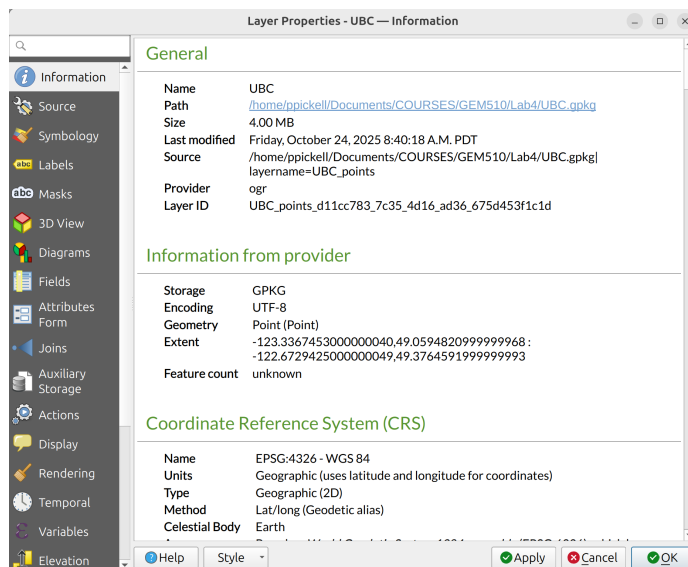
Step 7: Toggle off the OpenStreetMap base map and depending on how you made your selection in the last step, you should see something like below in your map.

Note: if you see fewer number of points (or other features) in your map, then left-click the point layer from your “Layers” panel to highlight it, then click the symbology button at the top-left of the “Layers” panel, then change the symbology type to “Single Symbol” from the drop-down menu. Do not worry if the number of features still looks a bit different after changing the symbology, because OpenStreetMap is constantly being edited since the screenshot was taken.



Task 2: Inspecting and processing OpenStreetMap data

Step 1: Right-click the osm point layer in the Layers pane, select **Properties...** and then select the **Information** tab on the left navigation menu of the **Layer Properties** dialogue box.



You should notice that the **Coordinate Reference System (CRS)** is **EPSG:4326 – WGS 84**. OpenStreetMap data will always be distributed with latitude and longitude coordinates, because this is the native coordinate system used to map the nodes, the fundamental building blocks of the database.

OSM files are comprised of **nodes**, **ways**, and **relations**. These generally correspond to points, lines, and polygons, but not exactly. Each of these feature types are described by **tags** or attributes. Nodes are a pair of latitude/longitude coordinates and can either be a building block for another type of feature (e.g., a way or relation) or they can have attributes of their own or both. A way is a list of two or more nodes that connect and form a line. Ways can be open (lines) or closed (areas). It is important to recognize that a node can participate in multiple ways simultaneously, while also describing itself as a discrete point feature. Relations are just that: they allow us to model relationships between existing nodes, ways, and even other relations. For example, individual islands might be represented by a closed way, but an archipelago would be represented by a relation that contains the closed ways for all the islands. In this way, the OSM data model is flexible and eliminates redundancies of spatial information

such that nodes and ways can be recycled into all kinds of relationships.

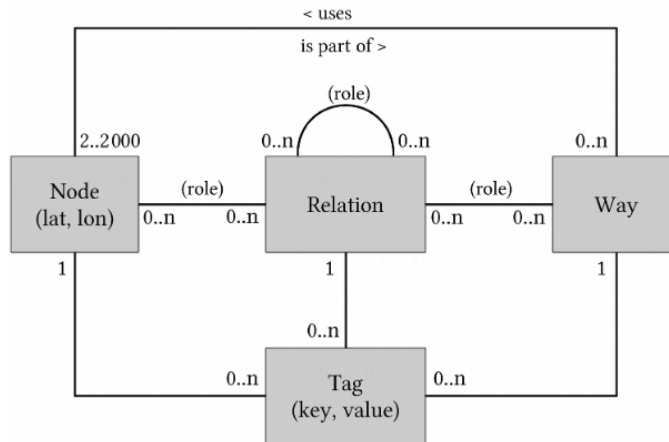


Figure 4.1: Figure Credit: <https://media.jochentopf.com/media/2018-07-30-talk-sotm2018-data-model-en-slides.pdf> Jochen Topf

Q1. Interpret the OSM entity-relationship data model shown below. In a brief sentence, describe each of the seven connections between nodes, ways, relations, and tags. (1 point each for 7 points total)

Step 2: Dismiss the **Layer Properties** dialogue box and then right-click on one of your layers again in the Layers pane, but this time select **Open Attribute Table**. Your table might have a different number of features and different values for the attributes than what is shown below, because OpenStreetMap data are constantly edited.

UBC — Features Total: 5041, Filtered: 5041, Selected: 0									
	fid	full_id	osm_id	osm_type	playground	unisex	denotation	e:maxlength:phax	
1	1	n25251494	25251494	node	NULL	NULL	NULL	NULL	NU
2	2	n25251500	25251500	node	NULL	NULL	NULL	NULL	NU
3	3	n25477650	25477650	node	NULL	NULL	NULL	NULL	NU
4	4	n25477653	25477653	node	NULL	NULL	NULL	NULL	NU
5	5	n25477656	25477656	node	NULL	NULL	NULL	NULL	NU
6	6	n26046238	26046238	node	NULL	NULL	NULL	NULL	NU
7	7	n26046241	26046241	node	NULL	NULL	NULL	NULL	NU
8	8	n26046281	26046281	node	NULL	NULL	NULL	NULL	NU
9	9	n26046285	26046285	node	NULL	NULL	NULL	NULL	NU
10	10	n26046319	26046319	node	NULL	NULL	NULL	NULL	NU
11	11	n26270973	26270973	node	NULL	NULL	NULL	NULL	NU
12	12	n27511886	27511886	node	NULL	NULL	NULL	NULL	NU
13	13	n27699300	27699300	node	NULL	NULL	NULL	NULL	NU

The second field **osm_id** is the primary key of the node for the data you have downloaded. This is the primary key of all nodes in the planetary database

of OpenStreetMap data (not just your file), which is why it starts from a very large number, 25,251,494 in the image above.

The remaining attributes are the tags, which might look random, and that is because OpenStreetMap was originally designed to store street information, but then evolved into a larger participatory community mapping project. But not all OpenStreetMap nodes can logically be described by every tag, which is why we see so many **NULL** values.

Q2. EXTRA CREDIT: How could an Hstore data encoding affect how you can interact with and query the data in a relational database? (3 points) **Step 3:** Inspect the attribute table for your MultiLineString layer.

Q3. What do these features all have in common? (1 point) Multi-LineString is a multipart line feature that is represented as a single tuple in the attribute table with multiple line parts. The line parts need not be connected or continuous, so this feature type allows for multiple line segments to be represented as a single tuple in the attribute table.



Step 4: Navigate to the UBC PostgreSQL database and add the “ubcv_legal_boundary” layer to your project. We are going to use this layer to actually select only the points/nodes that fall within UBC.

Step 5: Use the **Select by Location** tool to select your point layer by the campus polygon you just added from the UBC PostgreSQL server. Toggle on Intersect for the selection. Export the selected points to the same “UBC.gpkg” geopackage in your project and name the layer **osm_points_campus**. Ensure that **Save only selected features** is toggled on when you do the export.



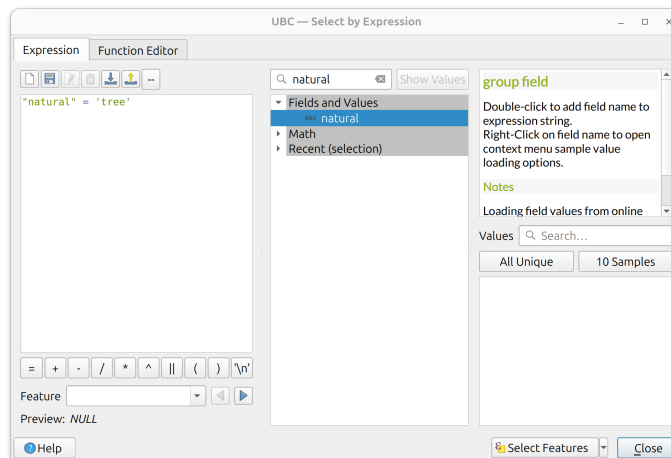
Once you are satisfied with the export, return to the attribute table and click the **Deselect all features** from the layer button at the top. This is an important step, because if you leave the selection, then all future geoprocessing may only occur on the features that you have selected!



Step 6: Open the attribute table for the **osm_points_campus** layer and click the **Select features using an expression** icon.



This displays the Select by Expression dialogue and you can enter in a query to subset the data. The expression goes into the blank left side of the dialogue, the center of the dialogue gives you options for constructing your query, and the right-side displays help information based on what is selected from the center. From the center, expand the **Fields and Values** item. There are a ton of tags! Let us search for a tag instead. Using the search bar above, type **natural**, then under Fields and Values double-click **natural**. This will add the field name to your statement on the left. On the right, click the **All Unique** button to see what values are available for this tag. We will select all trees, so double-click **tree**. Add an equal sign to your statement so that it reads `"natural" = 'tree'`, as shown below. Notice how the key is in “double quotes” and the value is in ‘single quotes’. Click **Select Features** to execute the statement then close the dialogue and inspect the result.



Step 7: With the selection active from the last step, right-click the “osm_points_campus” layer in the Layers pane and select **Export** and then **Save Selected Features As...** and name the output **osm_campus_trees**. Ensure that **Save only selected features** is toggled on then click **OK**. Inspect the output in the map. Remember to deselect the attribute table!

How do the trees mapped in OpenStreetMap compare with the official trees mapped in the “ubcv_campus_trees” layer of the ubcv database?

Notice that a lot of street trees are missing on campus? You can rectify this if you want, just sign up for an account at OpenStreetMap.org and start digitizing! You can also use any of a number of free apps on iOS and Android to start mapping from your phone or to record GPS tracks.

Step 8: Open the attribute table and use the Identify tool to find a key in the polygons layer that you can use to identify buildings. Once you have identified the key, select all the buildings with an expression. Hint: statements involving *NULL* values for keys need to use the *IS* and *NOT* operators, rather than =

(equal to) and != (not equal to).

Step 9: We are now going to modify the selection from the last step to only include buildings that are located on campus. Open the **Select by Location** tool and change **Select features** from to the polygon layer. Toggle on **are within** for the geometric predicate and then **By comparing to the features from** should be set to the “ubcv_legal_boundary” layer that you added to your project from the ubcv database on the UBC PostgreSQL server. Change **Modify current selection by** to **selecting within current selection**. This last part is important as we are basically taking what we select in the last step and selecting from that selection the buildings that are geographically within the UBC boundary. Click “Run” and then inspect the output to make sure only buildings within the UBC boundary are selected. When you are satisfied, export the selection to a your “UBC.gpkg” geopackage in your project and name the layer **osm_campus_buildings**. Again, make sure you toggle on **Save only selected features** and then deselect the layer when you are done.

How do the buildings mapped in OpenStreetMap compare with the official buildings mapped in the “ubcv_buildings” layer of the ubcv database?

Q4. How could a value of “YES” for a key impact your analysis of that key instead of a more specific value? (1 point)

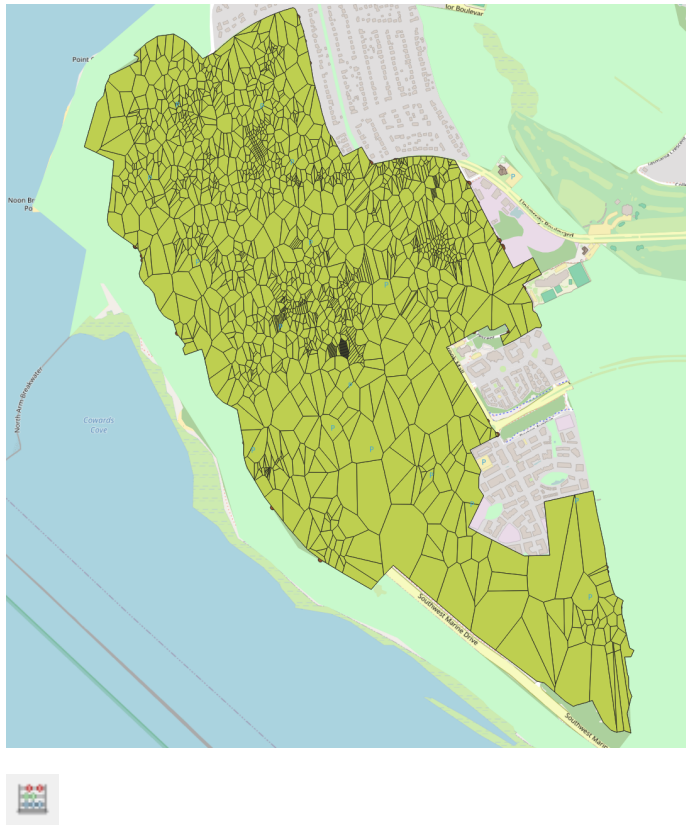
Task 3: Simple spatial analysis with QGIS

Step 1: From the top menu, select **Vector**, then **Geometry Tools**, then **Voronoi Polygons...** For the **Input layer**, select your points layer and change the **Buffer region** to 100.



Click the ellipse button next to the blank field for **Voronoi polygons** and select **Save to GeoPackage...** and select the “osm_points_campus” layer from your “UBC.gpkg” geopackage. Name the output **osm_campus_voronoi** and then click **Run**. Inspect the output in the map. You should now have a Voronoi diagram that is a little larger than campus.

Step 2: From the main menu, click **Processing** and then **Toolbox** to open the **Processing toolbox** pane (if it is not already open). Search for the **Clip** tool and then clip the Voronoi polygons by the UBC campus boundary and name the output **osm_campus_voronoi_clip** then click **Run**. Inspect the output in the map. You should now see Voronoi polygons within the campus boundary as shown below.



Step 3: Open the attribute table for the clipped Voronoi polygons and click the **Open field calculator** button that looks like an abacus. This will display the **Field Calculator** dialogue. We are going to calculate the polygon areas and save those values to a new field.

Step 4: Ensure that **Create a new field** is toggled on. In **Output field name**, the new field is going to be called **Area**. Then type $\$area$ into the **Expression** box on the left and click **OK**. The units of this new Area field are square meters because your QGIS project properties default to the WGS 84 ellipsoid with square meters as the unit for area measurements. You can change this if you want by navigating to **Project** in the main menu, selecting **Properties**, and then changing the units and ellipsoid to whatever you want.

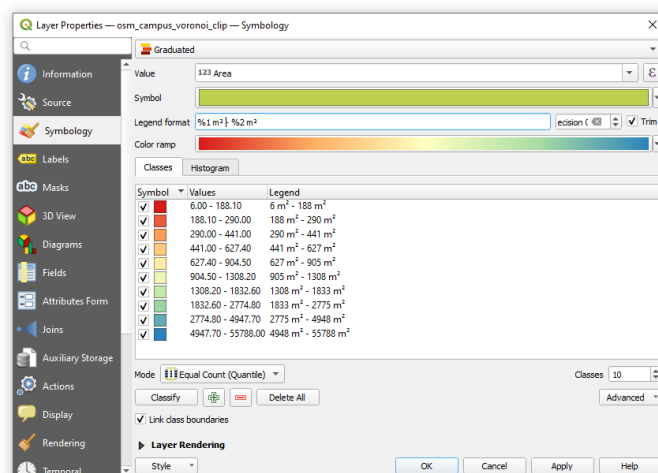
Step 5: Open the `osm_campus_voronoi_clip` layer properties. On the left, select **Symbolology**. By default, all features are symbolized the same, called **Single Symbol**. At the top drop-down menu, select **Graduated**.

Step 6: For **Value**, select the **Area** field that you calculated earlier. The **Symbol** determines how the features will be filled and should be a solid color. Leave the Symbol as-is.

Step 7: The **Legend format** is used to programmatically modify the legend text entry in the final map. Because we have selected a Graduated symbology, we will have values that range from %1 to the next value for that color %2. You can add additional text here to indicate units. For example, $\%1m^2 - \%2m^2$ will yield a legend entry that looks like “2425 m² – 3425 m²”. Add units of square meters to the legend format.

Step 8: The **Color Ramp** controls the range of hues (H) and their saturation (S) and value (V). If you click directly on the default color ramp icon, it will open the Select Color Ramp dialogue window where you can modify both discrete and continuous color ramps. You can even control opacity of these values to get some interesting effects on your map. If you opened the Select Color Ramp dialogue, close it. Another option, is to choose a pre-defined color ramp using the downward pointing triangle just to the right of the color ramp. Select **Spectral**.

Step 9: You should now see values and colors appear below in the **Classes** tab. If you do not see anything, then look for the **Mode** drop-down menu towards the bottom of the dialogue window and select **Equal Count (Quantile)**. Click **Apply** and inspect how the Voronoi polygons are now symbolized in your map. Try the other modes and increase or decrease the number of **Classes**, applying the changes each time so that you can inspect the map. Return the Mode to **Equal Count (Quantile)**.



Step 10: If you closed it, open the symbology of the Voronoi polygons again, and then expand the **Layer Rendering** at the bottom of the dialogue window. Change the **Opacity** of the layer to 50% then click **Apply**. Make sure you have other layers toggled off and the OpenStreetMap base map toggled on. Use the Voronoi diagram and the newly created symbology to interpret the density of OpenStreetMap nodes across campus.

Q5. Which areas of campus have a high density of nodes in OpenStreetMap on UBC Campus? What are some possible reasons why there might be low node density in the other areas? (3 points)

Summary

You learned how to handle the OpenStreetMap data model and perform some simple analysis from these data. Next, you might want to consider the implications for using Volunteered Geographic Information (VGI) generally and OpenStreetMap data specifically in future projects. What are the limitations of OpenStreetMap and where does this VGI perform well? Are there any similar, alternative data sources for your area of interest?

Return to the **Deliverables** section to check off everything you need to submit for credit in the course management system.

Chapter 5

Raster and Least Cost Path Analysis

Written by Paul D. Pickell

Lab Overview

In this lab, you will explore many of the different raster operations using the Geospatial Data Abstraction Library (GDAL). We will practice using these utilities directly from the command line and in QGIS. You will create a variety of different rasters from different sources and practice deriving new rasters through calculations. These efforts will culminate in a least cost path analysis in ArcGIS Pro. We will examine a critical conservation problem in western Alberta, Canada: Grizzly Bear (*Ursus arctos horribilis*) habitat and movement. Grizzly Bears are among the largest land predators in North America and have had much of their historical range reduced by human activity. The largest and most well-studied population now resides in the Rocky Mountain foothills of western Alberta.

Learning Objectives

- Describe the raster data model
- Convert between image coordinates and geographic coordinates
- Convert vector data to raster
- Distinguish data types in rasters and identify appropriate encoding of sampled values for different phenomena
- Mosaic rasters together

- Evaluate expressions on rasters using arithmetic and relational operators
 - Combine rasters and calculate weighted overlays
 - Calculate a least cost path of a landscape indicator species
-

Deliverables

Answers to the questions posed throughout the lab (20 points)

Create a map of your least cost path that supports your interpretations in Q8 (10 points)

Data

All data for this lab are accessible via the UBC PostgreSQL server. Instructions for connecting to the server are given in prior labs.

Task 1: Define a raster area of interest

Our first task is to define an Area of Interest (AOI) that will represent the extent of all of our raster processing.

Step 1: Open QGIS and create a new project named “GrizzlyBear”.

Step 2: Connect to the `bear` database on the UBC PostgreSQL server and load the “grizzlybearmanagementareas” layer into your map. These polygons represent the Grizzly Bear population units or Bear Management Areas (BMA) in the Rocky Mountain foothills of Alberta (Alberta Open Government License). For this lab, we will be focusing on the Yellowhead BMA for our AOI.

Step 3: Write an SQL query to select the Yellowhead BMA and then export it to your local QGIS project folder as a shapefile named “yellowhead_bma.shp”, and change the “CRS” to NAD83, UTM zone 11N (EPSG:26911).

We are going to rasterize this layer and do the basic raster processing outside of QGIS/ArcGIS Pro for a couple of reasons. First, setting the options and raster processing environments in QGIS/ArcGIS Pro is overly complicated and prone to error and unexpected behaviour. Second, you will learn how to use some valuable raster processing utilities of GDAL that can be easily batched and run in chain sequence (i.e., you can batch process large research datasets from a command line).

Step 4: Open the OSGeo4W shell and navigate to your QGIS project folder using the change directory `cd` command. To do this, first copy the path of your project folder, usually something like `C:\Users\paul\Documents\QGIS\GrizzlyBear`. Then in the console type `cd C:\Users\paul\Documents\QGIS\GrizzlyBear` to change to that directory.

The first GDAL utility we are going to use is `gdal_rasterize` (link to documentation). This utility takes a vector input datasource and converts it to a raster in the output. The simplest usage is `gdal_rasterize <src_datasource> <dst_filename>`. All of the flags preceded by `-` are indicated as optional by the square brackets `[]`. If you do not use any of the flags when you run the utility, then the utility will use the default values/settings for those flags that are indicated in the documentation. The AOI raster we are going to create will specify a few conditions of all the other raster processing we will do:

- raster extent (number of rows/columns and geographic coordinates)
- pixel/cell resolution
- spatial reference system

Step 5: In the OSGeo4W shell, enter the following command `gdal_rasterize -burn 1 -tr 20 20 -ot Byte -a_nodata -tap yellowhead_bma_utm.shp yellowhead_bma_utm.tif`.

- `-burn` flag indicates we want to assign a value of 1 to the output raster where the features intersect the new raster grid.
- `-tr` flag indicates we want the output cell resolution to be 20 m (we know its meters because those are the units of the coordinate system) and this is specified twice `20 20`, once for the x-axis and again for the y-axis.
- `-ot` flag indicates that we want the output data type of the raster to be "Byte", which translates to a storage cost of 8 bits per pixel (bits per pixel $\times$ rows $\times$ columns = size of raster on disk).
- `-a_nodata` flag indicates that we want to specifically set the nodata value as `nodata` (default is zero). This will come in handy later because `nodata` limits the number of cell calculations we need to perform while zero is a valid processing value.
- `-tap` flag (target aligned pixels) indicates that we want the new raster extent to be calculated using floor and ceiling functions:

```
ymax (top) = y_resolution * ceiling(ymax / y_resolution) = 20 * ceiling(5,936,280.447609 / 20) = 20 * 296814.0238045 = 5,936,280.47609
ymin (bottom) = y_resolution * floor(ymin / y_resolution) = 20 * floor(5,753,454.574941 / 20) = 20 * 287672.73724705 = 5,753,454.744941
xmin (left) = x_resolution * floor(xmin / x_resolution) = 20 * floor(401,496.997149 / 20) = 20 * 20074.84985745 = 401,496.987149
xmax (right) = x_resolution * ceiling(xmax / x_resolution) = 20 * ceiling(661,663.194125 / 20) = 20 * 33083.15970625 = 661,663.194125
```

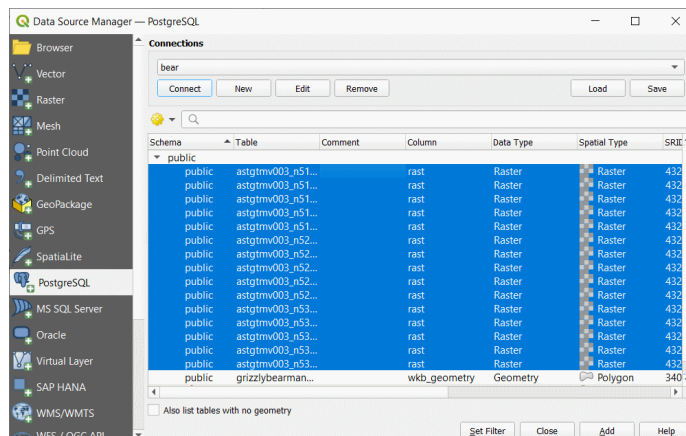
The above calculations are probably not immediately intuitive, but the idea is that we want to create a raster grid that is aligned to the ceiling and floor of our x- and y-resolution. The overall size of the raster is the same whether we do this alignment or not (same number of rows and columns), the only

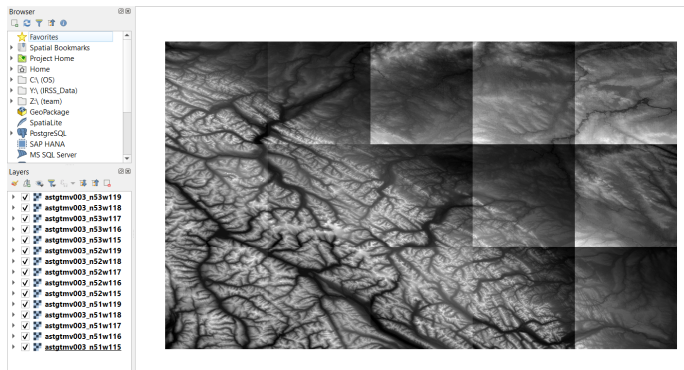
difference is that we are shifting the grid so that the extent coordinates are whole numbers (integers), which makes it easier for us to compute new grids. Bring “yellowhead_bma_utm.tif” into QGIS to inspect it. Check that the values, extent, cell size, and coordinate system are correct. You can also run the `gdalinfo` utility to check the metadata from the command line like `gdalinfo yellowhead_bma_utm.tif`.

Task 2: Seamless mosaicking

A common task in raster analysis is mosaicking multiple rasters together to perform analysis on a single file over a larger extent. We will practice doing this with some ASTER Digital Elevation Models (DEM) that are distributed as non-overlapping tiles in WGS 1984 (EPSG:4326). The source for these data can be found at NASA Earth Science Data Systems portal, but we will be retrieving these data from the UBC PostgreSQL server instead.

Step 1: Open your QGIS project and from the “Data Source Manager” connect to the `bear` database of the UBC PostgreSQL server. You should see a listing of 14 rasters there, which have names prefixed with “astgtmv003”. Select all 14 and then click “Add”. It is very important that you add the rasters using this method rather than from the “Browser” pane, otherwise you might have unexpected errors.





Step 2: Open the “Layer Properties” for “astgtmv003_n51w115” and inspect the extent values and the pixel size.

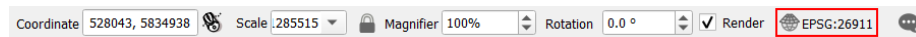
Q1. What units are the extent and pixel size in? (1 point) If you are wondering why the y pixel size is negative, it is to facilitate the conversion between image coordinates and geographic/projected coordinates. Image coordinates are the unique combination of rows i and columns j representing each pixel in the image. By convention, the origin $(0,0)$ for image coordinates is the upper left corner of the image. Image coordinates are always positive and increase as you move down the image to the lower right corner at (n,m) , where n is the number of columns in the image (think x-axis) and m is the number of rows in the image (think y-axis). Most geographic and projected coordinate systems decrease along the y-axis (as you move south), which is the inverse of image coordinates that are increasing. Thus, the y pixel size is usually stored as a negative number in order to convert image coordinates into geographic/projected coordinates.

For example, to convert pixel (i,j) from image coordinates to geographic coordinates where the image origin $(0,0)$ corresponds to geographic coordinates $(-115.0001388888889977, 52.0001388888888982)$ as is the case for “astgtmv003_n51w115” and the x pixel size is 0.000277778 and the y pixel size is -0.000277778:

$$\begin{aligned}\text{Longitude}(i) &= -115.0001388888889977 + i \times 0.000277778 \\ \text{Latitude}(j) &= 52.0001388888888982 + j \times -0.000277778\end{aligned}$$

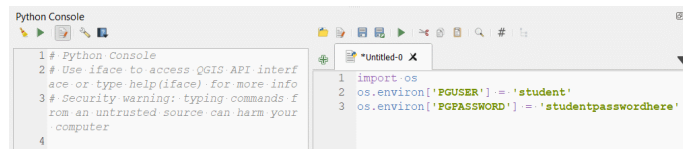
Q2. Given the equations above, what is the exact latitude and longitude of the pixel with image coordinates (1907,1918)? Show your work for full credit. (2 points) **Step 3:** Make sure the coordinate system for the map is initially set to EPSG:4326 then copy the lat/lon coordinates from Q2 and paste them into the “Coordinate” field at the bottom of your QGIS screen in the format `latitude, longitude` and press “Enter”. Change the scale to 1:500 and press “Enter”. You should now be able to distinguish individual pixels. Click the coordinate reference system at the bottom right of

the screen and change it from the default value of EPSG:4326 (WGS 1984) to EPSG:26911 (NAD 1983 UTM Zone 11N).



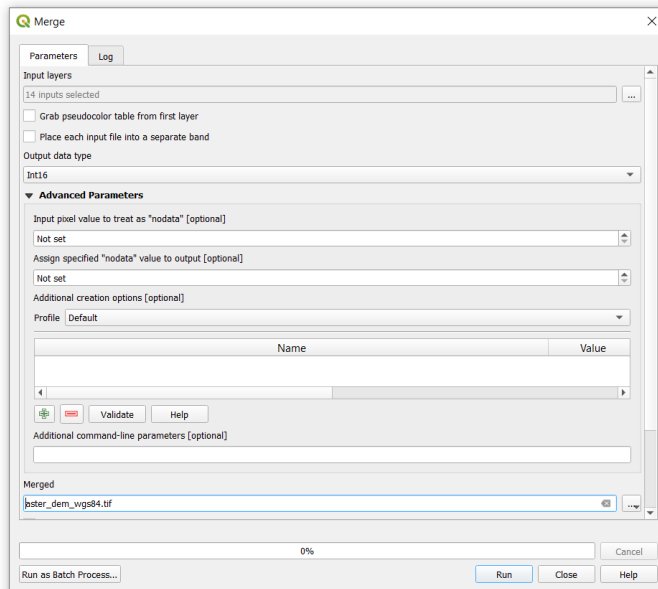
Q3. Why do the pixels now appear to be elongated along the y-axis? (2 points) **Step 4:** Save your PostgreSQL credential to your QGIS environment variables by opening the Python console from your toolbar in QGIS, click the small “Show editor” button, and then paste the following into the editor and click the “Run” button:

```
import os
os.environ["PGUSER"] = "student"
os.environ["PGPASSWORD"] = "studentpasswordhere"
```



This step will automatically authenticate you each time you need to transact with the PostgreSQL server through the GDAL utilities. Your credential is only saved for the duration of your QGIS session, so once you close QGIS and re-open your project, you will need to come back to this step to save your environment variables. Close the python console and editor when you are done.

Step 5: From the QGIS processing toolbox, search for the “Merge” tool, filed under “GDAL Raster Miscellaneous”. This tool is the equivalent of the `gdal_merge` command-line utility tool, and it takes a set of rasters as input and creates a mosaic of those data. Click the ellipse “...” next to the “Input Layers” field and then click “Select All” to toggle on all the ASTER DEMs currently in your project. Expand the menu for “Output data type” and select “Int16”. Name the output “aster_dem_wgs84.tif”. Click “Run”. Notice when you run these GDAL tools, the log shows you the exact GDAL command that you can run in the command line, like we did in Task 1. Inspect the output in QGIS. Notice that we did not download any of the raster data to our local machine until this very last step.



The mosaic still needs to be reprojected and clipped to the same extent as our AOI, which is what we will do in the next step using the `gdalwarp` utility. The QGIS tool, “Warp”, does not expose all the options that we need, so we will do this next step in the OSGeo4W shell.

Step 6: Open the OSGeo4W shell and navigate to your project folder where you saved “aster_dem_wgs84.tif”. Type the following command into the console and then hit “Enter”:

```
gdalwarp -s_srs EPSG:4326 -t_srs EPSG:26911 -te 401480 5753440 661680 5936300 -tr 20 20 -r cubic aster_dem_wgs84.tif aster_dem_utm.tif.
```

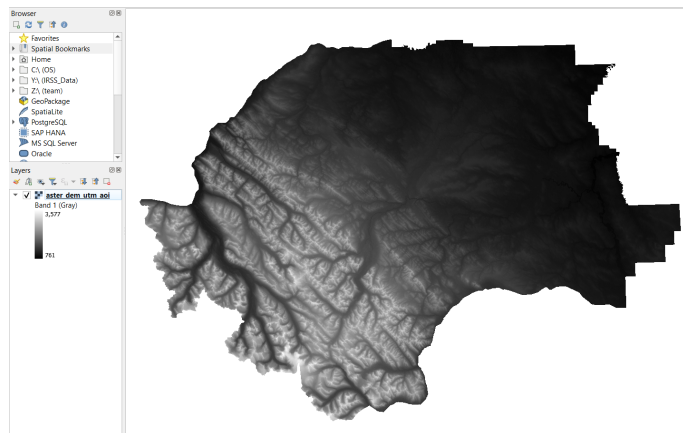
- `-s_srs` flag defines the input (s = source) spatial reference system (coordinate system), which is WGS 1984 or EPSG:4326.
- `-t_srs` flag defines our target SRS, which is NAD 1983 UTM Zone 11N or EPSG:26911.
- `-te` flag defines our target extent `xmin ymin xmax ymax`, which is the calculation we did in Task 1 401480 5753440 661680 5936300.
- `-tr` flag defines the target resolution, again specified twice for both axes 10 10.
- `-r` flag defines the re-sampling method. This is an important flag for `gdalwarp` because any time that you re-project a raster, you are resampling that raster. The term “resampling” comes from the fact that a raster is “sampled” to begin with. Each cell in a raster is a sample of some random variable over the extent of the raster, and the value of that sample

is correct for the center of the cell. When we re-project a raster, the locations of the cells change, therefore we need to recalculate the samples or “resample” the raster. Our selected method of resampling in this case is **cubic**, which is a smoothing algorithm and well-suited for continuous data like elevation.

- **aster_dem_wgs84.tif** is the input raster and **aster_dem_utm.tif** is the output (resampled) raster. Inspect the output in QGIS.

Note that we did not use the **-ot** output data type flag, so **gdalwarp** used the **Int16** data type of our input raster. If you set **-ot Float32**, then you would see that the cubic resampling algorithm actually yields decimals instead of the integers that you see in the current output, but for our purposes that sub-meter level of precision is not needed.

Step 7: Search for the “Raster Calculator” tool. Double-click the “aster_dem_utm.tif” mosaic to add it to the expression, then click the multiply button “*”, then double-click the “yellowhead_bma_utm.tif” raster to add it to the expression. Scroll down farther and click the ellipse “...” next to the “Output” field to save the output as “aster_dem_utm_aoi.tif”. Click “Run” and then inspect the output in your map.



Task 3: Calculating movement costs

In this task, we will calculate a few derivative rasters that are going to represent various costs for Grizzly Bears to move across the landscape. At the end of the task, we will produce a cost raster that will be one of the inputs to our least cost path model in the following task.

Step 1: Open your QGIS project and search for the “Slope” tool. Add “aster_dem_utm_aoi.tif” as the “Elevation layer” and save the output to

“aster_slope_utm_aoi.tif”. Inspect the output in your map.

Step 2: We are going to adjust the slope values to a range of 0 and 1. This is easily done by dividing all values by 90, which is the theoretical maximum slope in degrees. Open the “Raster Calculator” tool and type the expression “aster_slope_utm_aoi@1” / 90. Change the “Reference layer” to “yellowhead_bma_utm.tif”, click “OK”, and name the output “aster_slope_utm_aoi_cost.tif” then click “Run”. Inspect the output in your map, you should now have values between 0 and 1.

Step 3: Open the “Data source manager”, connect to the **bear** database on the UBC PostgreSQL server, and add the “ca_forest_vlce2_2019” raster to your map. This layer is a clipping of a national land cover dataset for Canada for the year 2019. The values of the raster refer to specific land covers. Open the properties for the layer. As you can see, it has a different coordinate system, resolution, and extent than our AOI. Export this raster as a GeoTiff to your local project folder and keep the output name the same.

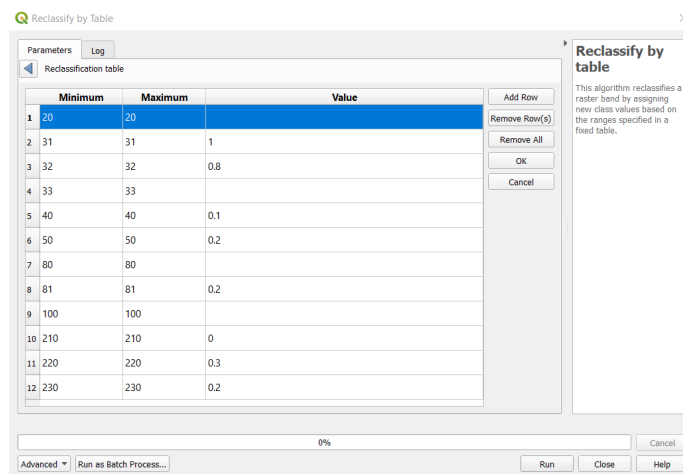
Step 4: Open the OSGeo4W shell and navigate to your project folder. Run the `gdalwarp` utility again, parameterize it the same as before in Task 2, but change the `-s_srs` flag to `EPSG:3978`, the `-r` flag to `near`, the input file to `ca_forest_vlce2_2019.tif`, and the output file to `land_cover_utm.tif`. Inspect the output in QGIS.

Q4. Why did we choose “near” as the resampling method? (1 point) **Step 5:** Use the “Raster Calculator” tool to multiply “yellowhead_bma_utm.tif” by “land_cover_utm.tif” and write the output to “land_cover_utm_aoi.tif”.

We want to use the land cover layer as an input to our cost surface, but the values of the land cover raster are codes, which are nominal data. These values cannot directly be compared as-is. For example, the code for water is 20 and the code for herbs is 100. These numbers do not imply that herbs is 5 times more costly than water, they are simply numerical placeholders for the concept of land cover. We need to define and then assign proper values for each land cover that represents the cost or resistance for Grizzly Bears to traverse a cell of that land cover. We can do this by reclassifying the land cover raster. Below is a table indicating the land cover class codes and some proposed cost values scaled between 0 (no cost) and 1 (highest cost):

Code	Land Cover	Proposed Cost
20	Water	?
31	Snow/Ice	1
32	Rock/Rubble	0.8
33	Exposed Barren Land	?
40	Bryoids	0.1
50	Shrubs	0.2
80	Wetland	?
81	Treed Wetland	0.2
100	Herbs	?
210	Coniferous	0
220	Broadleaf	0.3
230	Mixedwood	0.2

Step 6: Open the “Reclassify by Table” tool. Select “land_cover_utm_aoi.tif” as the “Raster layer” and then click the ellipse “...” next to “Reclassification table”. Populate the table with all the land cover codes. You will need to use some judgement and determine what would be reasonable values for the four missing codes in the table above. There is no right or wrong answer for these, but you will need to justify your choices. When you have filled in the table (see image below), change “Range boundaries” to “min $\leq$ value $\leq$ max”, click “OK”, and then name the output “land_cover_utm_aoi_cost.tif”. Inspect the output in your map, you should have values ranging from 0 to 1.



Q5. What was your rationale for cost choices for classes 20, 33, 80, and 100? (4 points) The last layer we are going to add is distance to roads as a simple proxy for human-related mortality.

Step 7: Open “yellowhead_roads” in QGIS from the UBC PostgreSQL

database, if you have not already. Open the “Reproject Layer” tool and change the projection for “yellowhead_roads” to EPSG:26911 and name the output “yellowhead_roads_utm.shp”.

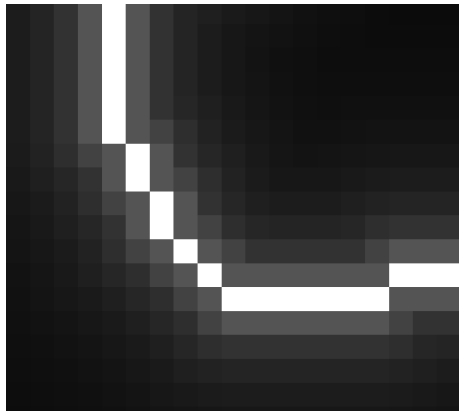
Step 8: In the OSGeo4W shell, enter the following command `gdal_rasterize -burn 1 -tr 20 20 -ot Byte -a_nodata nodata -te 401480 5753440 661680 5936300 yellowhead_roads_utm.shp yellowhead_roads_utm.tif`. You should recognize all these flags by now. Inspect the output in QGIS.

Step 9: Search for the “Proximity (Raster Distance)” tool. The input layer will be “yellowhead_roads_utm.tif”. Name the output as “yellowhead_roads_distance_utm.tif”. Ensure that your QGIS map project CRS is set to EPSG:26911 in the bottom right hand corner, then click “Run” and inspect the output in the map.

Step 10: Open the “Raster Calculator” tool and add the expression $(1 / ("yellowhead_roads_distance_utm@1" + 0.5)) / 2$. Let us break this one down:

- $1 / "yellowhead_roads_distance_utm@1"$ would give us the inverse distances, however, cells that are roads have a distance of zero, and $1 \div 0$ is undefined and returns NoData. Therefore, we need to add a fraction to the denominator.
- $1 / ("yellowhead_roads_distance_utm@1" + 0.5)$ avoids nodata values in the situation above, but now roads have a value of $1 \div 0.5 = 2$. We ultimately want these values to be between 0 and 1, with 1 representing roads.
- $(1 / ("yellowhead_roads_distance_utm@1" + 0.5)) / 2$ dividing everything by 2 causes the 0.5 fraction to be removed from all cells, and road cells to be assigned a value of $2 \div 2 = 1$.

This way, the cost of traversing a roadway is 1 and decreases as you move away from the road. Change the “Reference layer” to “yellowhead_bma_utm.tif” and save the output as “yellowhead_roads_distance_utm_cost.tif”. Inspect the output in your map and zoom in close to a road.



Now that we have three different cost layers, slope, land cover, and distance to roads, we simply need to decide how to combine them into a single cost raster. This is made easy by the fact that all layers are scaled between 0 and 1. The simplest approach would be to add them all together, but this assumes equal-weighting given to each cost factor. It is also common to weight layers differently and this can be achieved by multiplying a set of weights to the layers that add up to 1. For example, to weight slope 10%, land cover 30%, and road distance 60%, we would use the formula $totalcost = slope \times 0.1 + landcover \times 0.3 + road \times 0.6$. Note that $totalcost = slope \times 0.33 + landcover \times 0.33 + road \times 0.33$ is functionally the same as $totalcost = slope + landcover + road$. Differences in the percentage weightings are not absolute difference, but relative. For example, 30% weight to land cover implies that land cover is weighted 3 times more than slope at 10%: $0.3 \div 0.1 = 3 = 300\%$.

Step 11: Pick weightings for your three layers (the weights must sum to 1), then open the “Raster Calculator” tool and apply the formula above. Set the “Reference layer” to “yellowhead_bma_utm.tif” and save the output as “yellowhead_cost_raster.tif”.

Q6. What was your rationale for selecting your weights? (2 points)

Q7. What additional or alternative layers could you have used for costs in this analysis? Give at least two examples and explain why they are pertinent. (4 points) In these last steps, we are going to create the point layers that will be used to trace the least cost path in the following task.

Step 12: Add the “grizzly_bear_core_access_management_area” to your map from the UBC PostgreSQL server. These represent the core habitat of Grizzly Bear across all regions. Select the Yellowhead unit and then search for the “Centroids” tool under “Vector Geometry”, ensure “Input Layer” is set to “grizzly_bear_core_access_management_area”, “Selected features only” is

toggled on, and save the output as “yellowhead_bma_centroid.shp”. Reproject this layer to EPSG:26911 and name the output “source.shp”.

Step 13: Now let us randomly choose a point in the secondary habitat area. Add the “grizzly_bear_secondary_access_management_area” and select the Yellowhead unit. Search for the “Random points inside polygons” tool, ensure “Input layer is set to” grizzly_bear_secondary_access_management_area”, “Selected features only” is toggled on, and save the output as “yellowhead_bma_target.shp”. Reproject this layer to EPSG:26911 and name the output “target.shp”. You can repeat this as many times as you want until you get a satisfactory location.

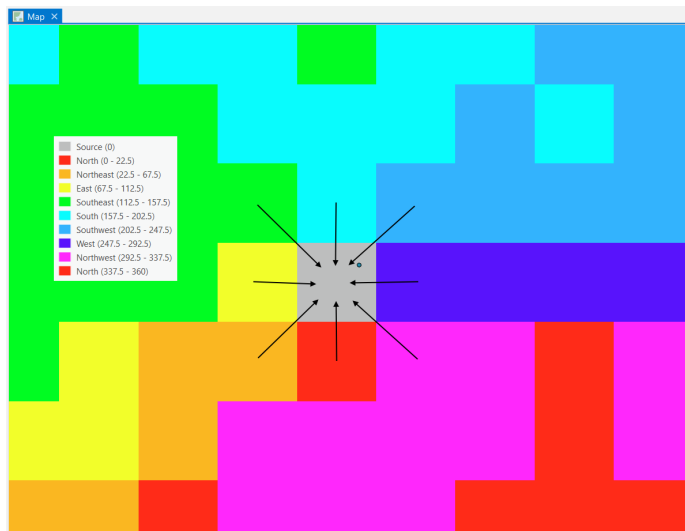
Task 4: Least cost path analysis in ArcGIS Pro

For this last task, we are going to perform the least cost path analysis in ArcGIS Pro. QGIS has a least cost path plugin, but it is not as robust as the ArcGIS Pro tools.

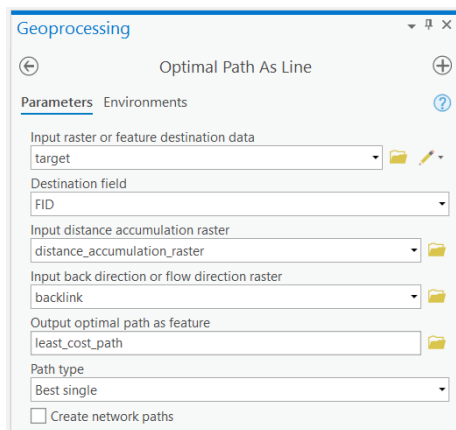
Step 1: Open a new ArcGIS Pro project and add the following layers to your map:

- Total cost raster: “yellowhead_cost_raster.tif”
- Aster DEM: “aster_dem_utm.tif”
- Slope raster: “aster_slope_utm_aoi.tif”
- Land cover raster: “land_cover_utm_aoi.tif”
- Road polylines: “yellowhead_roads_utm.shp”
- Yellowhead source: “source.shp”
- Yellowhead target: “target.shp”

Step 2: Open the “Distance Accumulation” tool and parameterize it according to the image below. You can save the two output rasters (“distance_accumulation_raster” and “backlink”) in your project geodatabase or in your QGIS project folder as geotiffs, just be mindful that you cannot save a geotiff inside your geodatabase. This tool will take your source point and calculate the accumulative cost to travel to all other cells using a combination of the cost raster and the real geographic distance, both horizontal and vertical, which is why we supply the ASTER DEM.



Step 3: Now we can trace the least cost path between our source and target locations. Open the “Optimal Path as Line” tool and parameterize like the image below then click “Run”.



Step 4: Inspect the output. You should be able to trace the line along the “backlink” raster. Toggle on the roads. How many roads did you path cross? Toggle on the land cover. Were there any land covers that were avoided? Toggle on the slope. Did the path avoid any steep slopes or otherwise maneuver around the terrain?

Q8. Interpret the impact that your choice of weightings had on the final least cost path that you observed. (4 points) **Step 5:** Create a map of your least cost path that supports your interpretations in Q8. Export it as a pdf and upload to the course management system.

Summary

Raster analysis is comprised of manipulating base data, deriving new rasters, and weighting and combining rasters together in order to answer a spatial question. In this lab, you learned how to effectively set up a raster analysis through careful planning and choice of factors like raster extent, coordinate system, cell size, alignment, data type, and resampling algorithms. Rasters can lead to powerful spatial analyses like least cost path analysis due to the explicit encoding of information in the spatial neighborhood.

Return to the **Deliverables** section to check off everything you need to submit for credit in the course management system.

Chapter 6

Digitization and Editing with Kart

Written by Paul D. Pickell

Lab Overview

Version control is an important concept to software development that has recently been adopted and adapted for use in GIS. The ability to track data changes, manage versions, and collaborate on data editing are significant advances for managing geospatial data assets in an enterprise environment.

In this lab, you will learn about version control using both Git and Kart software. Git is a widely used distributed version control software system for tracking changes to files. Kart simply extends this distributed version control to geospatial vector data including points, lines, polygons, and point cloud datasets. You will also practice digitizing and editing datasets in QGIS and ArcGIS Pro.

Learning Objectives

- Practice using distributed version control software systems Git and Kart
- Digitize and edit geospatial features in QGIS and ArcGIS Pro
- Commit edits to a local repository
- Push edits to a local repository
- Manage merge conflicts and diffs between working copies of a repository

Deliverables

Answers to the questions posed throughout the lab (20 points)

Map response to the question posed at the end of Task 4 (10 points)

Data

All data for this lab are accessible via the UBC PostgreSQL server, the public MGEM Data Store, and the public MGEM Geoserver. Instructions for connecting to the server and data store are given in the tasks below and prior labs.

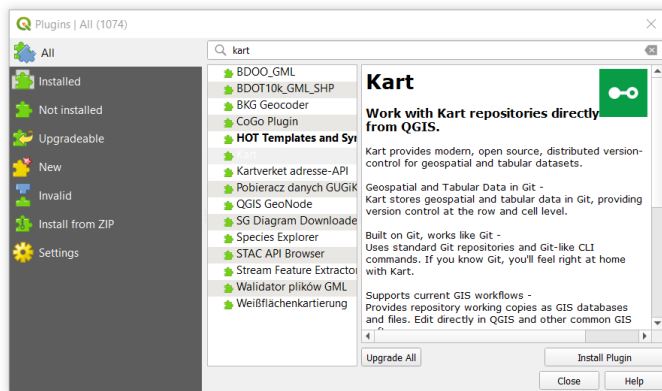
Task 1: Configure Git, install Kart, and create your first repository

At the core of version control with Git, each change made to a file is tracked with a message by a user, so we need to configure Git with a user name and e-mail address to sign our commit messages with. For the purpose of this lab, you can choose any username or e-mail address. You are not actually creating any account information and the user name and e-mail address you enter (real or fake) will only be stored on your local machine for this lab. We are not using GitHub for this lab, so you can just copy and paste the code in the steps below. However, if you will eventually start working with a hosted service like GitHub, then you will want to configure your local Git software with the user name and e-mail address associated with your GitHub account.

Step 1: Search for and open “Git Bash” in Windows.

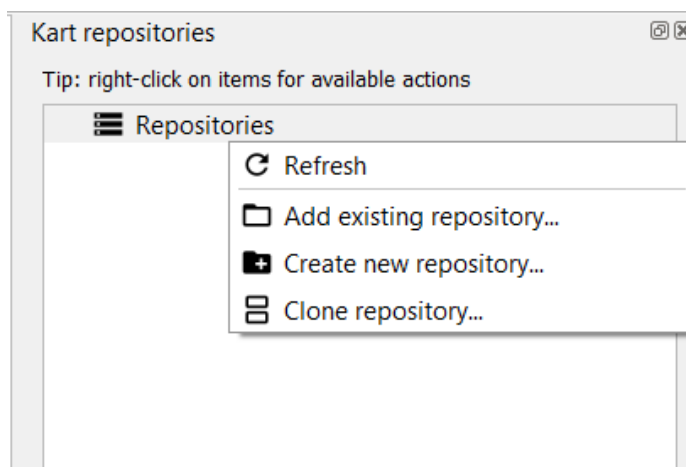
Step 2: Enter the following command `git config --global user.name "student"` and press enter. Next, enter the following command `git config --global user.email "you@example.com"`. Git is now configured and you can close the Git Bash.

Step 3: Open QGIS and install the Kart plugin using the Plugin Manager from the top toolbar. If you have not already installed the latest version of Kart on your computer, then you will be prompted to download and install the latest version.

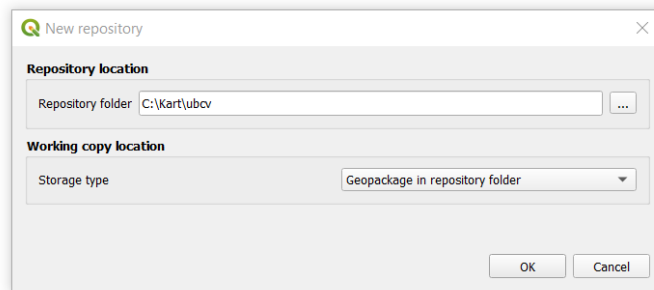


Step 4: Create a directory on your computer where you can store your first Kart repository, which we will call ubcv. For example, `C:\Kart\ubcv`.

Step 5: From the QGIS navigation bar at the top, select “Plugins”, select “Kart”, and then select “Repositories...”. This opens the Kart repositories pane.

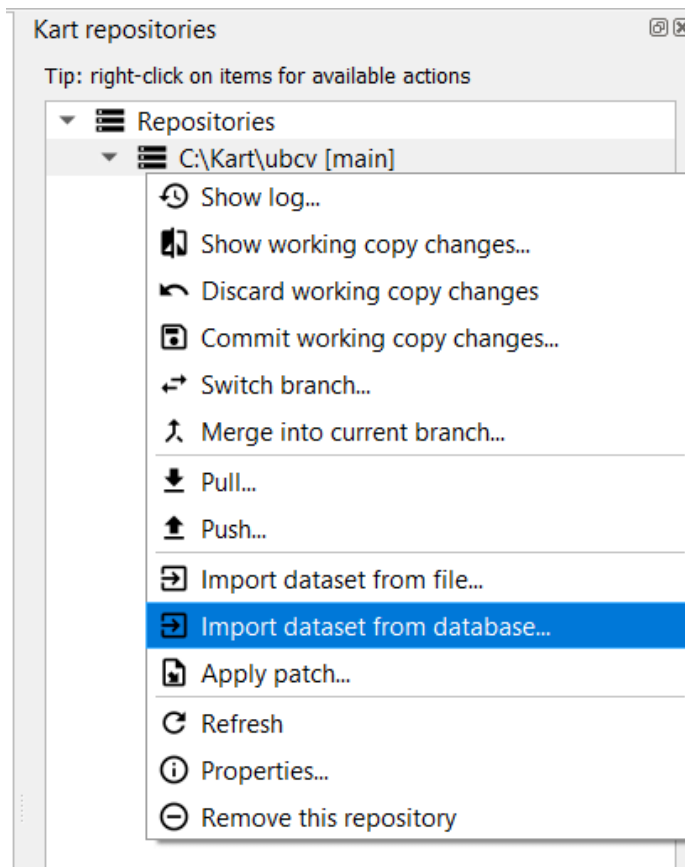


Step 6: From the Kart repositories pane, right-click “Repositories” and select “Create new repository...”. Add the path to the directory you created in Step 2, ensure “Storage Type” is set to Geopackage and click “OK” to initiate the Kart repository.

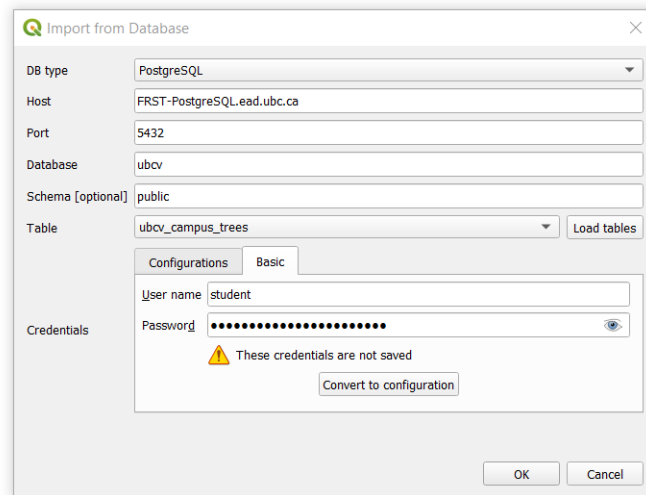


Expand your repositories list and you should see your new repository. If you expand “Datasets”, you will see nothing there. What we have done is basically told Kart to watch this directory for any changes to the files. If you navigate to the folder in your file system, you will see some new files `KART_README.txt`, `.git`, and `.kart` that Kart has added, which enables version control in this new repository. Now let us add some data and visualize it in QGIS.

Step 7: Right-click the repository `C:\Kart\ubcv [main]` and then select “Import dataset from database...”. We are going to connect to the `ubcv` database on the UBC PostgreSQL server.

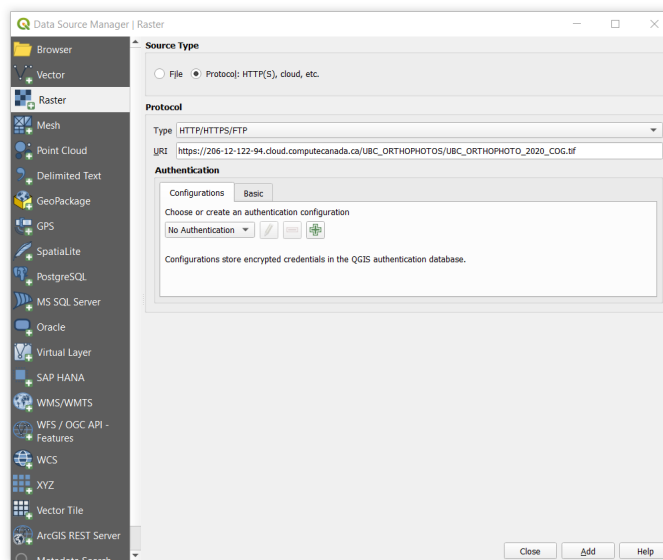


Step 8: Add the parameters to connect to the UBC PostgreSQL server. Be sure to switch to the “Basic” tab for credentials and add the student credential that was provided to the class. Once you have added the credential, click the “Load Tables” button to establish a soft connection to the database. If the connection was successful, then select “ubcv_campus_trees” from the drop-down menu and then click “OK” to add the table to your Kart repository.



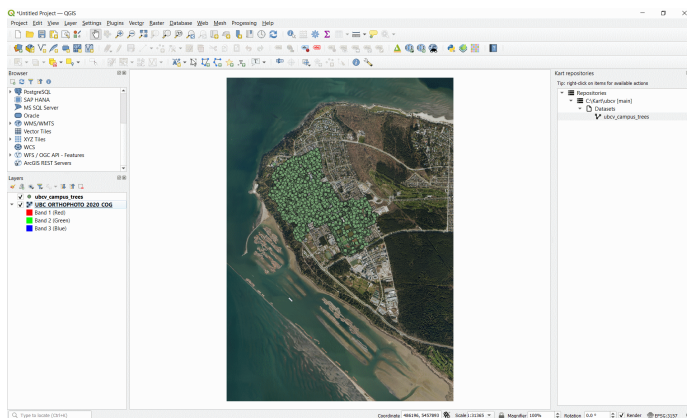
We will be comparing this point dataset to a recent orthophoto of UBC Vancouver campus and then making some edits.

Step 9: Add the most recent UBC orthophoto to your QGIS project. Navigate to the MGEM Orthophoto Data Store and then right-click the most recently dated “UBC_ORTHOPHOTO_YEAR_COG.tif” file listed there and then select “Copy Link”. Return to your QGIS project, open the Data Source Manager, select “Raster” from the left navigation bar, change “Source Type” to “Protocol: HTTP(S), cloud, etc.”, and then paste the link that you copied into the “URI” field and press “Add”.



UBC orthophotos are publicly available datasets that are also encoded as Cloud Optimized GeoTIFFs (COG), which enables fast retrieval and zoom levels for cloud-hosted raster datasets. If you find that the file hosted on the server is not very responsive, you can also download the GeoTIFF to your computer and add it as a regular file. Note that these orthophotos are large files (~7 GB) and ensure you have enough disk space in the location you want to save it.

Step 10: Drag and drop the “ubcv_campus_trees” dataset into your map from the Kart repository pane. Your QGIS project should look something like what is shown below.



Step 11: Zoom in to inspect the trees and the orthophoto.

Q1. What time of year do you think the orthophoto was collected? How do you know? (1 point)

Q2. Discuss three reasons why the campus trees do not perfectly match the locations in the orthophoto. (3 points)

Task 2: Edit a layer in QGIS with version control

Since the campus trees do not perfectly align with the locations in the orthophoto, we will practice editing some of them and then use the version control capabilities of Kart to inspect and manage the changes.

Step 1: From your QGIS toolbar, click the “Toggle Editing” icon to start an editing session.



Step 2: Once an editing session is started, you will be given the option to add new points or edit the vertices of existing points. Click the “Vertex Tool” icon to edit the existing point dataset.



Step 3: Zoom into an area where you can easily see where some points are not perfectly intersecting with the bottom of the tree trunk in the orthophoto. In the example below, we are looking at some trees just outside the Forest Sciences Centre at UBC on Agronomy Road. Right-click on the point you want to edit to show its current coordinate and it will highlight red in the map. Then left-click the same point and your cursor will now have a red “x” to indicate you are ready to place the new coordinate. Left-click again to move the point to the location of your cursor at the base of the tree in the orthophoto. If you are not satisfied with the location, just left-click the point and then left-click again to place it where you want. Repeat this step for 3-5 more points.

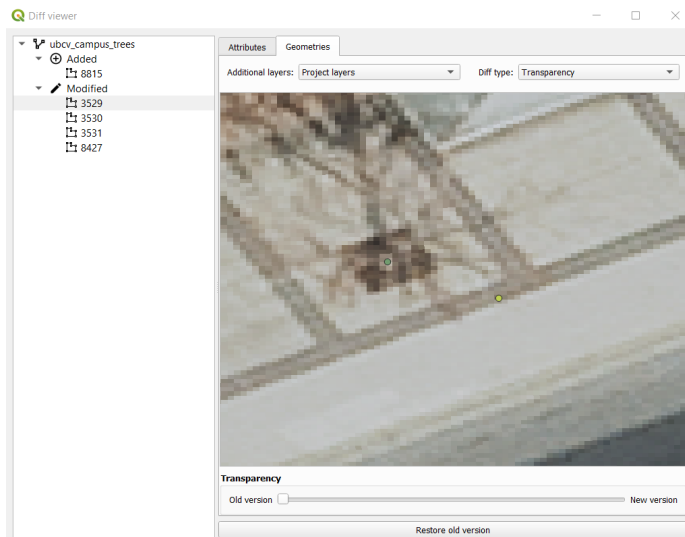
Step 4: Add a missing tree by clicking the “Add Point Feature” icon. Your cursor will now have a cross-hair and you simply left-click again anywhere on the map canvas to add the new point.



Step 5: A dialogue window should appear that prompts you to add values for the attributes. Since we do not really know any of these values, just let Kart autopopulate most of the values. Scroll down to “notes” and add a short message to this field then click “OK”. Add at least one point this way and then click the “Save Layer Edits” on the toolbar.



Step 6: Right-click the repository from the Kart repositories pane and select “Show working copy changes...”. This will open the Diff viewer that allows you to inspect the changes you just made to the layer. In the screenshot below, you can see that we added one point and modified four others. You can click the “Geometries” tab to view the change you made to the geometry of the feature, which is really handy.



Step 7: Right-click the repository again and this time select “Commit working copy changes...”. You will be prompted to enter a commit message. Commit messages help you and your collaborators understand what is happening in the changes that you just made. For this commit message, we are going to indicate that we edited four point locations and added one point.

You can think of committing as a way of saving your work progress in your repository. Every commit can be inspected and/or reverted if needed, so it is good practice to commit changes that are related to each other so that the commit messages can be concise and informative. Avoid committing every single edit separately as this can make navigating and interpreting the commit history difficult.

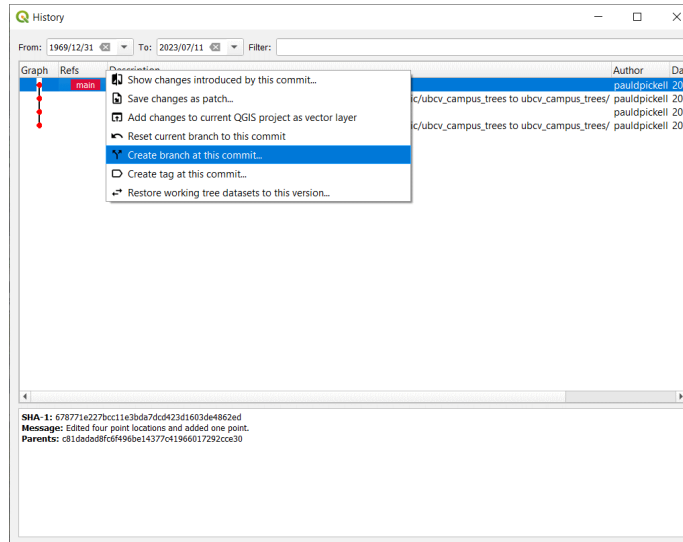
Step 8: Right-click the repository again and this time select “Show log...”. This will open the commit history for the repository and you should see the most recent commit you just made at the top along with the commit message you just entered in the last step. You can right-click any of the commits and select “Show changes introduced by this commit...”, which is another way to inspect the changes to attributes and geometries of the features with the diff viewer.

Task 3: Working on and merging different branches

Step 1: Right-click the repository and select “Show log...” to display the commit history.

Step 2: Right-click the commit you made in Task 2 and select “Create branch

at this commit...”. You will be prompted to enter a branch name, name it “mybranch”. You should now see both “main” and “mybranch” noted on the top commit.



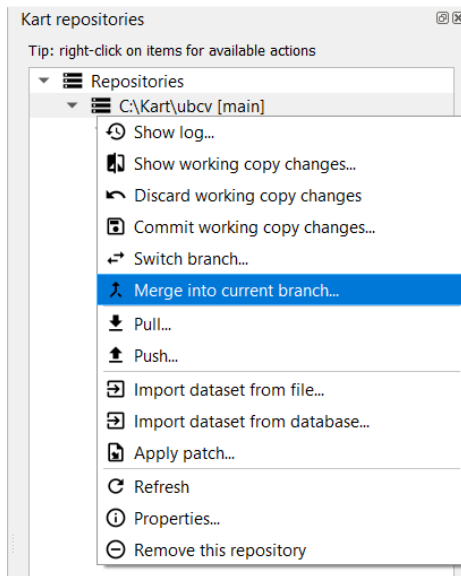
Step 3: Right-click the top commit again and select “Switch to branch mybranch...”.

Step 4: Now working on “mybranch”, start an editing session, pan around the orthophoto and add 5-10 more missing tree points. Do not change anything about the attributes, just click “OK” to dismiss after creating the point. Be sure to save your edits to the layer and then commit the change.

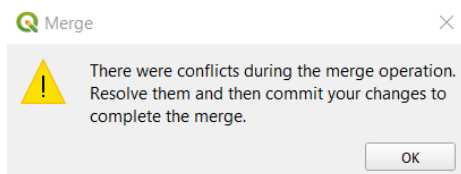
Step 5: Open the commit history of the repository. You will see now that “mybranch” is ahead of “main” by the commit you just made.

Step 6: Switch the branch back to “main” by right-clicking the commit tagged with “main” and then digitize 5-10 different tree points. Do not change anything about the attributes, just click “OK” to dismiss after creating the point. Commit your edits to the repository.

Step 7: Right-click the repository and select “Merge into current branch...”. In the dialogue window that appears, select “mybranch” from the drop-down menu for “Branch” and click “OK”. Since we are currently on the “main” branch, this will merge the edits/commits in “mybranch” into “main”.

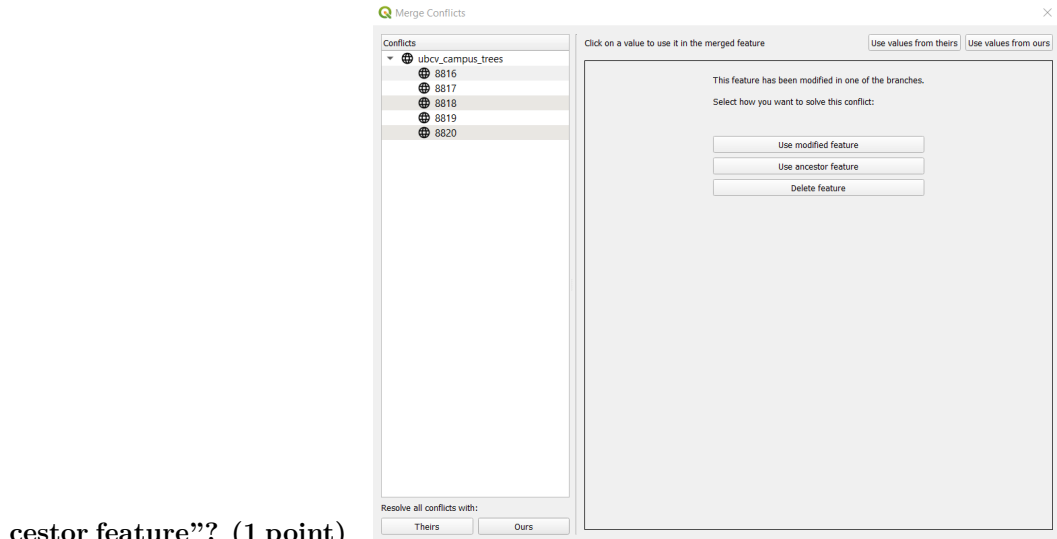


Step 8: You should see an error message appear warning you about a merge conflict between the two branches. Inspect the merge conflict by right-clicking the repository and selecting “Resolve conflicts...”. Inspect the conflicting commits then answer the question below. You may need to close the Merge Conflicts window and inspect the `ubcv_campus_trees` attribute table to really appreciate what is going on here.



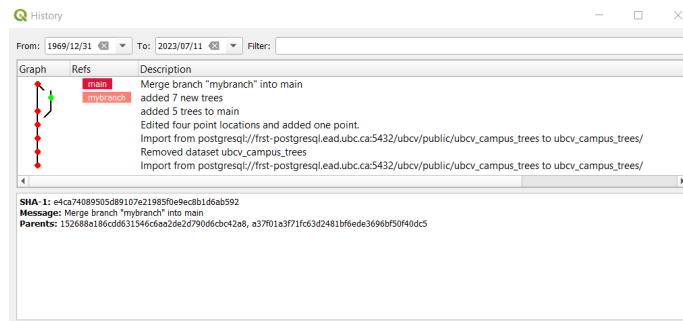
Q3. Describe why the merge conflict occurred. (3 points) Step 9:

Once you have answered the question above, open the Merge Conflicts window again. Click each feature one-at-a-time listed in the left then click “Use modified feature”. This is going to force Kart to recognize the most recent edits we made in “mybranch” and overwrite the conflicts in the “main” branch (our current branch). Once you have accepted all the modified feature edits, the Merge Conflict window will automatically close.

Q4. What is the difference between the “modified feature” and “an-

cestor feature”? (1 point)

Step 10: Open the commit history for the repository. You should now see the commit history between both branches, and we are now back to a single up-to-date branch (“main”).



In this example, we simulated two different branches to explore these tools. We generally try to avoid merge conflicts rather than produce them intentionally! Branches are really helpful for managing collaborative work on different aspects of the same project.

For example, we can allocate specific work to a specific branch, like a “digitize-trees” branch might be for digitizing trees only and a “identifytrees” branch might be for changing the attribute values for the tree species of the points that are made from the other branch. Then that leaves the “main” branch as the merge point for all the work. In this way, it is possible to collaboratively edit a large dataset with multiple people without causing merge conflicts between branches.

Task 4: Digitize some buildings in ArcGIS Pro

Step 1: Start a new ArcGIS Pro project.

As of the writing of this lab, ArcGIS Pro does not natively support adding a remote data source via a URL like you can do in QGIS. To get around this, we will create a reference to the remote COG by converting it to a virtual raster (VRT) format in QGIS.

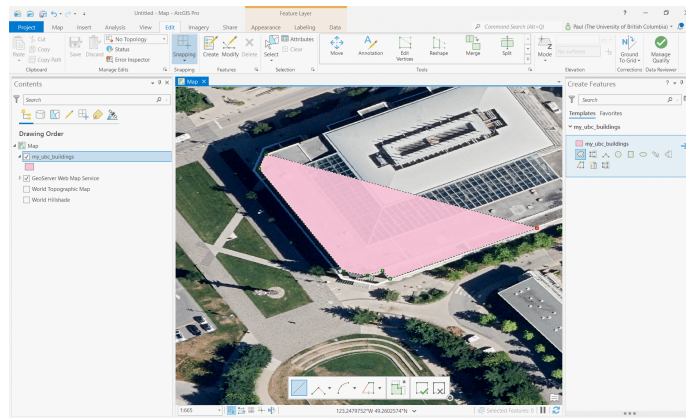
Step 2: In QGIS, select “Raster” from the top menu bar, then “Conversion”, and finally select “Translate (convert format)...”. The input file will be the UBC orthophoto COG that you added to your QGIS project earlier and scroll down to “Converted”, then navigate to a directory where you want to save the VRT on your machine. Your ArcGIS Pro project folder is a sensible choice. Name the output something like “UBC_ORTHOPHOTO_2022.vrt” and ensure that the file extension is VRT. Click “Run” and this tool should complete in a couple seconds.

Step 3: Navigate to the location of the VRT in ArcGIS Pro and add it to your project or just drop and drag it into your project map. It may take a few minutes to calculate statistics, but you can cancel that to immediately display the COG in ArcGIS Pro.

In the following steps, we are going to digitize a building on campus. To do so in ArcGIS Pro, we need to first create a new feature class that will hold the polygon we are going to digitize.

Step 4: In a Catalog window pane, expand “Databases” and then right-click the geodatabase for your project. Select “New” and then “Feature Class”. Name it “my_abc_buildings” and make sure the type is set to “Polygon”. Click “Next” to view fields. Add two fields: “Name” (data type is text) and “Vertices” (data type is double). Click “Next” to view the spatial reference. Probably the default is set to WGS 1984. Under “Projected Coordinate System” find NAD 1983 UTM Zone 10N. You can continue clicking “Next” to view more properties that you can set, but we will leave these as the defaults, so you can click “Finish” when you are done. You should now have an empty polygon feature class in you map.

Step 5: Click the “Edit” tab and click the “Create” button to start creating new features. The “Create Features” pane will open. Click on “my_abc_buildings” to highlight it and start the editing session. Now pick a building and zoom in so that you can clearly see its borders, then start left-clicking to add vertices along the building perimeter. Be sure to work either in a clockwise or counter-clockwise pattern otherwise you will criss-cross the boundary. Once you are happy, you can simply double left-click in the last position to finish the edit or press F2. Ta-da! You have now digitized your first building.



Step 6: At the top ribbon on the “Edit” tab, click “Attributes” to open the attribute editor for the selected feature. Change the name to the name of the building and click “Apply”.

Step 7: Digitize four more buildings on campus and add the building name to the attribute table. Once you are done, be sure to click “Save” on the top ribbon under the “Edit” tab to save your work to the feature class in the geodatabase.

Next, we are going to validate and compare your digitized buildings against the official building dataset that is produced by UBC Campus + Community Planning.

Step 8: Add the official “ubcv_buildings” data to your map from the ubcv database on the PostgreSQL server. For information about connecting to the server, refer to Lab 1. Since these data are read-only from the PostgreSQL server, we need to export the feature class to our local geodatabase so that we can make some calculations. We also need to change the coordinate system from WGS 1984 to our preferred projected coordinate system of NAD 1983 UTM Zone 10N. To do this, we can use a single tool “Project”. Save the output with a name of “ubcv_buildings_utm” to your geodatabase.

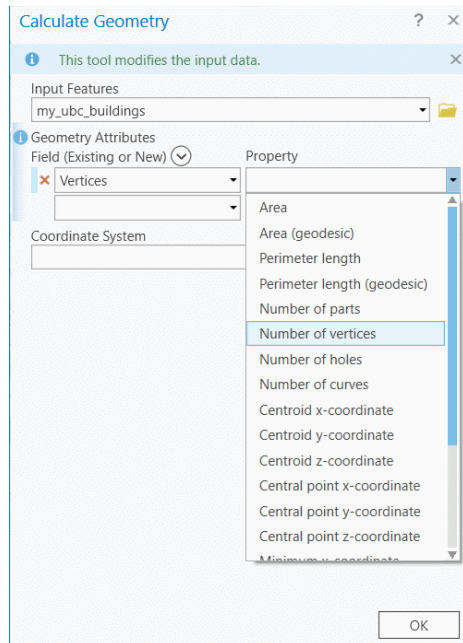
Step 9: Add the local copy of “ubcv_buildings_utm” to your map and compare your digitization to the official record. Make some observations and answer the questions below.

Q5. Describe why your boundary does not align with the official dataset? Give three possible sources of error. (3 points)

Q6. What do you observe about the location of the official building dataset compared with the apparent location of the buildings in the orthophoto? (3 points)

Q7. How could relief displacement have impacted your digitization? (2 points) **Step 10:** Open the attribute table of the “ubcv_buildings_utm”

layer and add a field called “Vertices” with double data type. Save the change then return to the attribute table, find the new field, right-click it, select “Calculate Geometry” and then under the “Property” drop-down, select “Number of vertices” and click “OK”. Repeat this step for “my_abc_buildings”.



Step 11: Inspect the attribute tables of “ubcv_buildings_utm” and “my_abc_buildings” and compare the “Shape_Area”, “Shape_Length”, and “Vertices” fields then answer the questions below.

Q8. Which building that you digitized had the largest deviation of Shape_Area compared with the official buildings dataset? Give the value difference in square meters and discuss why you think this building had the worst area error. (2 points)

Q9. Which building that you digitized had the largest deviation of Shape_Length compared with the official buildings dataset? Give the value difference in square meters and discuss why you think this building had the worst perimeter error. (2 points) Your last deliverable for the lab will be a mapped response to the following question: Did you digitize more or fewer vertices than the official UBC dataset? Discuss the implications of more or fewer vertices on positional accuracy, area accuracy, and perimeter accuracy. Your map should show a single building exemplar and should be annotated with supporting text and other elements (e.g., arrows, text boxes, etc.) to answer the question and illustrate your understanding of the relationship between digitization and accuracy measures.

Summary

By now, you should appreciate that digitization is an imperfect process. There are many sources of errors that can accumulate, especially when you are working on a crowd-sourced or collaborative editing project. It takes time to develop and practice good digitization skills. You might want to take a moment to reflect on the question, “which data set is right?” Invariably, the answer is the one with “authority”.

Return to the **Deliverables** section to check off everything you need to submit for credit in the course management system.

Chapter 7

Geocoding

Written by Amy Blood

Lab Overview

Working with and manipulating your own geospatial data is essential for many geomatics tasks. Often in urban forestry, you will come across datasets with addresses or coordinates. In this lab, you will learn how to import tabular data into ArcGIS Pro, and how to create an address locator and geocode addresses. You will work from data within Vancouver to create several new geospatial datasets and practice geocoding addresses.

Learning Objectives

- Import and convert tabular data to spatial data in ArcGIS Pro
 - Query and extract geospatial data from an online portal
 - Build a custom address locator using geospatial data
 - Geocode addresses and assess quality
-

Deliverables

Answers to the questions posed throughout the lab (20 points)

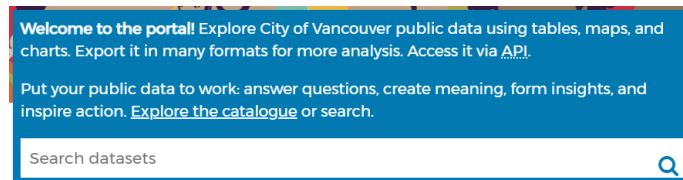
Final map (10 points)

- The map should show the neighbourhood for which you downloaded street tree data for

- Parks symbolized by tree density
 - Coordinate-created points (geocoded) and address-created points symbolized differently from each other
 - Roads symbolized
 - An appropriate basemap
-

Data

You will be downloading some of your data from the City of Vancouver Open Data Portal.



The remaining data for this lab are accessible via the UBC PostgreSQL server. Instructions for connecting to the server and data store are given in the tasks below and prior labs.

Task 1: Download and Prepare Data

Step 1: The first dataset you will download is “Public trees” data. Search for it with the search bar, then click the link for the dataset. You are going to want to export the data as a shapefile; click on the Export tab and look over the geographic file formats. You will notice that the download button for Shapefile is greyed out because there are too many records in the dataset. Keep the webpage open so you have access to the Metadata.

Q1. What is the maximum number of records you can export from the City of Vancouver open data portal as a shapefile? (1 point)

Step 2: Instead of working with the dataset for the entire city, you will focus on one neighbourhood. To the left, there are filtering options. Scroll down to NEIGHBOURHOOD_NAME and choose the neighborhood with the highest number of trees. Now you should have the option to download street tree data. Name the file “neighborhood_trees”.

Step 3: The second dataset you will download is “Parks - Polygon Representation”. There should be a point and a polygon dataset. Make sure you download the polygon dataset.

Task 2: Prepare Data

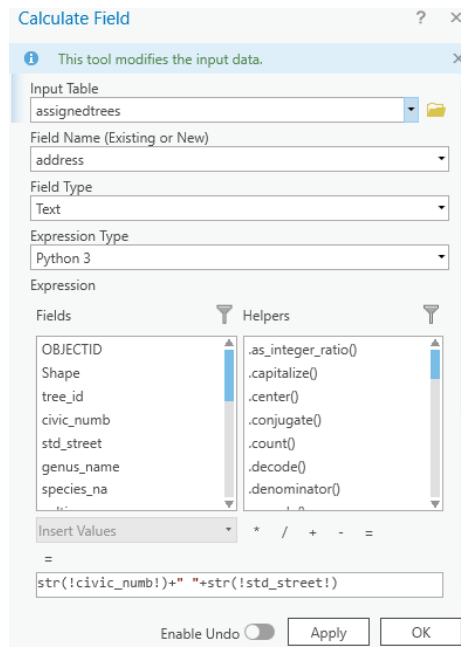
In this task, you will export a filtered table from a shapefile, and then you will add a field so the table can be used in the next task.

Step 1: Open up the metadata for the street trees file. It should be under the “Information” tab on the City of Vancouver data portal website. Look through the metadata and familiarize yourself with what each column means. For example, one column designates when a tree was planted.

Q2. What is the name of the column that designates when a tree was planted? (1 point) **Step 2:** Open the attribute table for “neighborhood_trees” that you downloaded and add a new text field called “Address”. On the top of the attribute table, there is a button that says “Add”. The table will switch to field view. Name the field “Address” and select “Text” as the data type. Click “Save” in the top bar. Close “Field View” and return to the attribute table. Your new field should appear as the very last column in the dataset. Right-click the field and select “Calculate Field.”

The address locator you create in Task 3 requires addresses to be written in a specific way: street numbers and names must be in the same column. You will need to specify to ArcGIS that you are combining two strings (another designation for text data). Within the Fields menu under “Expression,” double-click “civic_num”. You will notice that the field below the equal sign is now populated with !civic_num!.

To combine (i.e., concatenate) the two columns together, type the following code into the box: `str(!civic_num!)+ " "+str(!std_street!)`



Click “Apply”. The Address field should now have street numbers and street names.

Task 3: Geocoding a Table of Addresses

Many localities label their trees by address rather than by coordinates. In the street-trees file, all of the trees have addresses and coordinates. In this section of the lab, you will use a roads network layer for the Greater Vancouver Regional District (GVRD) to create an address locator for Vancouver. Then you will geocode the addresses from your neighborhood to examine the differences between collecting data with addresses versus coordinates. An address locator allows you to look up addresses either manually or by uploading a table of addresses. This is helpful when you are searching for specific trees!

Step 1: Connect to the **geocode** database on the UBC PostgreSQL server and load the “gvrd_roads” layer into your map.

Step 2: Create an address locator, which is located in the Geoprocessing pane by selecting “Analysis”, “Tools”, “Toolboxes”, “Geocoding Tools”, and “Create Locator”. You can also use the search bar to search for the tool by name.

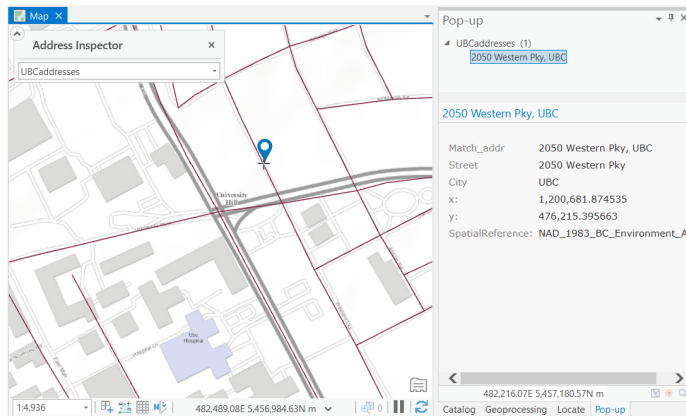
For the tool parameters:

- **Country or Region:** Canada
- **Primary Table(s):** gvrd_roads

- **Role:** Street Address
- **Field Map:**
 - **Feature ID:** ogc_fid
 - **Left House Number From:** from_left_house_number
 - **Left House Number To:** to_left_house_number
 - **Right House Number From:** from_right_house_number
 - **Right House Number To:** to_right_house_number
 - **Street Name:** structured_name_a
 - **Left City/Municipality:** left_loc_1
 - **Right City/Municipality:** right_loc_1
- All other fields leave as <NONE>
- **Language Code:** English

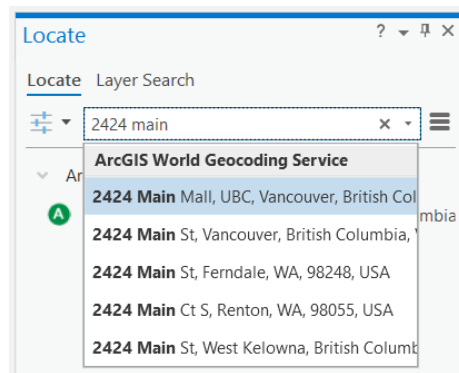
Save the file as “GVRDaddresses”.

Step 3: Select “Locate” underneath the “Map” tab, and then click on “Address Inspector”. Use your newly created address locator to click around Vancouver. You may need to double-click on your map in order to force the pop-up to stay visible and docked as a tab (see right pane below).



Choose “GVRDaddresses” in the dropdown.

You can also use the “Locate” tool to search an address using the ArcGIS World Geocoding Service. Type in the address of Queen Elizabeth Park (4600 Cambie St, Vancouver) and notice the map highlights near the park! The ArcGIS World Geocoding Service will consume credits through your ArcGIS account, so it is not the most cost-effective way to geocode many addresses.



Step 4: Geocode a table of addresses using “GVRDaddresses”. The table of addresses you will be geocoding is “neighborhood_trees” that you exported earlier. To access the geocoding tool, you can search for “Geocode Addresses”.

Earlier, we downloaded the neighborhood trees as a shapefile instead of a table, so we need to convert it to a table for this step. Right-click the layer in ArcGIS Pro and export it to a table.

Use the following tool parameters:

- **Input Table:** neighborhood_trees
- **Input Address Locator:** GVRDaddresses
- **Input Address Fields:** Single Field
- **Single Line Input:** Address

Leave the other default options and save the file as “geocoded”.

Step 5: Look at the attribute table. The score column shows from 0-100 based on how close the address in the address locator was able to match the input table. The mailbox icon with the arrows in the geocoding toolbar allows you to investigate the matches more in-depth. With a messy dataset, this becomes invaluable.

Q3. Are all of the addresses a 100% match? Why or why not? (2 points) **Step 6:** Toggle the “neighborhood_trees” and “geocoded” datasets on and off to look for similarities and differences.

Q4. How do they visually compare to each other? Use specific examples in your explanation and make inferences about why you are seeing differences/similarities. (4 points)

Task 4: Create a shapefile from tabular data

Oftentimes, you will encounter geospatial data in Comma Separated Format (CSV) or tabular format. For example, if you collect field data and enter it into a spreadsheet. Although each value in the dataset may have geographic or projected coordinates, the data may not be immediately accessible in a spatial data format. In this portion of the lab, you will practice converting tabular data into shapefiles. Then, you will work with the shapefiles you have created.

Step 1: Connect to the **geocode** database on the UBC PostgreSQL server and load the “parktrees” table into your map. There should be six columns and many rows. Each row represents a tree in a Vancouver Park.

Q5. How many trees are represented in this file? (1 point)

Q6. Look at the columns with the coordinates. Which units are the coordinates likely in? (1 point)

Q7. Does ArcGIS Pro consider these data to be spatial? (1 point)

Step 2: Right-click on the “parktrees” table in your Contents Pane and then select “Create Points From Table, then Make XY Event Layer”. Choose the appropriate column names for the X and Y fields. Leave the Z field blank. This is very important because if you choose the wrong columns the points will be nowhere near Vancouver! Specify the coordinate system by clicking on the Globe icon to the right-hand side of Coordinate System and locating NAD 1983 UTM Zone 10N under Projected Coordinate System, UTM, and NAD 1983.

Q8. What does the z field represent? (1 point)

Q9. Which column did you choose for x, which column for y? (2 points) You have now created an “Event Layer.” Export the Events layer as a shapefile (right-click the layer, select “Data”, and then select “Export Features”). Save it in your output folder as “parktrees”. Add “parktrees” to your display.

Event layers are temporary layers, and without Object-ID fields they are not editable or selectable.

Step 3: View “parktrees” in ArcGIS Pro. The trees should all be in Vancouver. If they are not, something was input incorrectly and you will have to repeat Step 2. Once the tree points are properly mapped, change the symbology. Right-click “parktrees” in the Contents Pane and choose “Symbology.” A Symbology pop-up should appear. Click the symbol and choose “Circle 3” from Gallery. Click the Properties Tab. Make the symbol smaller and change its colour.

Step 4: Add “parks_polygon_representation” and a satellite basemap to your map (from the “Map” tab, select “Basemap”, and then select “Imagery”). In the Contents Pane, drag the basemap to the bottom so that it does not cover

anything. Symbolize “parks_polygon_representation” so that you can clearly see the basemap for each park. You can do this in the Contents Pane by right-clicking the coloured rectangle beneath “parks_polygon_representation” and choosing “Color Properties...” at the bottom. Adjust the colours and transparency as you see fit.

Q10. What sort of discrepancies do you see between the number of trees in the satellite image versus the number of trees shown in the dataset? Do any specific parks stand out? Why might this be the case? (4 points) Now that all of the park trees are input, we will investigate tree density per park to get an idea about which parks in Vancouver have the highest tree density and which have the lowest. To answer this question, you will need to know the area of each park and the number of trees within them. Park area and name are in the “parks_polygon_representation” layer, but not in the “parktrees” shapefile. This information can be added to “parktrees” through a spatial join.

Step 5: Find the “Spatial Join” tool and use these parameters

- **Target Features:** parks_polygon_representation
- **Join Features:** parktrees
- **Output Feature Class:** parks_numtrees
- **Join Operation:** Join one to one
- **Match Option:** Intersect

Expand the Fields option and remove all of the output fields except for area_ha, park_id, park_name, and park_url. Click “OK”.

You should now have a park file with the number of trees (represented by JOIN_COUNT), along with the other park information. Add a new field called “treedensity”. Be sure to select the correct data type—the data in this field will be continuous values and include decimals. Tree density is the ratio of trees to park area. We will do this calculation only for parks greater than or equal to one acre in size.

Step 6: Using “Select By Attributes,” select all of the parks with an area greater than or equal to 1 ha. Then, use calculate field to get the tree density for each selected park.

Q11. How many parks in Vancouver are greater than 1 ha in size? (1 point)

Q12. From the selected data, which park has the highest tree density? (1 point) **Step 7:** Change the symbology of “parks_numtrees” from single symbol to unclassed colors displaying the tree density field.

Summary

Spatial is special, but not all data are explicitly spatial. Some data may contain attributes or information that is encoded in a way that GIS software may not immediately recognize as spatial. As well, a lot of information can be associated with addresses, which are not related to any coordinate system. Geocoding is a powerful method that can unlock information associated with addresses for use in other spatial analyses.

Return to the **Deliverables** section to check off everything you need to submit for credit in the course management system.